

este crecimiento exponencial del tiempo de cálculos cuando el tamaño del problema aumentaba. Muchos problemas importantes que surgían en la práctica no se podían resolver.

Más tarde llegó el siguiente cambio, a mediados de la década de 1980, con la introducción del *enfoque de ramificación y corte* para resolver problemas de PEB. Hubo reportes iniciales de problemas muy grandes con hasta 2 000 variables que se pudieron resolver mediante el uso de esta herramienta. Este avance creó un gran interés y condujo a una extensa investigación y actividades de desarrollo para perfeccionar el enfoque que se había utilizado desde entonces. Al principio, el enfoque se limitó a PEB *pura*, pero después se extendió a PEB *mixta*, y con posterioridad a problemas de PEM con algunas variables enteras generales. La descripción que se presentará aquí se limita al enfoque del caso de la PEB *pura*.

En la actualidad es bastante común utilizar el enfoque de ramificación y corte para resolver problemas con unos cuantos miles de variables, y en ocasiones hasta cientos de miles de variables. Como se mencionó en la sección 11.4, esta impresionante capacidad se debe al enorme progreso en tres áreas —mejoras sorprendentes en los algoritmos BIP luego de la incorporación y desarrollo del método de ramificar y cortar, mejoras significativas en los algoritmos de programación lineal que se utilizan ampliamente dentro de los algoritmos BIP y la gran velocidad a la que operan las computadoras (entre ellas, las de escritorio).

Es necesario agregar una nota de precaución. Este enfoque algorítmico no puede resolver siempre *todos* los problemas de PEB *pura* con unos cuantos miles, o cientos, de variables. Los problemas muy grandes de PEB *pura* que se ha resuelto tenían matrices *A* *ralas*, es decir, el porcentaje de coeficientes en las restricciones funcionales *distintos de cero* era bastante pequeño (quizá menos de 5%, o incluso menos de 1%). En realidad, en gran medida el enfoque depende de lo ralo. (Por fortuna, las formas ralas son frecuentes en los problemas prácticos.) Más aún, existen otros factores importantes, además de lo ralo y el tamaño, que incrementan o reducen la dificultad de la solución de ciertos problemas de programación entera. Por ello, debe tenerse mucho cuidado al desarrollar las formulaciones de PE de tamaño bastante grande.

Aunque está más allá del alcance de este libro describir con detalle el nuevo enfoque algorítmico, se dará una idea global breve. Como esta idea global se limita a la PEB *pura*, *todas* las variables que se introducen en adelante en esta sección son variables *binarias*.

Este enfoque utiliza en forma principal una combinación de los tres tipos de técnicas:⁹ el *preprocesado automático del problema*, la *generación de planos cortantes* o *cortaduras* y las ingeniosas técnicas de *ramificación y acotamiento*. Usted está familiarizado con las técnicas de ramificación y acotamiento y no se profundizará más en las versiones más elaboradas que se incorporan aquí. A continuación se presenta una introducción conceptual sobre los otros dos tipos de técnicas que se utilizan.

Preprocesado automático del problema de PEB pura

El preprocesado automático del problema incluye una “inspección por computadora” de la formulación del problema de PE que proporciona el usuario, con el fin de detectar algunas reformulaciones que permiten resolver el problema con más rapidez, sin eliminar ninguna solución factible. Estas reformulaciones se pueden clasificar en tres categorías:

1. *Fijación de variables*: Identificar las variables que pueden fijarse en uno de sus dos valores posibles (0 o 1) debido a que el otro valor no puede ser parte de una solución que sea tanto factible como óptima.
2. *Eliminación de restricciones redundantes*: Identificar y eliminar *restricciones redundantes* (restricciones que son satisfechas en forma automática cuando las soluciones satisfacen todas las demás).
3. *Estrechamiento de restricciones*: Estrechar algunas restricciones para reducir la región factible del relajamiento de PL sin eliminar ninguna solución factible del problema de PEB.

Estas categorías se describirán más adelante.

⁹ Como se explicó de manera breve en la sección 11.4, existe una técnica más que ha jugado un papel significativo en el progreso reciente, que es el uso de la *heurística* para encontrar soluciones factibles buenas con rapidez.

Fijación de variables. El principio general para fijar variables es el siguiente.

Si el valor de una variable no puede satisfacer cierta restricción, aun cuando las otras variables sean iguales a sus mejores valores que tratan de satisfacer la restricción, entonces esa variable debe fijarse en el otro valor.

Por ejemplo, *cada* una de las siguientes restricciones de tipo $\leq$ permitiría fijar $x_1 = 0$, puesto que $x_1 = 1$ con los mejores valores de las otras variables (0 con un coeficiente no negativo y 1 con un coeficiente negativo) violaría la restricción.

$$\begin{aligned} 3x_1 &\leq 2 &\Rightarrow x_1 &= 0, &\text{puesto que } 3(1) &> 2. \\ 3x_1 + x_2 &\leq 2 &\Rightarrow x_1 &= 0, &\text{puesto que } 3(1) + 1(0) &> 2. \\ 5x_1 + x_2 - 2x_3 &\leq 2 &\Rightarrow x_1 &= 0, &\text{puesto que } 5(1) + 1(0) - 2(1) &> 2. \end{aligned}$$

El procedimiento general para verificar cualquier restricción de tipo $\leq$ es identificar la variable con el *coeficiente positivo mayor*, y si la *suma de ese coeficiente* y cualesquiera *coeficientes negativos* excede la cantidad del lado derecho, entonces esa variable debe fijarse en 0. (Una vez que se fija la variable el procedimiento se puede repetir para la variable con el siguiente coeficiente positivo más grande, y así sucesivamente.)

Un procedimiento análogo con restricciones de tipo $\geq$ permite fijar una variable en 1 como se ilustra tres veces en seguida.

$$\begin{aligned} 3x_1 &\geq 2 &\Rightarrow x_1 &= 1, &\text{puesto que } 3(0) &< 2. \\ 3x_1 + x_2 &\geq 2 &\Rightarrow x_1 &= 1, &\text{puesto que } 3(0) + 1(1) &< 2. \\ 3x_1 + x_2 - 2x_3 &\geq 2 &\Rightarrow x_1 &= 1, &\text{puesto que } 3(0) + 1(1) - 2(0) &< 2. \end{aligned}$$

Una restricción de tipo $\geq$ también puede permitir que se fije una variable en 0, como se ilustra a continuación.

$$x_1 + x_2 - 2x_3 \geq 1 \quad \Rightarrow \quad x_3 = 0, \quad \text{puesto que } 1(1) + 1(1) - 2(1) < 1.$$

El siguiente ejemplo muestra una restricción de tipo $\geq$ en la que se fija una variable en 1 y otra en 0.

$$\begin{aligned} 3x_1 + x_2 - 3x_3 &\geq 2 &\Rightarrow x_1 &= 1, &\text{puesto que } 3(0) + 1(1) - 3(0) &< 2 \\ y & &\Rightarrow x_3 &= 0, &\text{puesto que } 3(1) + 1(1) - 3(1) &< 2. \end{aligned}$$

De igual manera, una restricción de tipo $\leq$ con un valor *negativo* en el lado derecho puede dar como resultado que 0 o 1 se conviertan en el valor fijo de una variable. Por ejemplo, ocurren ambas cosas con la siguiente restricción:

$$\begin{aligned} 3x_1 - 2x_2 &\leq -1 &\Rightarrow x_1 &= 0, &\text{puesto que } 3(1) - 2(1) &> -1 \\ y & &\Rightarrow x_2 &= 1, &\text{puesto que } 3(0) - 2(0) &> -1. \end{aligned}$$

El hecho de fijar una variable a partir de una restricción puede llegar a generar una reacción en cadena que permita después fijar otras variables a partir de otras restricciones. Por ejemplo, vea lo que ocurre con las tres restricciones siguientes.

$$3x_1 + x_2 - 2x_3 \geq 2 \quad \Rightarrow \quad x_1 = 1 \quad (\text{como antes}).$$

Entonces

$$x_1 + x_4 + x_5 \leq 1 \quad \Rightarrow \quad x_4 = 0, \quad x_5 = 0.$$

En consecuencia,

$$-x_5 + x_6 \leq 0 \quad \Rightarrow \quad x_6 = 0.$$

En algunos casos es posible combinar una o más restricciones de *alternativas mutuamente excluyentes* con otra restricción para fijar una variable, como se verá en seguida:

$$\begin{aligned} 8x_1 - 4x_2 - 5x_3 + 3x_4 &\leq 2 \\ x_2 + x_3 &\leq 1 \end{aligned} \quad \Rightarrow \quad x_1 = 0, \quad \text{puesto que } 8(1) - \max\{4, 5\}(1) + 3(0) > 2.$$

Existen otras técnicas para fijar variables, incluso algunas que toman en cuenta ciertos factores de optimalidad, pero no se profundizará más en este tema.

La fijación de variables puede tener un efecto muy marcado en la reducción del tamaño de un problema. No es inusual eliminar hasta la mitad de las variables del problema para su consideración posterior.

Eliminación de restricciones redundantes. A continuación se presenta una manera sencilla de detectar una restricción redundante.

Si una restricción funcional satisface incluso la solución binaria de mayor reto, entonces las restricciones binarias las han convertido en redundantes y en adelante pueden eliminarse. En el caso de una restricción de tipo $\leq$ la solución binaria más demandante tiene variables iguales a 1 cuando tiene coeficientes no negativos y otras variables son iguales a 0. (Estos valores se invierten si se trata de una restricción de tipo $\geq$.)

A continuación se presentan algunos ejemplos.

$$\begin{array}{ll} 3x_1 + 2x_2 \leq 6 & \text{es redundante porque } 3(1) + 2(1) \leq 6. \\ 3x_1 - 2x_2 \leq 3 & \text{es redundante porque } 3(1) - 2(0) \leq 3. \\ 3x_1 - 2x_2 \geq -3 & \text{es redundante porque } 3(0) - 2(1) \geq -3. \end{array}$$

En la mayoría de los casos, cuando se identifica una restricción como redundante, en el modelo original no era redundante sino que se convirtió después de fijar algunas variables. De los 11 ejemplos que se dieron para fijar variables, *todos* menos el último dejaron una restricción que en ese momento se convirtió en redundante.

Estrechamiento de restricciones.¹⁰ Considere el siguiente problema.

$$\text{Maximizar} \quad Z = 3x_1 + 2x_2,$$

sujeta a

$$2x_1 + 3x_2 \leq 4$$

y

$$x_1, x_2 \text{ binaria.}$$

Este problema de PEB tiene sólo tres soluciones factibles —(0, 0), (1, 0) y (0, 1)— de las cuales la solución óptima es (1, 0) con $Z = 3$. La región factible del relajamiento de PL de este problema se muestra en la figura 11.14. La solución óptima de este relajamiento de PL es $(1, \frac{2}{3})$ con $Z = 4\frac{1}{3}$, que no está muy cerca de la solución óptima del problema de PEB. Un algoritmo de ramificación y acotamiento tendrá que trabajar bastante para identificar la solución óptima.

Ahora observe lo que ocurre si la restricción funcional $2x_1 + 3x_2 \leq 4$ se sustituye por

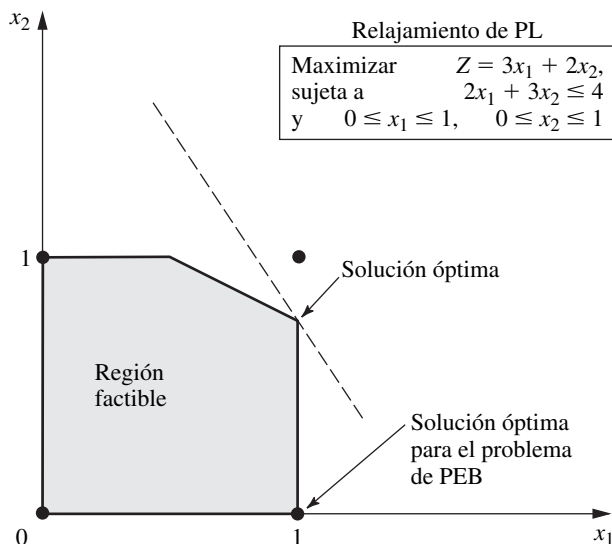
$$x_1 + x_2 \leq 1.$$

Las soluciones factibles para el problema de PEB son exactamente las mismas —(0, 0), (1, 0) y (0, 1)— de manera que la solución óptima todavía es (1, 0). Sin embargo, la región factible para el relajamiento de PL se ha reducido de manera considerable, como se muestra en la figura 11.15. De hecho, esta región factible se ha reducido tanto que la solución óptima del relajamiento de PL es ahora (1, 0), de manera que se ha encontrado la solución óptima del problema de PEB sin necesidad de trabajo adicional.

Éste es un ejemplo de cómo se estrecha una restricción de manera que se reduzca la región factible del relajamiento de PL sin eliminar ninguna de las soluciones factibles del problema de PEB. Fue sencillo realizarlo para este problema pequeño de dos variables que se podía dibujar en una gráfica. No obstante, con la aplicación de los mismos principios para estrechar restricciones sin eliminar ninguna de las soluciones factibles de PEB, se puede usar el siguiente procedimiento algebraico para cualquier restricción de tipo $\leq$ con cualquier número de variables.

¹⁰ También se conoce comúnmente como *reducción de coeficientes*.

■ **FIGURA 11.14**
Relajamiento de PL
(incluye su región factible
y la solución óptima) del
ejemplo de PEB usado para
ilustrar cómo estrechar una
restricción.



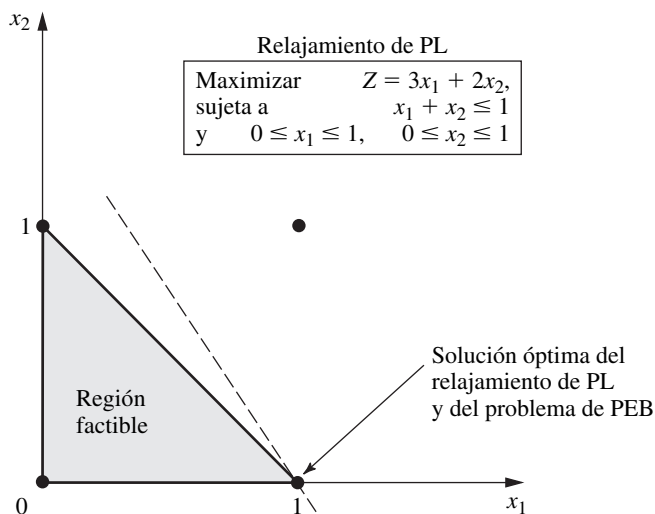
Procedimiento para estrechar o reducir una restricción de tipo $\leq$
Denote la restricción por $a_1x_1 + a_2x_2 + \dots + a_nx_n \leq b$.

1. Calcule S = suma de los a_j positivos.
2. Identifique cualquier $a_j \neq 0$ tal que $S < b + |a_j|$.
 - a) Si no existen, el procedimiento se detiene; la restricción ya no se puede reducir.
 - b) Si $a_j > 0$, vaya al paso 3.
 - c) Si $a_j < 0$, vaya al paso 4.
3. ($a_j > 0$) Calcule $\bar{a}_j = S - b$ y $\bar{b} = S - a_j$. Restablezca $a_j = \bar{a}_j$ y $b = \bar{b}$. Regrese al paso 1.
4. ($a_j < 0$) Aumente a_j a $a_j = b - S$. Regrese al paso 1.

La aplicación de este procedimiento a la restricción funcional del ejemplo anterior fluye de la siguiente manera:

La restricción es $2x_1 + 3x_2 \leq 4$ ($a_1 = 2, a_2 = 3, b = 4$).

■ **FIGURA 11.15**
Relajamiento de PL
después de estrechar la
restricción $2x_1 + 3x_2 \leq 4$, a
 $x_1 + x_2 \leq 1$ del ejemplo de
la figura 11.14.



1. $S = 2 + 3 = 5$.
2. a_1 satisface $S < b + |a_1|$, puesto que $5 < 4 + 2$. Además a_2 satisface $S < b + |a_2|$, puesto que $5 < 4 + 3$. Seleccione a_1 en forma arbitraria.
3. $\bar{a}_1 = 5 - 4 = 1$ y $\bar{b} = 5 - 2 = 3$, se restablece $a_1 = 1$ y $b = 3$. La nueva restricción reducida es

$$x_1 + 3x_2 \leq 3 \quad (a_1 = 1, a_2 = 3, b = 3).$$

1. $S = 1 + 3 = 4$.
2. a_2 satisface $S < b + |a_2|$, puesto que $4 < 3 + 3$.
3. $\bar{a}_2 = 4 - 3 = 1$ y $\bar{b} = 4 - 3 = 1$, se restablece $a_2 = 1$ y $b = 1$. La nueva restricción reducida o estrecha es

$$x_1 + x_2 \leq 1 \quad (a_1 = 1, a_2 = 1, b = 1).$$

1. $S = 1 + 1 = 2$.
2. Ninguna $a_j \neq 0$ satisface $S < b + |a_j|$, por lo cual el procedimiento se detiene; $x_1 + x_2 \leq 1$ es la restricción reducida que se busca.

Si al ejecutar el paso 2 la primera vez, en el ejemplo anterior, se hubiera seleccionado a_2 , entonces la primera restricción reducida habría sido $2x_1 + x_2 \leq 2$. La siguiente serie de pasos de nuevo habrían conducido a $x_1 + x_2 \leq 1$.

El siguiente es otro ejemplo en el que el procedimiento estrecha la restricción de la izquierda y la convierte en la de la derecha, y después reduce más a la segunda de la derecha.

$$\begin{aligned} 4x_1 - 3x_2 + x_3 + 2x_4 &\leq 5 &\Rightarrow & 2x_1 - 3x_2 + x_3 + 2x_4 \leq 3 \\ &&\Rightarrow & 2x_1 - 2x_2 + x_3 + 2x_4 \leq 3. \end{aligned}$$

(El problema 11.8-5 pide que se aplique el procedimiento para confirmar estos resultados.)

Una restricción de la forma $\geq$ se puede convertir a la forma $\leq$ (al multiplicar ambos lados por -1) para aplicar este procedimiento de manera directa.

Generación de planos cortantes para PEB pura

Un **plano cortante** (o **corte**) de un problema de PE es una nueva restricción funcional que reduce la región factible del relajamiento de PL sin eliminar soluciones factibles del problema de PE original. De hecho, se acaba de estudiar una manera de generar planos cortantes en problemas de PE pura, esto es, la aplicación del procedimiento anterior para estrechar restricciones. De esta forma, $x_1 + x_2 \leq 1$ es un plano cortante del problema PEB que se presenta en la figura 11.14, que conduce a la región factible reducida del relajamiento de PL que se muestra en la figura 11.15.

Además de este procedimiento, se ha desarrollado un buen número de técnicas para generar planos cortantes que tenderán a acelerar la rapidez con la que el método de ramificación y acotamiento puede encontrar una solución óptima a un problema de PEB pura. Se centrará la atención nada más en una de estas técnicas.

Para ilustrar esta técnica, considere el problema de PE pura de la California Manufacturing Co., que se presentó en la sección 11.1 y que se utilizó para ilustrar el algoritmo de ramificación y acotamiento de PEB en la sección 11.6. La solución óptima de su relajamiento de PL se presenta en la figura 11.5 como $(x_1, x_2, x_3, x_4) = (\frac{5}{6}, 1, 0, 1)$. Una de las restricciones funcionales es

$$6x_1 + 3x_2 + 5x_3 + 2x_4 \leq 10.$$

Ahora observe que, juntas, las restricciones binarias y esta restricción implican que

$$x_1 + x_2 + x_4 \leq 2.$$

Esta nueva restricción es un *plano cortante*. Elimina parte de la región factible del relajamiento de PL, incluso la que llegó a ser la solución óptima, $(\frac{5}{6}, 1, 0, 1)$, pero no elimina soluciones *enteras* factibles. Cuando se agrega este corte al modelo original, mejora el desempeño del algoritmo de ramificación y acotamiento de PEB de la sección 11.6 (vea la figura 11.9) en dos formas. Primero, la solución óptima del nuevo (y más estrecho) relajamiento

to de PL sería $(1, 1, \frac{1}{5}, 0)$, con $Z = 15\frac{1}{5}$, de manera que las cotas del nodo de *Todo*, el nodo $x_1 = 1$ y el nodo $(x_1, x_2) = (1, 1)$, serían ahora de 15 en lugar de 16. Segundo, se necesitaría una iteración menos, porque la solución óptima del relajamiento de PL en el nodo $(x_1, x_2, x_3) = (1, 1, 0)$ ahora sería $(1, 1, 0, 0)$, que proporciona una nueva *solución de apoyo* con $Z^* = 14$. Por tanto, en la *tercera* iteración (vea la figura 11.8) se sondearía este nodo por la prueba 3 y el nodo $(x_1, x_2) = (1, 0)$ se sondearía por la prueba 1, lo cual indicaría que esta nueva solución de apoyo es la solución óptima del problema de PEB original.

El siguiente es el procedimiento general que se utiliza para generar este corte.

Procedimiento para generar planos cortantes

1. Considere cualquier restricción funcional de la forma $\leq$ con sólo coeficientes no negativos.
2. Encuentre un grupo de variables (llamado **cubierta mínima** de la restricción) tales que
 - a) La restricción se viola si todas las variables del grupo son iguales a 1 y todas las demás variables son iguales a 0.
 - b) Pero la restricción se satisface si el valor de *cualquiera* de estas variables cambia de 1 a 0.
3. Si N denota el número de variables del grupo, el plano cortante obtenido tiene la forma

Suma de variables del grupo $\leq N - 1$.

Al aplicar este procedimiento a la restricción $6x_1 + 3x_2 + 5x_3 + 2x_4 \leq 10$, se ve que el grupo de variables $\{x_1, x_2, x_4\}$ es una *cubierta mínima* porque

- a) $(1, 1, 0, 1)$ viola la restricción.
- b) Pero la restricción se satisface si el valor de *cualquiera* de estas tres variables cambia de 1 a 0.

Como en este caso $N = 3$, el plano cortante que se obtiene es $x_1 + x_2 + x_4 \leq 2$.

Esta misma restricción tiene también una segunda cubierta mínima $\{x_1, x_3\}$, puesto que $(1, 0, 1, 0)$ viola la restricción, pero tanto $(0, 0, 1, 0)$ como $(1, 0, 0, 0)$ la satisfacen. Por tanto, $x_1 + x_3 \leq 1$ es otro corte válido.

El enfoque de ramificación y cortadura implica la generación de *muchas* cortaduras en forma parecida antes de aplicar las ingeniosas técnicas de ramificación y acotamiento. Los resultados que se obtienen cuando se incluyen cortes pueden ser importantes para estrechar los relajamientos de PL. En algunos casos la *distancia* entre el valor de Z para la solución óptima del relajamiento de PL del problema completo de PEB y el valor de Z para la solución óptima del problema se reduce hasta en 98%.

Irónicamente, los primeros algoritmos que se desarrollaron para programación entera, incluso el celebrado algoritmo que publicó Ralph Gomory en 1958, estaban basados en cortes (generados de otra manera), pero este enfoque resultó ser poco satisfactorio en la práctica (excepto para algunos tipos especiales de problemas). No obstante, estos algoritmos se apoyaban sólo en los cortes. Ahora se sabe que una *combinación* acertada de los cortes y las técnicas de ramificación y acotamiento (junto con el preprocesado automático del problema) proporciona un enfoque algorítmico poderoso para resolver problemas de PEB a gran escala. Ésta es una razón por la que se ha dado el nombre de algoritmo de *ramificación y corte* a este nuevo enfoque.

11.9 INCORPORACIÓN DE LA PROGRAMACIÓN DE RESTRICCIONES

Ninguna presentación de las ideas básicas de programación entera está completa en estos días si no se le introduce un interesante desarrollo reciente —la incorporación de las técnicas de *programación de restricciones*— que promete ampliar en gran medida las capacidades de formular y resolver modelos de programación entera. (Estas mismas técnicas se comienzan a utilizar en áreas relacionadas de programación matemática, en especial optimización combinatoria, pero esta presentación se limitará a su uso central en programación entera.)

Naturaleza de la programación de restricciones

A mediados de la década de 1980, investigadores de la comunidad de ciencias de la computación desarrollaron la programación de restricciones mediante la combinación de ideas de inteligencia

artificial con el desarrollo de lenguajes de programación de computadoras. La meta era contar con un sistema de programación de computadoras flexible que incluyera tanto *variables* como *restricciones* en sus valores, y que al mismo tiempo permitiera la descripción de procedimientos de búsqueda que generara valores factibles de las variables. Cada variable tiene un *dominio* de valores posibles; por ejemplo, $\{2, 4, 6, 8, 10\}$. En lugar de estar limitadas a los tipos de restricciones matemáticas que se utilizan en programación matemática, existe una gran flexibilidad en la manera de establecer las restricciones. En particular, las restricciones pueden ser de alguno de los siguientes tipos.

1. Restricciones matemáticas; por ejemplo, $x + y < z$.
2. Restricciones disyuntivas; por ejemplo, los tiempos de ciertas tareas en el problema que se modela no se pueden sobreponer.
3. Restricciones relacionales; por ejemplo, al menos tres tareas se deben asignar a cierta máquina.
4. Restricciones explícitas; por ejemplo, aunque x y y tengan dominios $\{1, 2, 3, 4, 5\}$, (x, y) debe ser $(1, 1)$, $(2, 3)$ o $(4, 5)$.
5. Restricciones unitarias; por ejemplo, z es un entero entre 5 y 10.
6. Restricciones lógicas; por ejemplo, si x es 5, entonces y está entre 6 y 8.

Cuando se expresan estos tipos de restricciones, la programación de restricciones permite el uso de varias funciones lógicas normales, como IF, AND, OR, NOT, etc. Excel incluye muchas de las mismas funciones lógicas. Ahora, LINGO cuenta con todas las funciones lógicas normales y se puede usar su optimizador global para encontrar una solución óptima global.

Para ilustrar los algoritmos que utiliza la programación de restricciones para generar soluciones factibles, suponga que un problema tiene cuatro variables — x_1, x_2, x_3, x_4 — y sus dominios son

$$x_1 \in \{1, 2\}, x_2 \in \{1, 2\}, x_3 \in \{1, 2, 3\}, x_4 \in \{1, 2, 3, 4, 5\},$$

donde el símbolo $\in$ significa que la variable de la izquierda pertenece al conjunto de la derecha. Suponga también que las restricciones son

- (1) Todas estas variables deben tener valores diferentes,
- (2) $x_1 + x_3 = 4$.

Por lógica directa, dado que los valores de 1 y 2 deben estar reservados para x_1 y x_2 , la primera restricción implica de inmediato que $x_3 \in \{3\}$, lo cual a su vez implica que $x_4 \in \{4, 5\}$. (Este proceso de eliminar valores posibles para variables se conoce como *dominio de reducción*.) Después, como se ha cambiado el dominio de x_3 , el proceso de *propagación de la restricción* se aplica a la segunda restricción para implicar que $x_1 \in \{1\}$. De nueva cuenta, esto dispara la primera restricción, de manera que por medio de

$$x_1 \in \{1\}, x_2 \in \{2\}, x_3 \in \{3\}, x_4 \in \{4, 5\}$$

se presenta la lista de las únicas soluciones factibles del problema. Este tipo de *razonamiento de factibilidad* basado en alternar la aplicación de los algoritmos de reducción del dominio y de propagación de la restricción es una parte crucial de la programación de restricciones.

Después de la aplicación de los algoritmos de propagación de la restricción y reducción del dominio a un problema, se utiliza un procedimiento de búsqueda para encontrar soluciones factibles. En el ejemplo anterior, como los dominios de todas las variables se han reducido a un solo valor excepto para x_4 , el procedimiento de búsqueda simplemente podría probar los valores $x_4 = 4$ y $x_4 = 5$ para determinar las soluciones factibles completas de ese problema. Sin embargo, por lo general, en el caso de un problema con muchas restricciones y variables, los algoritmos de propagación de la restricción y reducción del dominio no reducen el dominio de cada variable a un solo valor. Por tanto, es necesario escribir un procedimiento de búsqueda que probará diferentes asignaciones de valores para las variables. A medida que se prueban estas asignaciones, se dispara el algoritmo de propagación de la restricción y entonces ocurre la reducción del dominio. El proceso crea un *árbol de búsqueda*, el cual es similar al árbol de ramificación y acotamiento de programación entera.

El proceso global de aplicar la programación de restricciones a complicados problemas de PE (o problemas relacionados) implica el seguimiento de los siguientes tres pasos.

1. Formular un modelo compacto del problema mediante el uso de una variedad de tipos de restricción (la mayoría de los cuales no cumple con el formato de programación entera).
2. Encontrar soluciones factibles de manera eficiente para satisfacer todas las restricciones.
3. Buscar una solución óptima entre estas soluciones factibles.

El poder de la programación de restricciones reside en su gran capacidad para realizar los primeros dos pasos, mientras que la fortaleza principal de la programación entera y sus algoritmos consiste en la realización del tercer paso. De esta forma, idealmente, la programación de restricciones está diseñada para manejar problemas con muchas restricciones que no tienen función objetivo, de forma que la única meta es encontrar una solución factible. Sin embargo, también se puede extender hacia el tercer paso. Un método para hacerlo es *enumerar* las soluciones factibles y calcular el valor de la función objetivo de cada una. No obstante, este enfoque sería muy ineficiente para enfrentar problemas donde existen muchas soluciones factibles. Para contrarrestar esta desventaja, el enfoque común es agregar una restricción que acote en gran medida la función objetivo hasta valores que estén muy cerca de lo que se anticipa como una solución óptima. Por ejemplo, si el objetivo es *maximizar* la función objetivo y se anticipa que el valor Z aproximado de una solución óptima es $Z = 10$, se podría agregar la restricción de que $Z \geq 9$ de manera que las únicas soluciones factibles que restan por ser enumeradas sean aquellas que estén muy cerca de ser óptimas. Después, cada vez que se encuentre una nueva mejor solución durante la búsqueda, la cota sobre Z se puede estrechar aún más para considerar sólo las soluciones factibles que sean al menos tan buenas como la mejor solución actual.

Aunque éste es un enfoque razonable para el tercer paso, uno más atractivo podría ser integrar programación de restricciones y programación entera para que cada una se utilice de manera primordial en los pasos donde es más fuerte: pasos 1 y 2 con programación de restricciones y paso 3 con programación entera. Esta posibilidad es una parte del potencial de la programación de restricciones que se describe a continuación.

Potencial de la programación de restricciones

En la década de 1990, algunas características de la programación de restricciones, entre ellas poderosos algoritmos de solución de restricciones, se incorporaron de manera exitosa a varios lenguajes de programación de propósito general. Este avance condujo a la ciencia computacional cada vez más cerca del Santo Grial de la programación de computadoras; es decir, permitirle al usuario establecer el problema de manera simple para que después la computadora lo resolviera.

Cuando la noticia de este interesante desarrollo comenzó a esparcirse fuera de la comunidad de la ciencia computacional, los estudiosos de la investigación de operaciones comenzaron a darse cuenta del enorme potencial de integrar la programación de restricciones con las técnicas tradicionales de programación entera (así como de otras áreas de programación matemática). La flexibilidad mucho mayor para expresar las restricciones del problema aumentaría en gran medida la capacidad de formular modelos válidos para solucionar problemas complejos. También podría conducir a formulaciones mucho más compactas y directas. Además, al reducir el tamaño de la región factible que debe considerarse cuando se pretende encontrar soluciones factibles de manera eficiente dentro de esta región, los algoritmos de solución de restricciones dentro de la programación de restricciones podrían ayudar a acelerar el progreso de los algoritmos de programación entera para encontrar una solución óptima.

Debido a sus diferencias sustanciales, integrar la programación de restricciones con la programación entera es una tarea muy difícil. En razón de que esta última no reconoce la mayoría de las restricciones de la programación de restricciones, se requiere el desarrollo de procedimientos implantados por computadora para traducir el lenguaje de la programación entera al de programación de restricciones y viceversa. Se han hecho grandes progresos, pero sin duda esta cuestión seguirá vigente como una de las áreas de investigación de IO más activas durante los años por venir.

Para ilustrar la manera en que la programación de restricciones puede simplificar en gran medida la formulación de modelos de programación entera, a continuación se presentarán dos de las “restricciones globales” más importantes de la programación de restricciones. Una **restricción**

global es una restricción que expresa en forma sucinta un patrón global de la relación permisible entre múltiples variables. Por tanto, con frecuencia, una sola restricción global puede reemplazar aquello que requeriría de una gran cantidad de restricciones tradicionales de programación entera, al mismo tiempo que hace que el modelo sea mucho más fácil de leer. Para clarificar la presentación, se usarán ejemplos muy simples que no requieren el uso de programación de restricciones para ilustrar las restricciones globales, pero estos mismos tipos de restricciones también se pueden usar para resolver algunos problemas mucho más complicados.

Restricción todas diferentes

La restricción global *todas diferentes* sólo especifica que todas las variables de un conjunto dado deben tener valores diferentes. Si $x_1, x_2, \dots, x_n$ son las variables implicadas, la restricción puede escribirse de manera sucinta como

$$\text{todas diferentes } (x_1, x_2, \dots, x_n)$$

al mismo tiempo que también se especifican los dominios de las variables individuales del modelo. (Estos dominios en forma colectiva necesitan incluir al menos n valores diferentes para fortalecer la restricción de todas diferentes.)

Para ilustrar esta restricción, considere el *problema de asignación* clásico que se presentó en la sección 8.3. Recuerde que este problema implica conceder n asignados a n tareas en una base de uno a uno de manera que se minimice el costo total de estas asignaciones. Aunque el problema de asignación es muy fácil de resolver (como se describió en la sección 8.4), ilustra muy bien la forma en que la restricción de todas diferentes puede simplificar en gran medida la formulación del modelo.

Con la formulación tradicional que se presentó en la sección 8.3, las variables de decisión son las variables binarias

$$x_{ij} = \begin{cases} 1, & \text{si el asignado } i \text{ realiza la tarea } j \\ 0, & \text{si no es así} \end{cases}$$

para $i, j = 1, 2, \dots, n$. Si por ahora se hace caso omiso de la función objetivo, las restricciones funcionales son las siguientes.

Cada asignado i se asignará a sólo *una* tarea:

$$\sum_{j=1}^n x_{ij} = 1 \quad \text{para } i = 1, 2, \dots, n.$$

Cada tarea j será realizada por exactamente *un* asignado:

$$\sum_{i=1}^n x_{ij} = 1 \quad \text{para } j = 1, 2, \dots, n.$$

Así, hay n^2 variables y $2n$ restricciones funcionales.

Ahora se verá el modelo mucho más pequeño que puede proporcionar la programación de restricciones. En este caso, las variables son

$$y_i = \text{tareas que se encargan al asignado } i$$

para $i = 1, 2, \dots, n$. Existen n tareas y se enumeran $1, 2, \dots, n$, de forma que cada una de las y_i variables tiene el dominio $\{1, 2, \dots, n\}$. Como todos los asignados deben dirigirse a tareas diferentes, esta restricción sobre las variables es descrita en forma precisa por una sola restricción global,

$$\text{todas diferentes } (y_1, y_2, \dots, y_n).$$

Por tanto, en lugar de n^2 variables y $2n$ restricciones funcionales, este modelo de programación de restricciones completo (que excluye la función objetivo) tiene sólo n variables y *una sola* restricción (más un dominio para todas las variables).

Ahora se verá cómo la siguiente restricción global también permite incorporar la función objetivo a este pequeño modelo.

Restricción de elemento

La restricción global de *elemento* se utiliza casi siempre para buscar un costo o ganancia asociados con una variable entera. En particular, suponga que una variable y tiene un dominio $\{1, 2, \dots, n\}$ y que el costo asociado con cada una de estas variables es $c_1, c_2, \dots, c_n$, respectivamente. En consecuencia, la restricción

$$\text{elemento } (y, [c_1, c_2, \dots, c_n], z)$$

restringe la variable z a un valor igual a la y -ésima constante de la lista $[c_1, c_2, \dots, c_n]$. En otras palabras $z = c_y$. Ahora, esta variable z se puede incluir en la función objetivo para proporcionar el costo asociado con y .

Para ilustrar el uso de la restricción de elemento, considere de nuevo el problema de asignación; sea

$$c_{ij} = \text{costo de asignar a } i \text{ la tarea } j$$

para $i, j = 1, 2, \dots, n$. El modelo completo de programación de restricciones (que incluye la función objetivo) de este problema es

$$\text{Minimizar } Z = \sum_{i=1}^n z_i,$$

sujeta a

$$\begin{aligned} &\text{elemento } (y_i, [c_{i1}, c_{i2}, \dots, c_{in}], z_i) \quad \text{para } i = 1, 2, \dots, n, \\ &\text{todas diferentes } (y_1, y_2, \dots, y_n), \\ &y_i \in \{1, 2, \dots, n\} \quad \text{para } i = 1, 2, \dots, n. \end{aligned}$$

Este modelo completo ahora tiene $2n$ variables y $(n + 1)$ restricciones (además de un dominio para todas las variables), el cual es mucho más pequeño que el de la formulación de programación entera que se presentó en la sección 8.3. Por ejemplo, cuando $n = 100$, este modelo tiene 200 variables y 101 restricciones, mientras que el modelo tradicional de programación entera tiene 10 000 variables y 200 restricciones funcionales.

Como un ejemplo adicional, reconsidere el ejemplo 2 (violación de la proporcionalidad) que se presentó en la sección 11.4. En este caso, las variables de decisión originales son

$$x_j = \text{número de anuncios de televisión asignados al producto } j$$

para $j = 1, 2, 3$, donde un total de cinco anuncios de televisión se va a asignar a los tres productos. Sin embargo, como las ganancias que se presentan en la tabla 11.3 para valores diferentes de cada x_j no son proporcionales a x_j . En la sección 11.4 se formularon dos modelos alternativos de programación entera con variables binarias auxiliares para manejar este problema. Ambos modelos son bastante complicados.

Un modelo de programación de restricciones que utiliza la restricción de elemento es mucho más directo. Por ejemplo, la ganancia del producto 1 que se muestra en la tabla 11.3 es 0, 1, 3 y 3 para los valores respectivos $x_1 = 0, 1, 2$ y 3. Por tanto, esta ganancia simplemente es z_1 cuando el valor de z_1 está dado por la restricción

$$\text{elemento } (x_1 + 1, [0, 1, 3, 3], z_1).$$

(El primer componente es $x_1 + 1$ en lugar de x_1 porque $x_1 + 1 = 1, 2, 3$ o 4, y es el valor de este componente el que indica la elección de la posición 1, 2, 3 o 4 en la lista $[0, 1, 3, 3]$). Al proceder de la misma forma con los otros dos productos, el modelo completo es

$$\text{Maximizar } Z = z_1 + z_2 + z_3,$$

sujeta a

$$\begin{aligned} &\text{elemento } (x_1 + 1, [0, 1, 3, 3], z_1), \\ &\text{elemento } (x_2 + 1, [0, 0, 2, 3], z_2), \\ &\text{elemento } (x_3 + 1, [0, -1, 2, 4], z_3), \end{aligned}$$

$$\begin{aligned} x_1 + x_2 + x_3 &= 5, \\ x_j &\in \{0, 1, 2, 3\} \quad \text{para } j = 1, 2, 3. \end{aligned}$$

Ahora compare este modelo con los dos modelos de programación entera con el mismo modelo en la sección 11.4. Observe cómo el uso de las restricciones de elemento proporciona un modelo mucho más compacto y transparente.

Las restricciones *todas diferentes* y de *elemento* son sólo dos de las muchas restricciones globales disponibles (en la referencia seleccionada 6 se describen casi 40), pero ilustran muy bien el poder de la programación de restricciones para proporcionar un modelo compacto y fácil de leer de un problema complejo.

Investigación actual

La investigación actual relativa a la integración de la programación de restricciones y la programación entera se desarrolla en algunas trayectorias paralelas. Por ejemplo, el enfoque más directo es utilizar de manera simultánea un modelo de programación de restricciones y un modelo de programación entera para representar partes complementarias de un problema. Así, cada restricción relevante se incluye en cualquier modelo al que se ajuste o, cuando es factible, en ambos modelos. Como se aplican algoritmos de programación de restricciones y de programación entera a los modelos respectivos, la información fluye en ambos sentidos para enfocar la búsqueda de soluciones factibles (aquellas que satisfagan las restricciones de los dos modelos).

Este tipo de esquema de doble modelación se puede implantar con el lenguaje de programación para optimización (OPL, *Optimization Programming Language*) que se incorpora en el Sistema de Desarrollo ILOG OPL-CPLEX (ILOG es la compañía que produce el software de optimización CPLEX que se incluye en el OR Courseware). Después de emplear el lenguaje de modelación OPL, el Sistema de Desarrollo ILOG OPL-CPLEX puede invocar a un algoritmo de programación de restricciones (Optimizador ILOG CP) y a un solucionador de programación matemática (CPLEX) y después enviar información de uno al otro.

Aunque la doble programación es un buen primer paso, la meta es integrar por completo la programación de restricciones y la programación entera de manera que se pueda usar un solo modelo híbrido y un solo algoritmo. Cuando se logre este tipo de integración se podrá contar por completo con las fortalezas complementarias de las dos técnicas. A pesar de que el logro absoluto de esta meta permanece como un formidable reto de investigación, se continúa con buenos progresos en esta dirección. La referencia seleccionada 6 describe los avances más recientes en esta área.

Aún en esta etapa inicial ha habido un gran número de aplicaciones exitosas de la fusión de la programación matemática y la programación de restricciones. Las áreas de aplicación incluyen diseño de redes, determinación de rutas, rotación de tripulaciones, el problema clásico de transporte con costos lineales, administración de inventarios, gráficas por computadora, ingeniería de software, bases de datos, finanzas, ingeniería y optimización combinatoria, entre otras. Además, en la referencia seleccionada 2 se describe cómo las aplicaciones en la programación de actividades están demostrando ser un área particularmente fructífera para la aplicación de la programación de restricciones. Por ejemplo, debido a las múltiples y complicadas restricciones implicadas, en la actualidad se emplea la programación de restricciones para determinar el calendario regular de la Liga Nacional de Fútbol (NFL) de Estados Unidos.

Estas aplicaciones apenas comienzan a cubrir el potencial de la integración de la programación de restricciones y la programación entera. El progreso subsecuente en la consecución de esta integración promete abrir muchas oportunidades nuevas e interesantes para aplicaciones importantes.

11.10 CONCLUSIONES

Los problemas de PE surgen con frecuencia cuando los valores de algunas o de todas las variables de decisión deben restringirse a valores enteros. Existen también muchas aplicaciones que necesitan decisiones sí o no (aquí se incluyen las relaciones combinatorias que pueden expresarse en términos de tales decisiones) que se puede representar por variables binarias (0-1). Estos factores han hecho que la programación entera sea una de las técnicas de IO de mayor aplicación.

Los problemas de PE son más difíciles de lo que serían sin la restricción de valores enteros, de manera que los algoritmos disponibles para programación entera, en general, son mucho menos eficientes que el método símplex. Sin embargo, ha habido enormes progresos en las dos últimas décadas en cuanto a la capacidad para resolver algunos (pero no todos) problemas enormes de IP con decenas y aun cientos de miles de variables enteras. Dicho progreso se ha debido a la combinación de tres factores: mejoras significativas en los algoritmos IP, mejoras en los algoritmos de programación lineal que se utilizan con los algoritmos IP y la gran velocidad de las computadoras. Sin embargo, los algoritmos IP también llegan en ocasiones a fallar en la resolución de problemas pequeños (aun en los que tienen solamente cien variables enteras). Diferentes características de un problema IP además de su tamaño, tienen una gran influencia en la velocidad a la que se puede resolver.

Los factores que determinan el tiempo de cálculo son el número de *variables enteras* y el hecho de que el problema tenga una *estructura especial* que pueda explotarse. En el caso de un número fijo de variables enteras, casi siempre es más fácil resolver problemas de PEB que problemas con variables enteras generales; pero agregar variables continuas (PEM) puede no significar un incremento sustancial del tiempo de cálculo. Para problemas especiales de PEB que contienen alguna estructura especial que puede aprovechar un *algoritmo especial*, es posible resolver problemas muy grandes (muy por encima de mil variables binarias) en forma rutinaria.

En la actualidad es común que se disponga de paquetes de computadora para algoritmos de PE en el software de programación matemática. Estos algoritmos casi siempre se basan en la técnica de *ramificación y acotamiento* o en alguna variación de ésta.

Hoy en día, los algoritmos más modernos de PE emplean el enfoque de *ramificación y corte*. Este enfoque algorítmico involucra la combinación del preprocesado automático del problema, la generación de cortes y las técnicas de ramificación y acotamiento. Continúa la investigación en esta área, junto con el desarrollo de paquetes de software elaborados para incorporar estas técnicas.

El último desarrollo en PE es comenzar a incorporar la *programación de restricciones*. En apariencia este enfoque expandirá de manera considerable la capacidad de formular y resolver problemas de PE.

En años recientes se han llevado a cabo muchas investigaciones con el fin de desarrollar algoritmos (incluso heurísticos) para programación entera *no lineal*, y en esta área la investigación continúa muy activa. (La referencia seleccionada 8 presenta los avances más actuales en esta área.)

■ REFERENCIAS SELECCIONADAS

1. Appa, G., L. Pitsoulis y H. P. Williams (eds.), *Handbook on Modeling for Discrete Optimization*, Springer, Nueva York, 2006.
2. Baptise, P., C. LePape y W. Nuijten, *Constraint-Based Scheduling: Applying Constraint Programming to Scheduling Problems*, Kluwer Academic Publishers (actualmente Springer), Boston, 2001.
3. Barnhart, C., P. Belobaba y A. R. Odoni, "Applications of Operations Research in the Air Transport Industry", en *Transportation Science*, **37**(4): 368-391, 2003.
4. Bixby, R. E., Z. Gu, E. Rothberg y R. Wunderling, "Mixed Integer Programming: A Progress Report", pp. 309-326 en M. Grötschel (ed.), *The Sharpest Cut: The Impact of Manfred Padberg and His Work*, MPS/SIAM Series en Optimización.
5. Hillier, F. S. y M. S. Hillier, *Introduction to Management Science: A Modeling and Case Studies Approach with Spreadsheets*, 3a. ed., McGraw-Hill/Irwin, Burr Ridge, IL, 2008, capítulo 7.
6. Hooker, J. N., *Integrated Methods for Optimization*, Springer, Nueva York, 2007.
7. Karlof, J. K., *Integer Programming: Theory and Practice*, CRC Press, Boca Raton, FL, 2006.
8. Li, D. y X. Sun, *Nolinear Integer Programming*, Springer, Nueva York, 2006.
9. Lübbecke, M. E. y J. Desrosiers: "Selected Topics in Column Generation", en *Operations Research*, **53**(6): 1007-1023, noviembre-diciembre de 2005.
10. Lustig, I. y J.-F. Puget, "Program Does Not Equal Program: Constraint Programming and its Relationship to Mathematical Programming", en *Interfaces*, **31**(6): 29-53, noviembre-diciembre de 2001.
11. Nemhauser, G. L., "Need and Potential for Real-Time Mixed Integer Programming", en *OR/MS Today*, **34**(1): 21-22, febrero de 2007.
12. Nemhauser, G. L. y L. A. Wolsey, *Integer and Combinatorial Optimization*, Wiley, Nueva York, 1988, reimpreso en 1999.

13. Schriver, A., *Theory of Linear and Integer Programming*, Wiley, Nueva York, 1986.
14. Williams, H. P., *Model Building in Mathematical Programming*, 4a. ed., Wiley, Nueva York, 1999.
15. Wolsey, L. A., *Integer Programming*, Wiley, Nueva York, 1998.
16. Wolsey, L. A., “Strong Formulations for Mixed Integer Programs: Valid Inequalities and Extended Formulations”, en *Mathematical Programming Series B*, **97**(1-2): 423-447, 2003.

Algunas aplicaciones de programación entera ganadoras de premios

(En nuestro sitio en internet —www.mhhe.com/hillier— se proporciona una liga hacia estos artículos.)

- A1. Abbink, E., M. Fischetti, L. Kroon, G. Timmer y M. Vromans, “Reinventing Crew Scheduling at Netherlands Railways”, en *Interfaces*, **35**(5): 393-401, septiembre-octubre de 2005.
- A2. Armacost, A. P., C. Barnhart, K. A. Ware y A. M. Wilson, “UPS Optimizes Its Air Network”, en *Interfaces*, **34**(1): 15-25, enero-febrero de 2004.
- A3. Bertsimas, D., C. Darnell y R. Soucy, “Portfolio Construction Through Mixed-Integer Programming at Grantham, Mayo, Van Otterloo and Company”, en *Interfaces*, **29**(1): 49-66, enero-febrero de 1999.
- A4. Camm, J. D., T. E. Chorman, F. A. Dill, J. R. Evans, D. J. Sweeney y G. W. Wegryn, “Blending OR/MS, Judgment and GIS: Restructuring P&G’s Supply Chain”, en *Interfaces*, **27**(1): 128-142, enero-febrero de 1997.
- A5. Denton, B. T., J. Forrest y R. J. Milne, “IBM Solves a Mixed-Integer Program to Optimize Its Semiconductor Supply Chain”, en *Interfaces*, **36**(5): 386-399, septiembre-octubre de 2006.
- A6. Gendron, B., “Scheduling Employees in Quebec’s Liquor Stores with Integer Programming”, en *Interfaces*, **35**(5): 402-410, septiembre-octubre de 2005.
- A7. Gryffenberg, I., J. L. Lausberg, W. J. Smit, S. Uys, S. Botha, F. R. Hofmeyr, R. P. Nicolay, W. L. van der Merwe y G. J. Wessells, “Guns or Butter: Decision Support for Determining the Size and Shape of the South African National Defense Force”, en *Interfaces*, **27**(1): 7-28, enero-febrero de 1997.
- A8. Martin, C., D. Jones y P. Keskinocak, “Optimizing On-Demand Aircraft Schedules for Fractional Aircraft Operators”, en *Interfaces*, **33**(5): 22-35, septiembre-octubre de 2003.
- A9. Metty, T., R. Harlan, Q. Samelson, T. Moore, T. Morris, R. Sorenson, A. Scneur, O. Raskina, R. Schneur, J. Kanner, K. Potts y J. Robbins, “Reinventing the Supplier Negotiation Process at Motorola”, en *Interfaces*, **35**(1), 7-23, enero-febrero de 2005.
- A10. Smith, B. C., R. Darrow, J. Elieson, D. Guenther, B. V. Rao y F. Zouaoui, “Travelocity Becomes a Travel Retailer”, en *Interfaces*, **37**(1): 68-81, enero-febrero de 2007.
- A11. Spencer III, T., A. J. Brigandi, D. R. Dargon y M. J. Sheehan, “AT&T’s Telemarketing Site Selection System Offers Customer Support”, en *Interfaces*, **20**(1): 83-96, enero-febrero de 1990.
- A12. Subramanian, R., R. P. Scheff, Jr., J. D. Quillinan, D. S. Wiper y R. E. Marsten, “Coldstart: Fleet Assignment at Delta Air Lines”, en *Interfaces*, **24**(1): 104-120, enero-febrero de 1994.
- A13. Tyagi, R. y S. Bollapragada, “SES Americom Maximizes Satellite Revenues by Optimally Configuring Transponders”, en *Interfaces*, **33**(5): 36-44, septiembre-octubre de 2003.
- A14. Yu, G., M. Argüello, G. Song, S. M. McCowan y A. White, “A New Era for Crew Recovery at Continental Airlines”, en *Interfaces*, **33**(1): 5-22, enero-febrero de 2003.

■ AYUDAS DE APRENDIZAJE PARA ESTE CAPÍTULO EN EL SITIO EN INTERNET DE ESTE LIBRO (www.mhhe.com/hillier)

Ejemplos resueltos:

Ejemplos del capítulo 11

Ejemplo de demostración del OR Tutor:

Programación entera binaria, algoritmo de ramificación y acotamiento (*Binary Integer Programming Branch-and-Bound Algorithm*)

Programación entera mixta, algoritmo de ramificación y acotamiento (*Mixed Integer Programming Branch-and-Bound Algorithm*)

Rutinas interactivas del IOR Tutorial:

- Introducción o revisión de un modelo general de programación lineal (*Enter or Revise a General Linear Programming Model*)
- Solución interactiva de un programa entero binario (*Solve Binary Integer Program Interactively*)
- Solución interactiva de un programa entero mixto (*Solve Mixed Integer Program Interactively*)

Complemento de Excel:

Premium Solver for Education

Archivos para resolver los ejemplos “Ch. 11—Integer Programming”:

- Archivos de Excel
- Archivos de LINGO/LINDO
- Archivos de MPL/CPLEX

Glosario para el capítulo 11:

Vea en el apéndice 1 la documentación del software.

PROBLEMAS

Los símbolos a la izquierda de algunos problemas (o de sus incisos) significan lo siguiente:

- D: El ejemplo de demostración indicado que se mencionó en las ayudas de aprendizaje puede ser útil.
- I: Se sugiere que utilice las rutinas interactivas correspondientes (la impresión registra su trabajo).
- C: Use la computadora para resolver el problema; para ello utilice cualquier software disponible (o el que indique el profesor).

Un asterisco en el número del problema indica que al final del libro se proporciona al menos una respuesta parcial.

11.1-1. Reconsidere el ejemplo de la sección 11.1 de la California Manufacturing Co. El alcalde de San Diego se puso en contacto con el presidente de la compañía para persuadirlo de construir una fábrica y quizá un almacén en esa ciudad. Debido a los incentivos de impuestos que ofrece, el equipo del presidente estima que el valor presente neto de construir la planta sería de \$7 millones y el capital que se requiere sería de \$4 millones. El valor presente neto de construir el almacén sería de \$5 millones y el capital requerido, de \$3 millones. (Esta opción se considera sólo si se construye la planta en ese lugar.)

El presidente de la compañía quiere revisar el estudio de IO previo para incorporar estas nuevas alternativas al problema global. El objetivo todavía es encontrar la combinación factible de inversiones que maximice el valor presente neto total, dado que el capital disponible es de \$10 millones.

- a) Formule un modelo de PEB para este problema.
- b) Muestre el modelo en una hoja de cálculo de Excel.
- c) Use la computadora para resolver este modelo.

11.1-2.* Una joven pareja, Eve y Steven, quiere dividir las principales tareas del hogar (ir de compras, cocinar, lavar platos y lavar ropa) entre los dos, de manera que cada uno tenga dos obligaciones y el tiempo total para hacer estas tareas sea mínimo. La eficiencia en cada una de las tareas difiere entre ellos; la siguiente tabla proporciona el tiempo que cada uno necesita para cada tarea:

	Horas necesarias por semana			
	Compras	Cocinar	Lavar platos	Lavar ropa
Eve	4.5 horas	7.8 horas	3.6 horas	2.9 horas
Steven	4.9 horas	7.2 horas	4.3 horas	3.1 horas

- a) Formule un modelo de PEB para este problema.
- b) Muestre el modelo en una hoja de cálculo de Excel.
- c) Use la computadora para resolver este modelo.

11.1-3. Una empresa de bienes raíces, Peterson & Johnson, analiza cinco proyectos de desarrollo posibles. La siguiente tabla muestra las ganancias a largo plazo estimadas (valor presente neto) que generaría cada proyecto y la inversión que se requiere para emprenderlo, en millones de dólares.

	Proyecto de desarrollo				
	1	2	3	4	5
Ganancia estimada	1	1.8	1.6	0.8	1.4
Capital requerido	6	12	10	4	8

Los propietarios de la empresa, Dave Peterson y Ron Johnson, reunieron \$20 millones de capital de inversión para estos proyectos. Ellos quieren elegir la combinación de proyectos que maximice la ganancia total estimada a largo plazo (valor presente neto) sin invertir más de \$20 millones.

- a) Formule un modelo de PEB para este problema.
- b) Muestre el modelo en una hoja de cálculo de Excel.
- c) Use la computadora para resolver este modelo.

11.1-4. El consejo directivo de General Wheeis Co., estudia seis grandes inversiones de capital. Cada inversión se puede hacer sólo una vez. Estas inversiones difieren en la ganancia estimada a largo

plazo (valor presente neto) que generarán, así como en la cantidad de capital que requiere cada uno, como se muestra en la siguiente tabla (en millones de dólares):

	Oportunidad de inversión					
	1	2	3	4	5	6
Ganancia estimada	15	12	16	18	9	11
Capital requerido	38	33	39	45	23	27

Se dispone de \$100 millones de dólares como capital total para estas inversiones. Las oportunidades de inversión 1 y 2 son mutuamente excluyentes, lo mismo que 3 y 4. Más aún, la oportunidad 3 o la 4 no se pueden aprovechar a menos que se invierta en una de las dos primeras opciones. No existen restricciones de este tipo sobre las oportunidades de inversión 5 y 6. El objetivo es elegir la combinación de inversiones de capital que maximice la ganancia estimada a largo plazo (valor presente neto).

a) Formule el modelo de PEB para este problema.

c b) Use la computadora para resolver este modelo.

11.1-5. Reconsidere el problema 8.3-4, donde el entrenador de un equipo de natación debe asignar nadadores a los distintos equipos del combinado de 200 metros. Formule un modelo de PEB para este problema. Identifique los grupos de alternativas mutuamente excluyentes.

11.1-6. Vincent Cardoza es el propietario y director de un taller de maquinado que trabaja sobre pedido. El miércoles por la tarde recibió llamadas de dos clientes que necesitan órdenes urgentes. Un transportista de autos compactos necesita barras estabilizadoras. Una compañía de enganches para remolques requiere barras de remolque especiales para trabajo pesado. Ambos clientes quieren la mayor cantidad posible para el fin de semana (dos días hábiles). Como los dos productos usarán las mismas dos máquinas, Vincent debe decidir e informarles esta tarde cuántos productos de cada uno fabricará en los dos días siguientes.

Cada barra de remolque requiere 3.2 horas en la máquina 1 y 2 horas en la 2. Cada barra estabilizadora requiere 2.4 horas en la máquina 1 y 3 en la 2. La máquina 1 estará disponible 16 horas en los próximos dos días y la 2, 15 horas. La ganancia de cada barra de remolque producida será de \$130 y la de cada barra estabilizadora será de \$150.

Vincent quiere determinar la mezcla de estas cantidades de producción que maximizará su ganancia total.

a) Formule un modelo de PE para este problema.

b) Use un enfoque gráfico para resolver el modelo.

c c) Use la computadora para resolver este modelo.

11.1-7. Reconsidere el problema 8.2-21 de la contratista (Susan Meyer) que necesita acarrear grava de dos fosos a tres construcciones.

Ahora Susan necesita contratar camiones (y sus choferes) para el acarreo; cada camión puede usarse sólo para llevar grava de un foso a una construcción. Además de los costos de acarreo y grava que se especificaron, existe un costo fijo de \$150 asociado a la contratación de cada camión. Un camión puede cargar 5 toneladas pero no tiene que ir lleno. Deben tomarse dos decisiones para cada combinación de foso y construcción, el número de camiones que se empleará y la cantidad de grava que van a acarrear.

a) Formule un modelo de PEM para este problema.

c b) Use la computadora para resolver el modelo.

11.2-1. Lea el artículo de referencia que describe en su totalidad el estudio de OR que se resume en el recuadro de aplicación que se presentó en la sección 11.2. Describa en forma breve la forma en que la programación entera se aplicó a este estudio. Después, elabore una lista de los diferentes beneficios financieros y no financieros que arrojó este estudio.

11.2-2. Seleccione una de las aplicaciones reales de PEB de las compañías u oficinas de gobierno que se mencionaron en la sección 11.2. Lea el artículo que la describe en la referencia de *Interfaces*. Escriba un resumen de dos páginas de la aplicación y sus beneficios.

11.2-3. Seleccione tres de las aplicaciones reales de PEB de compañías u oficinas de gobierno que se mencionó en la sección 11.2. Lea los artículos que las describen en las referencias de *Interfaces*. Escriba un resumen de una página de la aplicación y sus beneficios de cada uno.

11.3-1.* La división de investigación y desarrollo de la Progressive Company está en proceso de desarrollar cuatro líneas de posibles nuevos productos. La administración debe decidir cuáles de estos cuatro productos fabricar y a qué niveles. Ha pedido al departamento de IO que formule un modelo de programación matemática para encontrar la mezcla de productos más redituable.

La puesta en marcha de la fabricación de cualquier producto se asocia a un costo sustancial, que se proporciona en el primer renglón de la tabla. El objetivo de la administración es encontrar la mezcla de productos que maximice la ganancia total (ingreso neto total menos costos fijos).

	Producto			
	1	2	3	4
Costo de arranque	\$50 000	\$40 000	\$70 000	\$60 000
Ingreso marginal	\$ 70	\$ 60	\$ 90	\$ 80

Defina las variables de decisión continuas x_1 , x_2 , x_3 y x_4 como los niveles de producción de los productos 1, 2, 3 y 4. Por políticas de la empresa, la administración ha impuesto las siguientes restricciones sobre estas variables:

1. Como máximo, sólo deben fabricarse dos de estos productos.

2. Cualquiera de los productos 3 o 4 se puede producir sólo si se fabrica el producto 1 o el 2.

3. O bien $5x_1 + 3x_2 + 6x_3 + 4x_4 \leq 6\,000$
o $4x_1 + 6x_2 + 3x_3 + 5x_4 \leq 6\,000$.

a) Introduzca variables binarias auxiliares para formular un modelo de PEB mixta de este problema.

c b) Use la computadora para resolver este modelo.

11.3-2. Suponga que un modelo matemático se ajusta a programación lineal excepto por la restricción $|x_1 - x_2| = 0$, o 3, o 6. Reformule esta restricción para ajustarla a PEM.

11.3-3. Suponga que un modelo matemático se ajusta a programación lineal excepto por las siguientes restricciones:

1. Al menos una de las dos desigualdades se cumple:

$$\begin{aligned} 3x_1 - x_2 - x_3 + x_4 &\leq 12 \\ x_1 + x_2 + x_3 + x_4 &\leq 15. \end{aligned}$$

2. Al menos dos de las tres desigualdades se cumplen:

$$\begin{aligned} 2x_1 + 5x_2 - x_3 + x_4 &\leq 30 \\ -x_1 + 3x_2 + 5x_3 + x_4 &\leq 40 \\ 3x_1 - x_2 + 3x_3 - x_4 &\leq 60. \end{aligned}$$

Reformule estas restricciones para que se ajusten a PEM.

11.3-4. La compañía Toys-R-4-U ha desarrollado dos nuevos juguetes para su posible inclusión en la línea de productos la próxima temporada de Navidad. La preparación de instalaciones para iniciar la fabricación costaría \$50 000 en el caso del juguete 1 y \$80 000 en el del juguete 2. Una vez cubiertos estos costos, se obtendría una ganancia unitaria de \$10 por el juguete 1 y \$15 por el juguete 2.

La compañía tiene dos plantas que pueden producir estos juguetes. Sin embargo, para evitar la duplicidad de costos de preparación, sólo se usará una de ellas, y la elección depende de la maximización de la ganancia. Por razones administrativas, se usará la misma planta para ambos juguetes nuevos si se producen los dos.

El juguete 1 se puede producir a una tasa de 50 unidades por hora en la planta 1 y 40 por hora en la 2. El juguete 2 se puede producir a una tasa de 40 unidades por hora en la planta 1 y 25 por hora en la 2. Las plantas 1 y 2 tienen 500 y 700 horas de producción disponibles, respectivamente, antes de Navidad, que se pueden usar para producir estos juguetes.

No se sabe si estos juguetes continuarán fabricándose después de Navidad. Por lo tanto, el problema es determinar cuántas unidades (si se fabrican) de cada juguete nuevo deben producirse antes de Navidad a fin de maximizar la ganancia total.

a) Formule un modelo de PEM para este problema.

c b) Use la computadora para resolver este modelo.

11.3-5.* La línea aérea Northeastern piensa comprar jets de pasajeros grandes, medianos y chicos. El precio de compra de cada avión grande será de \$67 millones, \$50 millones el de los medianos y \$35 millones el de los chicos. El consejo directivo ha autorizado un compromiso máximo de \$1.5 mil millones para realizar estas compras. Sin que importe qué aviones se compren, se espera que las distancias de los trayectos sean lo suficientemente grandes como para que los aviones se utilicen, en esencia, a su capacidad máxima. Se estima que la ganancia neta anual (después de restar los costos de recuperación de capital) de un avión grande será de \$4.2 millones, \$3 millones si se trata de un avión mediano y \$2.3 millones de cada avión chico.

Se piensa que la compañía podrá disponer de suficientes pilotos entrenados para operar 30 aviones nuevos. Si sólo se compraran aviones chicos, las instalaciones de mantenimiento podrían manejar 40 aviones, pero cada avión mediano equivale a aviones chicos y cada avión grande equivale a aviones chicos, en términos de la utilización de las instalaciones de mantenimiento.

Esta información se obtuvo en un análisis preliminar del problema. Más adelante se llevará a cabo un estudio más detallado. Si se toman estos datos como una primera aproximación, la gerencia desea saber cuántos aviones de cada tipo debe comprar a fin de maximizar la ganancia.

a) Formule un modelo de PE para representar este problema.

c b) Use la computadora para resolverlo.

c) Utilice una representación binaria de las variables para reformular el modelo de PE del inciso a) como un modelo de PEB.

c d) Utilice la computadora para resolver el modelo de PEB del inciso c). Después use esta solución óptima para identificar una solución óptima para el modelo de PE que se formuló en el inciso a).

11.3-6. Considere el ejemplo de PE de dos variables de la sección 11.5, que se presentó en la figura 11.3.

a) Use una representación binaria de variables para reformular este modelo como un problema de PEB.

c b) Utilice la computadora para resolver este problema de PEB. Después use la solución óptima para identificar una solución óptima para el modelo original de PE.

11.3-7. La compañía aérea Fly-Right construye jets pequeños que vende a corporaciones para uso ejecutivo. Para cumplir con sus necesidades, en ocasiones los clientes ordenan aviones con diseño especial. Cuando es así, se incurre en un costo de preparación para iniciar la producción de las aeronaves.

Fly-Right acaba de recibir pedidos de tres clientes con fechas de entrega cercanas. Debido a que las instalaciones de producción están comprometidas para cumplir contratos anteriores, no podrán aceptar los tres pedidos. En consecuencia, debe decidirse el número de aviones que producirán (si lo hacen) para cada uno de los tres clientes.

Los datos relevantes se presentan en la siguiente tabla. El primer renglón contiene los costos fijos para iniciar la producción de aviones de cada cliente. Con la producción en marcha, el ingreso neto marginal (precio de compra menos costo marginal de producción) de cada avión se presenta en el segundo renglón. El tercero contiene los porcentajes de capacidad de producción disponibles para cada avión. El último renglón indica el número máximo de aviones pedidos por cada cliente (pero aceptarían menos).

	Cliente		
	1	2	3
Costo de arranque	\$3 millones	\$2 millones	0
Ingreso neto marginal	\$2 millones	\$3 millones	\$0.8 millones
Capacidad usada/avión	20%	40%	20%
Pedido máximo	3 aviones	2 aviones	5 aviones

Fly-Right desea determinar cuántos aviones debe producir para cada cliente (si lo hace) de modo que se maximice su ganancia total (ingresos netos menos costos fijos).

a) Formule un modelo con variables enteras y variables binarias para representar este problema.

c b) Use la computadora para resolver el modelo.

11.4-1. Reconsidere el problema 11.3-7 de Fly-Right. Un análisis más detallado de los factores de costo e ingreso indica que la ganancia potencial de producir los aviones de cada cliente no se puede expresar sólo en términos del *costo fijo* y un *ingreso neto marginal* por avión producido. Más bien, las ganancias se proporcionan en la siguiente tabla.

Aviones producidos	Ganancias por cliente		
	1	2	3
0	0	0	0
1	-\$1 millón	\$1 millón	\$1 millón
2	\$2 millones	\$5 millones	\$3 millones
3	\$4 millones		\$5 millones
4			\$6 millones
5			\$7 millones

- a) Formule un modelo de PEB para manejar este problema que incluya restricciones para las *alternativas mutuamente excluyentes*.
- c b) Use la computadora para resolver el modelo formulado en el inciso a). Después, utilice esta solución óptima para identificar el número óptimo de aviones que se debe fabricar para cada cliente.
- c) Formule otro modelo de PEB para representar este problema que incluya restricciones sobre las *decisiones contingentes*.
- c d) Repita el inciso b) para el modelo formulado en el inciso c).

11.4-2. Reconsidere el problema de la Wyndor Glass Co., que se presentó en la sección 3.1. La administración ha decidido que sólo debe fabricarse uno de los dos nuevos productos y la elección debe hacerse para maximizar la ganancia. Introduzca *variables binarias auxiliares* para formular un modelo de PEM para construir esta nueva versión del problema.

11.4-3.* Reconsidere el problema 3.1-11, en el cual la administración de Omega estudia la posibilidad de dedicar el exceso de capacidad a uno o más de tres productos. (Vea información adicional de este problema en las respuestas al final del libro.) Ahora agregue la restricción de la administración de que no deben producirse más de dos de los tres nuevos productos.

- a) Introduzca *variables binarias auxiliares* para formular un modelo de PEM para presentar esta nueva versión del problema.
- c b) Use la computadora para resolver el modelo.

11.4-4. Considere el siguiente problema de programación entera no lineal.

$$\text{Maximizar } Z = 4x_1^2 - x_1^3 + 10x_2^2 - x_2^4,$$

sujeta a

$$x_1 + x_2 \leq 3$$

y

$$x_1 \geq 0, \quad x_2 \geq 0$$

$$x_1 \text{ y } x_2 \text{ son enteros.}$$

Este problema se puede reformular de dos maneras como un problema de PEB equivalente (con una función objetivo lineal) y seis variables binarias (y_{1j} y y_{2j} para $j = 1, 2, 3$), según la interpretación que se dé a las variables binarias.

- a) Formule un modelo de PEB que represente este problema donde las variables binarias tienen la siguiente interpretación,

$$y_{ij} = \begin{cases} 1 & \text{si } x_i = j \\ 0 & \text{de otra manera.} \end{cases}$$

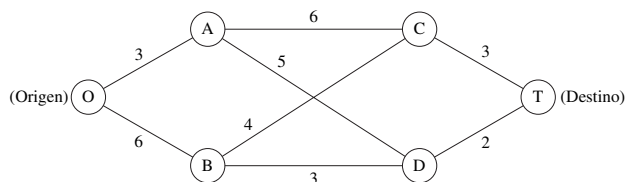
- c b) Utilice la computadora para resolver el modelo que se formuló en el inciso a), y después encuentre una solución óptima para (x_1, x_2) en el problema original.

- c) Formule un modelo de PEB para este problema donde las variables binarias tengan la siguiente interpretación:

$$y_{ij} = \begin{cases} 1 & \text{si } x_i \geq j \\ 0 & \text{de otra manera.} \end{cases}$$

- c d) Use la computadora para resolver el modelo que se formuló en el inciso c), y después encuentre una solución óptima para (x_1, x_2) en el problema original.

11.4-5.* Considere el siguiente tipo especial de *problema de la ruta más corta* (sección 9.3) en el que los nodos están en columna y las únicas trayectorias posibles se mueven siempre hacia adelante, una columna a la vez.



Los números colocados junto a las trayectorias son distancias y el objetivo es encontrar la ruta más corta del origen al destino.

Este problema también se puede formular como un modelo de PEB que abarca tanto alternativas mutuamente excluyentes como decisiones contingentes.

- a) Formule este modelo. Identifique las restricciones de las alternativas mutuamente excluyentes y las de decisiones contingentes.
- c b) Use la computadora para resolver este problema.

11.4-6. Speedy Delivery proporciona un servicio que entrega paquetes grandes en dos días, en todo Estados Unidos. Cada mañana se cargan los paquetes que llegaron a cada centro de recolección durante la noche en los camiones de reparto para su entrega en el área. En razón de que la competencia en este negocio se basa en la rapidez de la entrega, los paquetes se dividen según sus destinos geográficos de manera que se minimice el tiempo promedio necesario para realizar las entregas.

Esta mañana, la despachadora del centro de recolección de Blue River Valley, Sharon Lofton, tiene mucho trabajo. Sus tres choferes llegarán en menos de una hora para el reparto. Hay nueve paquetes que entregar en lugares muy alejados entre sí. Como siempre, Sharon introduce estos lugares en la computadora para usar Dispatcher, el software especial del sistema de apoyo. El programa usa las ubicaciones para generar un buen número de rutas posibles para cada camión. Estas rutas se muestran en la siguiente tabla (donde los números en cada columna indican el orden de las entregas), junto con los tiempos que se requieren para el recorrido.

Ubicación de la entrega	Rutas atractivas posibles									
	1	2	3	4	5	6	7	8	9	10
A	1				1				1	
B		2		1		2			2	2
C			3	3			3		3	
D	2					1		1		
E			2	2		3				
F		1			2					
G	3						1	2		3
H			1		3					1
I		3		4			2			
Tiempo (en horas)	6	4	7	5	4	6	5	3	7	6

Dispatcher es un sistema interactivo que muestra estas rutas para que Sharon las apruebe o modifique. (Quizá la computadora no sepa que una inundación ha hecho que una ruta sea no factible.) Si Sharon aprueba las rutas como posibilidades atractivas con tiempos estimados razonables, el programa formula y resuelve un modelo de PEB para elegir las tres rutas que minimizan el tiempo total e incluye cada lugar de entrega sólo en una ruta. Esta mañana, Sharon aprueba todas las rutas.

a) Formule el modelo de PEB.

c b) Use la computadora para resolver el problema.

11.4-7. Cada vez más estadounidenses se mudan a un clima más templado cuando se retiran. Para aprovechar esta tendencia, Sunny Skies Unlimited emprenderá un proyecto de desarrollo urbano. Se creará una comunidad nueva por completo (llamada Pilgrim Heaven) que cubrirá varias millas cuadradas. Una de las decisiones que se debe tomar es la ubicación más conveniente de las dos estaciones de bomberos que se le asignaron a la comunidad. Para propósitos de planeación, se dividió Pilgrim Heaven en cinco sectores, con sólo una estación de bomberos en cada uno de ellos. Cada estación debe responder a *todos* los llamados que reciba del sector en el que se localiza y las de otros que se le asignen. Entonces, las decisiones son: 1) los sectores que albergarán una estación de bomberos y 2) la asignación de cada uno de los otros sectores a una de las estaciones. El objetivo es minimizar el promedio global de los *tiempos de respuesta* a los incendios.

La siguiente tabla proporciona el tiempo promedio de respuesta (en minutos) a un incendio en cada sector (columnas) si el servicio se presta desde la estación de un sector dado (renglones). El último renglón proporciona el pronóstico del número promedio de incendios diarios que ocurrirán en cada uno de los sectores.

Estación asignada localizada en el sector	Tiempos de respuesta en minutos Incendios en el sector				
	1	2	3	4	5
1	5	12	30	20	15
2	20	4	15	10	25
3	15	20	6	15	12
4	25	15	25	4	10
5	10	25	15	12	5
Frecuencia de emergencia (por día)	2 por día	1 por día	3 por día	1 por día	3 por día

Formule un modelo completo de PEB para representar el problema. Identifique la restricción de las alternativas mutuamente excluyentes o de las decisiones contingentes.

11.4-8. Reconsidere el problema 11.4-7. Ahora, la administración de Sunny Skies desea que la decisión sobre el lugar en donde se ubicarán las estaciones de bomberos se base en los costos.

El costo de asignar una estación de bomberos en el sector 1 es de \$300 000; de \$350 000 en el 2; de \$600 000 en el 3; de \$450 000 en el 4 y de \$700 000 en el 5. Ahora, el nuevo objetivo de la gerencia es:

Determinar cuáles sectores deben tener una estación para minimizar el costo total de las estaciones al mismo tiempo que se asegure que cada sector tenga al menos una estación lo suficientemente cerca para responder a un incendio en no más de 12 minutos (en promedio).

Observe que al contrario del problema original, el número total de estaciones de bomberos no es fijo. Aún más, si un sector sin estación tiene más de una estación a 12 minutos o menos, ya no es necesario asignar este sector a sólo una de las estaciones.

a) Formule un modelo de PEB pura con cinco variables binarias para representar este problema.

b) ¿Éste es un *problema de cobertura de conjunto*? Explique e identifique los conjuntos relevantes.

c c) Use la computadora para resolver el modelo formulado en a).

11.4-9. Suponga que un estado envía R personas a la cámara de diputados. Existen D municipios en el estado ($D > R$) y la legislatura estatal quiere agrupar estos municipios en R distritos electorales, cada uno de los cuales elegirá un diputado. La población total del estado es P , y la legislatura desea formar distritos cuya población se aproxime a $p = P/R$. Suponga que el comité legislativo que estudia el problema ha generado una larga lista de N *candidatos* a ser convertidos en distritos ($N > R$). Cada candidato abarca municipios contiguos y una población total p_j ($j = 1, 2, \dots, N$) aceptablemente cercana a p . Defina $c_j = |p_j - p|$. Cada municipio i ($i = 1, 2, \dots, D$) está incluido al menos con un candidato a ser distrito y casi en todos los casos estará incluido con un gran número de candidatos (con el fin de proporcionar muchas maneras factibles de seleccionar un conjunto de R candidatos que incluyan a cada municipio exactamente una vez). Defina

$$a_{ij} = \begin{cases} 1 & \text{si el municipio } i \text{ se incluye con el candidato } j \\ 0 & \text{si no.} \end{cases}$$

Dados los valores de las c_j y las a_{ij} el objetivo es seleccionar R de estos N posibles distritos, tales que cada municipio esté contenido en un solo distrito y que la más grande de las c_j asociadas sea lo más pequeña posible.

Formule un modelo de PEB para este problema.

11.5-1. Lea el artículo de referencia que describe en su totalidad el estudio de OR que se resume en el recuadro de aplicación que se presentó en la sección 11.5. Describa brevemente la forma en que se aplicó la programación entera en el estudio. Después, elabore una lista de los beneficios financieros y de otro tipo que arrojó este estudio.

11.5-2.* Considere el siguiente problema de PE.

$$\text{Maximizar } Z = 5x_1 + x_2,$$

sujeta a

$$\begin{aligned} -x_1 + 2x_2 &\leq 4 \\ x_1 - x_2 &\leq 1 \\ 4x_1 + x_2 &\leq 12 \end{aligned}$$

y

$$\begin{aligned} x_1 &\geq 0, \quad x_2 \geq 0 \\ x_1, x_2 &\text{ son enteros.} \end{aligned}$$

a) Resuelva este problema de manera gráfica.

b) Resuelva el relajamiento de PL de manera gráfica. Redondee esta solución a la solución entera *más cercana* y verifique si es factible. Después enumere *todas* las soluciones redondeadas. Para ello redondee esta solución del relajamiento de PL de *todas* las maneras posibles (es decir, redondee cada valor no entero tanto

hacia arriba como hacia abajo). Verifique la factibilidad de cada solución redondeada y, si es factible, calcule Z . ¿Alguna de estas soluciones redondeadas es óptima para el problema de PE?

11.5-3. Siga las instrucciones del problema 11.5-2 para resolver el siguiente problema de PE.

$$\text{Maximizar } Z = 220x_1 + 80x_2,$$

sujeta a

$$\begin{aligned} 5x_1 + 2x_2 &\leq 16 \\ 2x_1 - x_2 &\leq 4 \\ -x_1 + 2x_2 &\leq 4 \end{aligned}$$

y

$$\begin{aligned} x_1 &\geq 0, \quad x_2 \geq 0 \\ x_1, x_2 &\text{ son enteros.} \end{aligned}$$

11.5-4. Siga las instrucciones del problema 11.5-2 para solucionar el siguiente problema de PEB.

$$\text{Maximizar } Z = 10x_1 + 25x_2,$$

sujeta a

$$\begin{aligned} 19x_1 + 6x_2 &\leq 15 \\ 5x_1 - 15x_2 &\leq 15 \end{aligned}$$

y

$$x_1, x_2 \text{ son binarios.}$$

11.5-5. Siga las instrucciones del problema 11.5-2 para solucionar el siguiente problema de PE.

$$\text{Maximizar } Z = -5x_1 + 25x_2,$$

sujeta a

$$\begin{aligned} -3x_1 + 30x_2 &\leq 27 \\ 3x_1 + x_2 &\leq 4 \end{aligned}$$

y

$$x_1, x_2 \text{ son binarios.}$$

11.5-6. Diga si cada una de las afirmaciones siguientes es falsa o verdadera y justifique su respuesta con referencia al texto específico (con la cita de página) del capítulo.

- En general es más fácil resolver problemas de programación lineal que problemas de PE.
- Por lo general, en los problemas de PE el número de variables enteras es más importante para determinar la dificultad en los cálculos que el número de restricciones funcionales.
- Para resolver un problema de PE con un procedimiento aproximado se puede aplicar el método simplex al relajamiento de PE y redondear cada valor no entero al entero más cercano. El resultado será factible pero no necesariamente la solución óptima del problema de PE.

D.I 11.6-1.* Utilice el algoritmo de ramificación y acotamiento de PEB de la sección 11.6 para resolver el siguiente problema de manera interactiva.

$$\text{Maximizar } Z = 2x_1 - x_2 + 5x_3 - 3x_4 + 4x_5,$$

sujeta a

$$\begin{aligned} 3x_1 - 2x_2 + 7x_3 - 5x_4 + 4x_5 &\leq 6 \\ x_1 - x_2 + 2x_3 - 4x_4 + 2x_5 &\leq 0 \end{aligned}$$

y

$$x_j \text{ es binario, para } j = 1, 2, \dots, 5.$$

D.I 11.6-2. Aplique el algoritmo de ramificación y acotamiento de PEB de la sección 11.6 para resolver el siguiente problema de manera interactiva.

$$\text{Minimizar } Z = 5x_1 + 6x_2 + 7x_3 + 8x_4 + 9x_5,$$

sujeta a

$$\begin{aligned} 3x_1 - x_2 + x_3 + x_4 - 2x_5 &\geq 2 \\ x_1 + 3x_2 - x_3 - 2x_4 + x_5 &\geq 0 \\ -x_1 - x_2 + 3x_3 + x_4 + x_5 &\geq 1 \end{aligned}$$

y

$$x_j \text{ es binario, para } j = 1, 2, \dots, 5.$$

D.I 11.6-3. Utilice el algoritmo de ramificación y acotamiento de PEB de la sección 11.6 para resolver el siguiente problema de manera interactiva.

$$\text{Maximizar } Z = 3x_1 + 3x_2 + 5x_3 - 2x_4 - x_5,$$

sujeta a

$$\begin{aligned} x_1 + 2x_2 - 3x_4 - x_5 &\leq 0 \\ -15x_1 + 30x_2 - 35x_3 + 45x_4 + 45x_5 &\geq 50 \end{aligned}$$

y

$$x_j \text{ es binario, para } j = 1, 2, \dots, 5.$$

D.I 11.6-4. Reconsidere el problema 11.3-6a). Use el algoritmo de ramificación y acotamiento de PEB de la sección 11.6 para resolver este modelo de manera interactiva.

D.I 11.6-5. Reconsidere el problema 11.4-8a). Utilice el algoritmo de PEB de la sección 11.6 para resolver este problema de manera interactiva.

11.6-6. Considere las siguientes afirmaciones sobre un problema de PE pura (en la forma de maximización) y su relajamiento de PE. Diga si cada una es falsa o verdadera y justifique su respuesta.

- La región factible del relajamiento de PE es un subconjunto de la región factible del problema de PE.
- Si una solución óptima del relajamiento de PE es una solución entera, entonces el valor óptimo de la función objetivo es el mismo en ambos problemas.
- Si una solución no entera es factible para el relajamiento de PE, entonces la solución entera más cercana (al redondear cada variable al entero más próximo) es una solución factible para el problema de PE.

11.6-7.* Considere el problema de asignación que tiene la siguiente tabla de costos:

		Tarea				
		1	2	3	4	5
Asignado	1	39	65	69	66	57
	2	64	84	24	92	22
	3	49	50	61	31	45
	4	48	45	55	23	50
	5	59	34	30	34	18

- a) Diseñe un algoritmo de ramificación y acotamiento para resolver problemas de asignación; especifique cómo se deben realizar los pasos de ramificación, acotamiento y sondeo. (*Sugerencia:* Para los asignados que todavía no se incluyen en el subproblema actual, forme el relajamiento de PL mediante la eliminación de las restricciones de que cada uno debe realizar exactamente una tarea.)
- b) Utilice este algoritmo para resolver el problema.

11.6-8. Es necesario realizar cinco trabajos en un cierto número de máquinas. El tiempo de preparación de estos trabajos depende del trabajo que les antecede, según se muestra en la siguiente tabla:

		Tiempo de preparación				
		Trabajo				
		1	2	3	4	5
Trabajo precedente inmediato	Ninguno	4	5	8	9	4
	1	—	7	12	10	9
	2	6	—	10	14	11
	3	10	11	—	12	10
	4	7	8	15	—	7
	5	12	9	8	16	—

El objetivo es programar la *secuencia* de trabajos que minimice la suma de los tiempos de preparación resultantes.

- a) Diseñe un algoritmo de ramificación y acotamiento para problemas de secuenciación; especifique los pasos de ramificación, de acotamiento y de sondeo.
- b) Use este algoritmo para resolver el problema.

11.6-9.* Considere el siguiente problema de PEB *no lineal*.

$$\text{Maximizar } Z = 80x_1 + 60x_2 + 40x_3 + 20x_4 - (7x_1 + 5x_2 + 3x_3 + 2x_4)^2,$$

sujeta a

$$x_j \text{ es binario, para } j = 1, 2, 3, 4.$$

Dados los valores de las primeras k variables $x_1, \dots, x_k$, donde $k = 0, 1, 2$ o 3 , una cota superior del valor de Z que se puede lograr mediante las soluciones factibles es

$$\sum_{j=1}^k c_j x_j - \left(\sum_{j=1}^k d_j x_j \right)^2 + \sum_{j=k+1}^4 \max \left\{ 0, c_j - \left[\left(\sum_{i=1}^k d_i x_i + d_j \right)^2 - \left(\sum_{i=1}^k d_i x_i \right)^2 \right] \right\},$$

donde $c_1 = 80, c_2 = 60, c_3 = 40, c_4 = 20, d_1 = 7, d_2 = 5, d_3 = 3, d_4 = 2$. Utilice esta cota para resolver el problema mediante la técnica de ramificación y acotamiento.

11.6-10. Considere el relajamiento de Lagrange descrito casi al final de la sección 11.6.

- a) Si $\mathbf{x}$ es una solución factible para un problema de PEM, demuestre que $\mathbf{x}$ también debe ser factible para el relajamiento de Lagrange correspondiente.
- b) Si $\mathbf{x}^*$ es una solución óptima para un problema de PEM, con Z como valor de la función objetivo, demuestre que $Z \leq Z_R^*$, donde Z_R^* es el valor óptimo de la función objetivo del relajamiento de Lagrange correspondiente.

11.7-1. Lea el artículo de referencia que describe en su totalidad el estudio de OR que se resume en el Recuadro de aplicación que se presentó en la sección 11.7. Describa brevemente la forma en que se aplicó la programación entera en el estudio. Después, elabore una lista de los beneficios financieros y no financieros que arrojó este estudio.

11.7-2.* Considere el siguiente problema de PE.

$$\text{Maximizar } Z = -3x_1 + 5x_2,$$

sujeta a

$$5x_1 - 7x_2 \geq 3$$

y

$$x_j \leq 3$$

$$x_j \geq 0$$

$$x_j \text{ es entero, para } j = 1, 2.$$

- a) Resuelva este problema de manera gráfica.
- b) Utilice el algoritmo de ramificación y acotamiento de PEM que se presentó en la sección 11.7 para resolver este problema en forma manual. Resuelva el relajamiento de PL de cada subproblema por el método *gráfico*.
- c) Utilice la representación binaria de variables enteras para reformular este problema como uno de PEB.
- d) Aplique el algoritmo de ramificación y acotamiento de PEB de la sección 11.6 para resolver el inciso c) del modelo.

11.7-3. Siga las instrucciones del problema 11.7-2 para resolver el siguiente modelo de PE.

$$\text{Minimizar } Z = 15x_1 + 10x_2,$$

sujeta a

$$15x_1 + 5x_2 \geq 30$$

$$10x_1 + 10x_2 \geq 30$$

y

$$x_1 \geq 0, \quad x_2 \geq 0$$

$$x_1, x_2 \text{ son enteros.}$$

11.7-4. Reconsidere el problema 11.5-2.

- a) Use el algoritmo de ramificación y acotamiento de PEM de la sección 11.7 para resolver este problema en forma manual. Resuelva el relajamiento de PL de cada subproblema por el método *gráfico*.
- b) Utilice la rutina interactiva para este algoritmo del OR Courseware para resolver el problema.
- c) Verifique su respuesta mediante el uso de la rutina automática para resolver el problema.

D.I 11.7-5. Considere el ejemplo de PE de la sección 11.5 que se presentó en la figura 11.3. Utilice el algoritmo de ramificación y acotamiento de PEM de la sección 11.7 para resolver este problema de manera interactiva.

D.I 11.7-6. Reconsidere el problema 11.3-5a). Use el algoritmo de ramificación y acotamiento de la sección 11.7 para resolver este problema de PE de manera interactiva.

11.7-7. Un taller de producción intermitente fabrica dos productos. Cada unidad del primer producto requiere 3 horas en la máquina 1 y 2 horas en la máquina 2. Cada unidad del segundo producto requiere 2 horas en la máquina 1 y 3 horas en la 2. La máquina 1 está disponible 8 horas al día y la 2 sólo 7 horas al día. La ganancia por unidad vendida del primer producto es de 16 y la del segundo, de 10. La cantidad diaria que se fabrica de cada producto debe ser un múltiplo entero de 0.25. El objetivo es determinar la mezcla de las cantidades de producción que maximizará la ganancia.

- Formule un modelo de PE para resolver este problema.
- Resuelva este modelo de manera gráfica.
- Utilice el análisis gráfico para aplicar el algoritmo de ramificación y acotamiento de PEM de la sección 11.7 para resolver el modelo.

D.I d) Aplique el algoritmo interactivo del IOR Tutorial para resolver el modelo.

C e) Verifique sus respuestas de los incisos b), c) y d) con una rutina automática para resolver el modelo.

D.I 11.7-8. Use el algoritmo de ramificación y acotamiento de PEM de la sección 11.7 para resolver el siguiente problema de PEM de manera interactiva.

$$\text{Maximizar } Z = 20x_1 + 10x_2 + 25x_3 + 20x_4,$$

sujeta a

$$\begin{aligned} x_1 + x_2 + x_3 + 2x_4 &\leq 12 \\ 3x_1 + x_2 + 2x_3 + 2x_4 &\leq 20 \\ x_1 + 2x_2 + 5x_3 + 3x_4 &\leq 30 \end{aligned}$$

y

$$\begin{aligned} x_j &\geq 0, & \text{para } j = 1, 2, 3, 4 \\ x_j &\text{ es entero,} & \text{para } j = 1, 2, 3. \end{aligned}$$

D.I 11.7-9. Utilice el algoritmo de ramificación y acotamiento de PEM de la sección 11.7 para resolver el siguiente problema en forma interactiva.

$$\text{Maximizar } Z = 3x_1 + 4x_2 + 2x_3 + x_4 + 2x_5,$$

sujeta a

$$\begin{aligned} 2x_1 - x_2 + x_3 + x_4 + x_5 &\leq 3 \\ -x_1 + 3x_2 + x_3 - x_4 - 2x_5 &\leq 2 \\ 2x_1 + x_2 - x_3 + x_4 + 3x_5 &\leq 1 \end{aligned}$$

y

$$\begin{aligned} x_j &\geq 0, & \text{para } j = 1, 2, 3, 4, 5 \\ x_j &\text{ es binario,} & \text{para } j = 1, 2, 3. \end{aligned}$$

D.I 11.7-10. Utilice el algoritmo de ramificación y acotamiento de PEM de la sección 11.7 para resolver el siguiente problema de manera interactiva.

$$\text{Minimizar } Z = 5x_1 + x_2 + x_3 + 2x_4 + 3x_5,$$

sujeta a

$$\begin{aligned} x_2 - 5x_3 + x_4 + 2x_5 &\geq -2 \\ 5x_1 - x_2 &+ x_5 \geq 7 \\ x_1 + x_2 + 6x_3 + x_4 &\geq 4 \end{aligned}$$

y

$$\begin{aligned} x_j &\geq 0, & \text{para } j = 1, 2, 3, 4, 5 \\ x_j &\text{ es entero,} & \text{para } j = 1, 2, 3. \end{aligned}$$

11.8-1.* Utilice cada una de las siguientes restricciones de PEB pura para fijar tantas variables como sea posible.

- $4x_1 + x_2 + 3x_3 + 2x_4 \leq 2$
- $4x_1 - x_2 + 3x_3 + 2x_4 \leq 2$
- $4x_1 - x_2 + 3x_3 + 2x_4 \geq 7$

11.8-2. Utilice cada una de las siguientes restricciones de PEB pura para fijar tantas variables como sea posible.

- $20x_1 - 7x_2 + 5x_3 \leq 10$
- $10x_1 - 7x_2 + 5x_3 \geq 10$
- $10x_1 - 7x_2 + 5x_3 \leq -1$

11.8-3. Use el conjunto de restricciones para el *mismo* problema de PEB pura para fijar tantas variables como sea posible. También identifique las restricciones que se convierten en redundantes al fijar una variable.

$$\begin{aligned} 3x_3 - x_5 + x_7 &\leq 1 \\ x_2 + x_4 + x_6 &\leq 1 \\ x_1 - 2x_5 + 2x_6 &\geq 2 \\ x_1 + x_2 - x_4 &\leq 0 \end{aligned}$$

11.8-4. Identifique cuáles restricciones de problemas de PE pura se convierten en redundantes debido a las restricciones binarias. Explique por qué son o no redundantes.

- $2x_1 + x_2 + 2x_3 \leq 5$
- $3x_1 - 4x_2 + 5x_3 \leq 5$
- $x_1 + x_2 + x_3 \geq 2$
- $3x_1 - x_2 - 2x_3 \geq -4$

11.8-5. En la sección 11.8, al final de la subsección sobre estrechar las restricciones, se indicó que la restricción $4x_1 - 3x_2 + x_3 + 2x_4 \leq 5$ se pueden estrechar a $2x_1 - 3x_2 + x_3 + 2x_4 \leq 3$ y después a $2x_1 - 2x_2 + x_3 + 2x_4 \leq 3$. Use el procedimiento para estrechar las restricciones para confirmar los resultados.

11.8-6. Aplique el procedimiento para *estrechar las restricciones* a la restricción de un problema de PEB pura.

$$5x_1 - 10x_2 + 15x_3 \leq 15.$$

11.8-7. Aplique el procedimiento para *estrechar las restricciones* a la restricción de un problema de PEB pura:

$$x_1 - x_2 + 3x_3 + 4x_4 \geq 1.$$

11.8-8. Aplique el procedimiento para *estrechar las restricciones* a las restricciones de un problema de PEB pura:

- $x_1 + 3x_2 - 4x_3 \leq 2$
- $3x_1 - x_2 + 4x_3 \geq 1$

11.8-9. En la sección 11.8 se utilizó un ejemplo de PEB pura con la restricción $2x_1 + 3x_2 \leq 4$ para ilustrar cómo estrechar restriccio-

nes. Demuestre que al aplicar el procedimiento para generar planos cortantes a esta restricción se llega a la misma nueva restricción $x_1 + x_2 \leq 1$.

11.8-10. Una de las restricciones de cierto problema de PEB puros

$$x_1 + 3x_2 + 2x_3 + 4x_4 \leq 5.$$

Identifique todas sus cubiertas mínimas y proporcione los planos cortantes correspondientes.

11.8-11. Una de las restricciones de cierto problema de PEB puros

$$25x_1 + 15x_2 + 20x_3 + 10x_4 \leq 35.$$

Identifique todas las cubiertas mínimas para ella y proporcione los planos cortantes correspondientes.

11.8-12. Genere tantos planos cortantes como sea posible a partir de la siguiente restricción de un problema de PEB puro.

$$3x_1 + 5x_2 + 4x_3 + 8x_4 \leq 10.$$

11.8-13. Genere tantos planos cortantes como sea posible a partir de la siguiente restricción de un problema de PEB puro:

$$5x_1 + 3x_2 + 7x_3 + 4x_4 + 6x_5 \leq 9.$$

11.8-14. Considere el siguiente problema de PEB puro.

$$\begin{aligned} \text{Maximizar } Z = & 2x_1 + 3x_2 + x_3 + 4x_4 + 3x_5 \\ & + 2x_6 + 2x_7 + x_8 + 3x_9, \end{aligned}$$

sujeta a

$$\begin{aligned} 3x_2 + x_4 + x_5 & \geq 3 \\ x_1 + x_2 & \leq 1 \\ x_2 + x_4 - x_5 - x_6 & \leq -1 \\ x_2 + 2x_6 + 3x_7 + x_8 + 2x_9 & \geq 4 \\ -x_3 + 2x_5 + x_6 + 2x_7 - 2x_8 + x_9 & \leq 5 \end{aligned}$$

y

todo x_j es binario.

Desarrolle la formulación más estrecha posible por medio de las técnicas del procesamiento automático del problema (fijar variables, eliminar restricciones redundantes y estrechar restricciones). Utilice la formulación estrecha para determinar una solución óptima por inspección.

11.9-1. Considere el siguiente problema.

$$\text{Maximizar } Z = 10x_1 + 30x_2 + 40x_3 + 30x_4,$$

sujeta a

$$\begin{aligned} x_1 \in \{2, 3\}, x_2 \in \{2, 4\}, x_3 \in \{3, 4\}, x_4 \in \{1, 2, 3, 4\}, \\ \text{todas estas variables deben tener valores diferentes,} \\ x_1 + x_2 + x_3 + x_4 \leq 10. \end{aligned}$$

Use las técnicas de programación de restricciones (reducción del dominio, propagación de restricciones, procedimiento de búsqueda y enumeración) para identificar todas las soluciones factibles y después encontrar una solución óptima. Muestre su trabajo.

11.9-2. Considere el siguiente problema.

$$\begin{aligned} \text{Maximizar } Z = & 5x_1 - x_1^2 + 8x_2 - x_2^2 + 10x_3 - x_3^2 + 15x_4 \\ & - x_4^2 + 20x_5 - x_5^2, \end{aligned}$$

sujeta a

$$\begin{aligned} x_1 \in \{3, 6, 12\}, x_2 \in \{3, 6\}, x_3 \in \{3, 6, 9, 12\}, \\ x_4 \in \{6, 12\}, x_5 \in \{9, 12, 15, 18\}, \\ \text{todas estas variables deben tener valores diferentes,} \\ x_1 + x_3 + x_4 \leq 25. \end{aligned}$$

Utilice las técnicas de programación de restricciones (reducción del dominio, propagación de restricciones, procedimiento de búsqueda y enumeración) para identificar todas las soluciones factibles y después encontrar una solución óptima. Muestre su trabajo.

11.9-3. Considere el siguiente problema.

$$\begin{aligned} \text{Maximizar } Z = & 100x_1 - 3x_1^2 + 400x_2 - 5x_2^2 + 200x_3 \\ & - 4x_3^2 + 100x_4 - 2x_4^4, \end{aligned}$$

sujeta a

$$\begin{aligned} x_1 \in \{25, 30\}, x_2 \in \{20, 25, 30, 35, 40, 50\}, \\ x_3 \in \{20, 25, 30\}, x_4 \in \{20, 25\}, \\ \text{todas estas variables deben tener valores diferentes,} \\ x_2 + x_3 \leq 60, \\ x_1 + x_3 \leq 50. \end{aligned}$$

Utilice las técnicas de programación de restricciones (reducción del dominio, propagación de restricciones, procedimiento de búsqueda y enumeración) para identificar todas las soluciones factibles y después encontrar una solución óptima. Muestre su trabajo.

11.9-4. Considere el ejemplo de la Job Shop Co., que se presentó en la sección 8.3. En la tabla 8.25 se muestra su formulación como un problema de asignación. Use *restricciones globales* para formular un modelo de programación de restricciones compacto para este problema de asignación.

11.9-5. Considere el problema de asignar nadadores a las diferentes pruebas del equipo de natación que se presentó en el problema 8.3-4. En las respuestas al final del libro se muestra la formulación de este problema como uno de asignación. Utilice *restricciones globales* para formular un modelo de programación de restricciones compacto de este problema de asignación.

11.9-6. Considere el problema en el que se trata de determinar el mejor plan de horas dedicadas al estudio para cada uno de cuatro exámenes finales, que se presentó en el problema 10.3-3. Formule un modelo de programación de restricciones compacto de este problema.

11.9-7. El problema 10.3-2 señala que el propietario de una cadena de tres supermercados necesita determinar cuántas cargas de fresas frescas debe asignar a cada una de las tiendas. Formule un modelo de programación de restricciones compacto para representar este problema.

11.9-8. Una característica poderosa de la programación de restricciones es que las variables se pueden usar como subíndices de los términos de la función objetivo. Por ejemplo, considere el siguiente *problema del agente viajero*. El agente de ventas necesita visitar cada una de las n ciudades (ciudad 1, 2, ..., n) sólo una vez, si comienza en

la ciudad 1 (su lugar de residencia) y regresa a la ciudad 1 después de completar el viaje. Sea c_{ij} la distancia desde la ciudad i hasta la ciudad j para $i, j = 1, 2, \dots, n$ ($i \neq j$). El objetivo es determinar cuál ruta debe seguir el vendedor para minimizar la distancia total del viaje. (Como se explica en el capítulo 13, este problema del agente viajero es un famoso problema clásico de IO con muchas aplicaciones que no tienen nada que ver con los agentes de ventas.)

Si la variable de decisión x_j ($j = 1, 2, \dots, n, n+1$) denota la j -ésima ciudad visitada por el agente viajero, donde $x_1 = 1$ y $x_{n+1} = 1$, la programación de restricciones permite escribir el objetivo como

$$\text{Minimizar } Z = \sum_{j=1}^n c_{x_j x_{j+1}}.$$

Utilice esta función objetivo para formular un modelo completo de programación de restricciones para representar el problema.

11.10-1. De la parte inferior de las referencias seleccionadas que se proporcionaron al final del capítulo, seleccione una de las aplicaciones de la programación entera ganadora de premios. Lea este artículo y después escriba un resumen de dos páginas de la aplicación y los beneficios que brinda (incluya los no financieros).

11.10-2. De la parte inferior de las referencias seleccionadas que se proporcionaron al final del capítulo, seleccione tres de las aplicaciones de la programación entera ganadora de premios. Léalos y después escriba un resumen de una página de cada aplicación y los beneficios que brinda (incluya los no financieros).

CASOS

Caso 11.1 Aspectos de capacidad

Bentley Hamilton coloca la sección de negocios de *The New York Times* sobre la mesa de conferencias y observa cómo sus asociados se mueven incómodos en sus sillones.

El señor Hamilton desea que entiendan algo.

Pone la primera plana de *The Wall Street Journal* junto a la de *The New York Times* y observa a sus asociados abrir los ojos que ya no muestran aburrimiento.

El señor Hamilton quiere que entiendan bien.

Después arroja la primera página del *Financial Times* encima de los otros y observa que sus asociados comienzan a sudar.

El señor Hamilton quiere que lo que ocurre se grave en las mentes de sus asociados.

“Les acabo de presentar tres periódicos financieros líderes con los chismes de negocios de hoy”, declara con voz irritada. “Queridos asociados, ¡nuestra compañía se está yendo a pique! ¿Debo leerles los encabezados? De *The New York Times*: ‘CommuniCorp baja al mínimo en 52 semanas’. De *The Wall Street Journal*: ‘CommuniCorp pierde 25% del mercado de localizadores en sólo un año’. ¡Ah!, y en mi favorito, el *Financial Times*: ‘CommuniCorp no puede ComuniCar: las acciones de CommuniCorp bajan debido a problemas de comunicación internos’. ¿Por qué la compañía cayó en una situación tan terrible?”

El señor Hamilton presenta una diapositiva que muestra una recta con una pendiente ligera hacia arriba. “Ésta es una gráfica de nuestra productividad en los últimos 12 meses. Como pueden ver, la productividad de la planta de localizadores ha aumentado en el último año. Es obvio que la productividad no es la causa del problema.”

El señor Hamilton muestra una segunda diapositiva de una recta con una pendiente pronunciada hacia arriba. “Ésta es la gráfica de las órdenes que entregamos tarde o perdimos en los últimos 12 meses.” Se escuchan movimientos incómodos de los asociados. “Como pueden ver en la gráfica, las órdenes tardías o

perdidas han aumentado de manera firme y significativa en los últimos 12 meses. Creo que esta tendencia explica por qué ha caído nuestro porcentaje de mercado a su nivel más bajo en 52 semanas. Hemos hecho enojar y perdido a los distribuidores, clientes que dependían de entregas a tiempo para cumplir con las demandas de sus clientes.”

“¿No cumplimos con las fechas de entrega cuando nuestro nivel de productividad debió permitir cumplir con las órdenes?”, pregunta el señor Hamilton. “Hice esta pregunta a varios departamentos.”

“¡Resulta que hemos producido localizadores por producirlos!”, dice sin poderlo creer. “Los departamentos de mercadotecnia y ventas no se comunican con el de manufactura, por lo que los ejecutivos de éste último departamento no saben qué tipos de localizadores deben producir para cumplir con las órdenes. Ellos quieren que la planta siga trabajando, y producen localizadores sin importar si los han ordenado. Los localizadores terminados se envían al almacén, pero los ejecutivos de ventas y mercadotecnia no saben qué cantidades y estilos hay en él. Tratan de comunicarse con los ejecutivos del almacén para determinar si los localizadores inmovilizados en el inventario pueden satisfacer las órdenes, pero rara vez reciben una respuesta a sus preguntas.”

El señor Hamilton hace una pausa y mira directo a sus asociados. “Señores y señoras, me parece que tenemos un problema de comunicación interna serio. Intento corregir este problema de inmediato. Quiero iniciar la instalación de una red de cómputo en toda la compañía para asegurar que todos los departamentos tengan acceso a documentos críticos y puedan comunicarse con facilidad entre sí por el correo electrónico. Como esta red interna (o intranet) representará un cambio importante en la infraestructura actual de comunicación, espero algunas fallas en el sistema y cierta resistencia de los empleados. Por todo ello, quiero que la instalación de la red se haga por etapas.”

El señor Hamilton pasa las siguientes tablas de programación y requerimientos a sus asociados (donde IN = intranet).

Mes 1	Mes 2	Mes 3	Mes 4	Mes 5
Capacitación IN	Instalar IN en Ventas	Instalar IN en Manufactura	Instalar IN en Almacén	Instalar IN en Mercadotecnia

Departamento	Número de empleados
Ventas	60
Manufactura	200
Almacén	30
Comercialización	75

El señor Hamilton procede a explicar la tabla de tiempos y requerimientos. “No quiero que en el primer mes se hagan conexiones a la red; sólo deseo diseminar la información al respecto y tratar de que los empleados la acepten. El segundo mes se conectará el departamento de ventas, ya que recibe toda la información crítica de nuestros clientes. El tercer mes se conec-

tará el departamento de manufactura. Durante el cuarto mes se instalará la red en el almacén y en el quinto y último quiero incluir al departamento de mercadotecnia en la red. La tabla de requerimientos que sigue a la programación contiene el número de empleados que requieren acceso a la red en cada departamento.” Después se dirige a Emily Jones, jefa de administración de información corporativa. “Necesito tu ayuda para planear la instalación de la red. En particular, la compañía necesita comprar servidores para la red interna. Los empleados se conectarán a los servidores y bajarán información a sus computadoras.” El señor Hamilton entrega a Emily la siguiente tabla con detalles de los tipos de servidores disponibles, el número de empleados que acepta cada servidor y los costos.

Tipo de servidor	Número de empleados que puede atender el servidor	Costo del servidor
PC con Intel Pentium estándar	Hasta 30 empleados	\$ 2 500
PC con Intel Pentium mejorado	Hasta 80 empleados	\$ 5 000
Estación de trabajo SGI	Hasta 200 empleados	\$10 000
Estación de trabajo Sun	Hasta 2 000 empleados	\$25 000

“Emily, necesito que decidas qué servidores comprar y cuándo comprarlos para minimizar el costo y asegurar que la compañía tiene suficiente capacidad de servidores para la implantación de la red interna a tiempo”, dice el señor Hamilton. “Por ejemplo, puedes decidir comprar un servidor grande el primer mes para que acepte a todos los empleados o comprar un servidor pequeño cada mes de manera que un nuevo grupo de empleados tenga acceso a la red.” “Existen varios factores que complican tu decisión”, continúa el señor Hamilton: “Dos fabricantes de servidores están dispuestos a ofrecer descuentos a CommuniCorp. SGI nos dará un descuento de 10% en cada servidor que compremos, pero sólo si se compra durante el primero o segundo meses. Sun está dispuesto a dar 25% de descuento en todos los servidores comprados los dos primeros meses. También tienes limitaciones en la cantidad de dinero que puedes gastar el primer mes. CommuniCorp ya ha asignado el presupuesto para los próximos dos meses, por lo cual sólo cuentas con un total de \$9 500 para las compras en los meses 1 y 2. Por último, el departamento de manufactura requiere al menos uno de los tres servidores más poderosos. Quiero tener tu decisión en mi escritorio para el fin de semana.”

- a) En primer lugar, Emily decide evaluar el número y tipo de servidores que va a comprar cada mes. Formule un modelo de PE para determinar qué servidores debe comprar Emily cada mes para minimizar los costos de ese periodo y dar el apoyo a los nuevos usuarios. ¿Cuántos servidores y de qué tipos debe comprar cada mes? ¿Cuál es el costo total del plan?
- b) Emily se da cuenta de que tal vez logre algunos ahorros si compra un servidor más grande en los primeros meses para dar acceso a los usuarios en los últimos meses. Por lo tanto, decide evaluar el número y tipo de servidores que debe comprar durante todo el horizonte de planeación. Formule un modelo de PE para determinar qué servidores debe comprar Emily y en qué meses, de manera que se minimice el costo total y se dé acceso a todos los nuevos usuarios. ¿Cuántos servidores y de qué tipos debe comprar cada mes? ¿Cuál es el costo total del plan?
- c) ¿Por qué es diferente la respuesta que se obtiene según el primer método que la que ofrece el segundo?
- d) ¿Existen otros costos que Emily no ha tomado en cuenta en la formulación de su modelo? Si es así, ¿cuáles son?
- e) ¿Qué otros aspectos pueden preocupar a los distintos departamentos de CommuniCorp respecto de la red interna?

■ RESUMEN DE LOS CASOS ADICIONALES EN EL SITIO EN INTERNET DE ESTE LIBRO (www.mhhe.com/hillier)

CASO 11.2 Asignación de arte

Se realizan planes para llevar a cabo una exposición de artistas modernos emergentes en el museo de arte moderno de San Francisco. Se ha recopilado una larga lista de posibles artistas, sus obras disponibles y los precios nominales de estas obras. También existen varias restricciones en relación con la mezcla de obras que se pueden elegir. El siguiente paso es aplicar PEB para hacer la selección de las obras para la exposición bajo tres escenarios diferentes.

CASO 11.3 Juegos de cocina en almacén

Una mala administración de inventarios en el almacén local de Furniture City ha causado la acumulación excesiva de muchos artículos y el déficit de algunos otros. Para comenzar a rectificar esta situación, se han identificado los 20 juegos de cocina más populares en el departamento correspondiente. Estos juegos se componen de hasta ocho características en diversos estilos, de forma que cada uno de dichos estilos debe tener un correcto

nivel de inventario en el almacén. Sin embargo, la cantidad limitada de espacio de almacenaje asignado al departamento de cocina significa que se deben tomar algunas decisiones difíciles en este aspecto. Después de recopilar los datos relevantes de los 20 juegos de cocina, ahora es necesario aplicar PEB para determinar qué cantidad de características y estilos de Furniture City se deben conservar en el almacén local bajo tres escenarios diferentes.

CASO 11.4 Asignación de estudiantes a escuelas (de nuevo)

Como se presentó en el caso 4.3 y se revisó en el caso 6.3, la oficina escolar de Springfield necesita asignar a los estudiantes de educación media de las seis áreas residenciales de la ciudad en las tres escuelas que aún se conservan. La nueva complicación de la oficina escolar es que ha decidido no permitir que los estudiantes de una misma área se dividan entre distintas escuelas. Por lo tanto, como cada una de las seis áreas debe ser asignada a una sola escuela, ahora se debe aplicar PEB para hacer estas asignaciones bajo los distintos escenarios considerados en el caso 4.3.

12

CAPÍTULO

Programación no lineal

El papel fundamental de la programación lineal en IO se refleja con exactitud en el hecho de que es el tema central de *un tercio* de este libro. Un supuesto importante de programación lineal es que *todas sus funciones* (objetivo y de restricción) son lineales. Aunque, en esencia, este supuesto se cumple en el caso de muchos problemas prácticos, con frecuencia no es así. Por lo tanto, muchas veces es necesario manejar problemas de programación no lineal, que es el área importante en la que se enfocará la atención en este capítulo.

De manera general,¹ el *problema de programación no lineal* consiste en encontrar $\mathbf{x} = (x_1, x_2, \dots, x_n)$ para

$$\text{maximizar } f(\mathbf{x}),$$

sujeta a

$$g_i(\mathbf{x}) \leq b_i, \quad \text{para } i = 1, 2, \dots, m,$$

y

$$\mathbf{x} \geq \mathbf{0},$$

donde $f(\mathbf{x})$ y $g_i(\mathbf{x})$ son funciones dadas de n variables de decisión.²

Existen muchos tipos de problemas de programación no lineal, lo cual depende de las características de las funciones $f(\mathbf{x})$ y $g_i(\mathbf{x})$. Se emplean varios algoritmos para resolver los diferentes tipos de problemas. Para ciertos tipos donde las funciones tienen formas simples, los problemas pueden resolverse de manera relativamente eficiente. En algunos otros casos, incluso la solución de pequeños problemas representa un verdadero reto.

Debido a que existen muchos tipos de problemas y muchos algoritmos, la programación no lineal es un tema muy extenso. No se cuenta con el espacio suficiente para realizar un estudio completo. Sin embargo, se presentarán algunos ejemplos de aplicación y después se introducirán algunas ideas básicas para resolver ciertos tipos importantes de problemas de programación no lineal.

En los apéndices 2 y 3 se proporcionan antecedentes útiles para este capítulo y se recomienda consultarlos al estudiar las secciones que siguen.

12.1 APLICACIONES DE MUESTRA

Los siguientes ejemplos ilustran unos cuantos de los muchos tipos importantes de problemas a los que se aplica la programación no lineal.

¹ Las otras *formas legítimas* corresponden a aquellas para *programación lineal* que se mencionan en la sección 3.2. La sección 4.6 describe cómo convertir estas otras formas en la forma que se proporciona aquí.

² Para simplificar, se supondrá a lo largo del capítulo que *todas* estas funciones o bien son *diferenciables* en todas partes o son funciones lineales por partes (tema que se estudió en las secciones 12.1 y 12.8).

Problema de mezcla de productos con elasticidad de precios

En problemas de *mezcla de productos*, como el de la Wyndor Glass Co., de la sección 3.1, la meta es determinar la mezcla óptima de los niveles de elaboración de los productos de una empresa, dadas las limitaciones sobre los recursos que se necesitan para manufacturarlos, con el objeto de maximizar la ganancia total de la empresa. En algunos casos existe una ganancia unitaria fija asociada a cada producto, con lo cual la función objetivo que se obtiene es lineal. Sin embargo, en muchos problemas de mezcla de productos, ciertos factores introducen no linealidades en la función objetivo.

Por ejemplo, un gran fabricante puede encontrar *elasticidad de precios*, es decir, que la cantidad de un producto que se puede vender tiene relación inversa con el precio que se cobra por él. Así, la *curva precio-demanda* de un producto puede parecerse a la que se muestra en la figura 12.1, donde $p(x)$ es el precio que se necesita para poder vender x unidades. Como sabemos, la ganancia de la empresa por producir y vender x unidades es el ingreso que generan las ventas $xp(x)$ menos los costos de elaboración y distribución. Por lo tanto, si el costo unitario de producir y distribuir el artículo está fijo en c (vea la línea punteada de la figura 12.1), la ganancia de la empresa por producir y vender x unidades está dada por la función no lineal

$$P(x) = xp(x) - cx,$$

como se puede observar en la figura 12.2. Si *cada uno* de los n productos de la empresa tiene una función de ganancia parecida, por ejemplo $P_j(x_j)$ por producir y vender x_j unidades del producto j ($j = 1, 2, \dots, n$), entonces la función objetivo global es

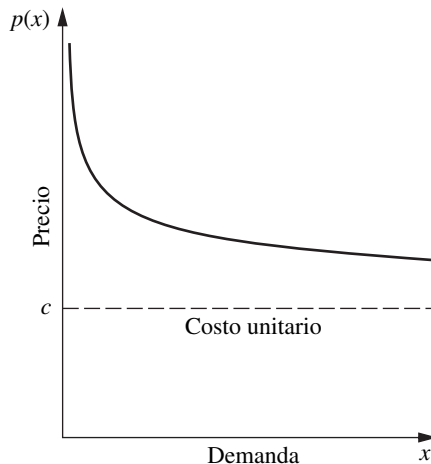
$$f(\mathbf{x}) = \sum_{j=1}^n P_j(x_j),$$

esto es, una suma de funciones no lineales.

Otra razón por la que pueden surgir no linealidades en la función objetivo es que el *costo marginal* de producir otra unidad más de un producto dado puede variar según el nivel de elaboración. Por ejemplo, el costo marginal puede disminuir cuando aumenta el nivel de producción, gracias al *efecto de curva de aprendizaje* (mayor eficiencia a medida que aumenta la experiencia). Por otro lado, pueden aumentar a causa de ciertas medidas especiales, como tiempos extra o instalaciones fabriles más costosas, necesarias para aumentar la producción.

La no linealidad también puede surgir en las funciones de restricción $g_i(\mathbf{x})$ en forma bastante parecida. Por ejemplo, si existe una restricción de presupuesto sobre el costo total de producción, la función de costo será no lineal si el costo marginal de producción varía como se acaba de describir. En el caso de restricciones sobre otro tipo de recursos, $g_i(\mathbf{x})$ será no lineal siempre que el uso del recurso correspondiente no sea estrictamente proporcional a los niveles de los respectivos productos.

■ FIGURA 12.1
Curva precio-demanda.



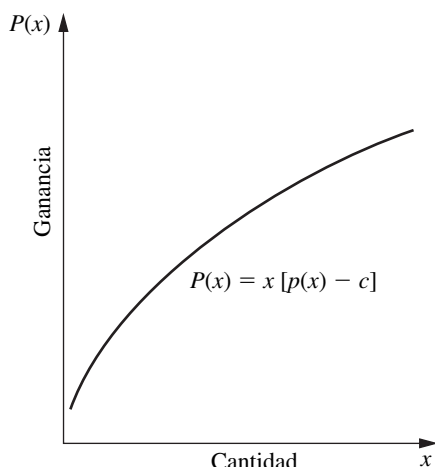
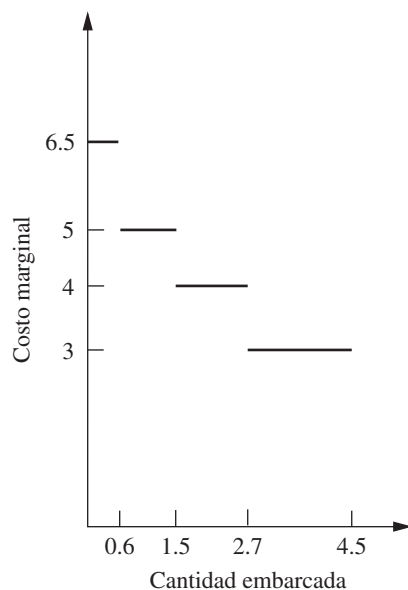


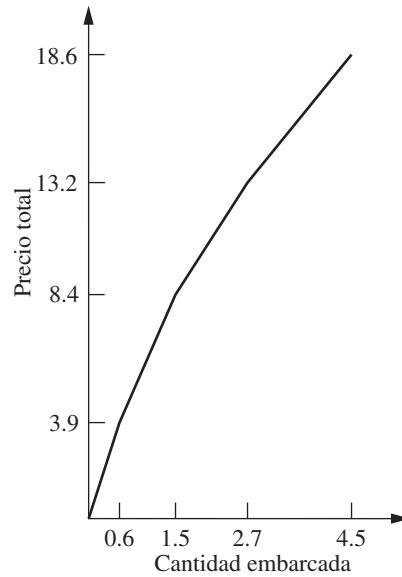
Figura 12.2
Función de ganancia.

Problema de transporte con descuento de precios por volumen de embarque

Como se ilustró en el ejemplo de la P & T Company, en la sección 8.1, una aplicación común del problema de transporte es determinar un plan óptimo para enviar bienes desde varios orígenes hasta varios destinos, dadas las restricciones de recursos y demanda, con el fin de minimizar el costo total de transporte. En el capítulo 8 se supuso que el *costo por unidad enviada* de un origen a un destino dados *es fijo*, independientemente de la cantidad que se envíe. En la realidad, este costo puede no ser fijo. A veces se dispone de *descuentos por cantidad* para volúmenes grandes, con lo que el *costo marginal* de enviar una unidad más puede seguir un patrón como el que se muestra en la figura 12.3. En consecuencia, el costo que resulta de embarcar x unidades está dado por la función *no lineal* $C(x)$ que es una *función lineal por partes* con pendiente igual al costo marginal, como la que se muestra en la figura 12.4. [La función de la figura 12.4 consiste en un segmento de recta con pendiente 6.5 desde $(0, 0)$ hasta $(0.6, 3.9)$, un segundo segmento de recta con pendiente 5 desde $(0.6, 3.9)$ hasta $(1.5, 8.4)$, un tercer segmento de recta con pendiente 4 desde $(1.5, 8.4)$ hasta $(2.7, 13.2)$ y uno más con pendiente 3 de $(2.7, 13.2)$ hasta $(4.5, 18.6)$]. En consecuencia, si *cada*

■ **FIGURA 12.3**
Costo marginal de transporte.





■ **FIGURA 12.4**
Función del costo de transporte.

combinación de origen y destino tiene una función de costos similar, es decir, si el costo de enviar x_{ij} unidades del origen i ($i = 1, 2, \dots, m$) al destino j ($j = 1, 2, \dots, n$) está dado por una función no lineal $C_{ij}(x_{ij})$, entonces la función objetivo global que se va a minimizar es

$$f(\mathbf{x}) = \sum_{i=1}^m \sum_{j=1}^n C_{ij}(x_{ij}).$$

Aun con esta función objetivo no lineal, es normal que las restricciones sean del tipo lineales especiales que se ajustan al modelo del problema de transporte de la sección 8.1.

Selección de una cartera de inversiones riesgosas

En la actualidad, una práctica común entre los administradores de grandes carteras de inversión es usar, como guía, modelos de computadora basados en parte en programación no lineal. Debido a que los inversionistas se preocupan tanto por el *rendimiento esperado* (ganancia), como por el *riesgo* asociado con su inversión, la programación no lineal se emplea para determinar una cartera que, bajo ciertos supuestos, proporcione un trueque óptimo entre estos dos factores. Este enfoque se basa en gran parte en la novedosa investigación realizada por Harry Markowitz y William Sharpe, que los ayudó a ganar el Premio Nobel de Economía en 1990.

Se puede formular un modelo de programación no lineal para representar este problema de la siguiente manera. Suponga que se están tomando en cuenta n tipos de acciones para incluirlas en la cartera; las variables de decisión x_j ($j = 1, 2, \dots, n$) representan el número de acciones j que se van a incluir. Sean μ_j y σ_{jj} la *media* y la *varianza* (estimadas) del rendimiento de cada acción del tipo j , donde σ_{jj} mide el riesgo de estas acciones. Para $i = 1, 2, \dots, n$ ($i \neq j$), sea σ_{ij} la *covarianza* del rendimiento de una acción de cada tipo i y tipo j . (Como sería difícil estimar todas las σ_{ij} , lo usual es basarse en ciertos supuestos sobre el comportamiento del mercado que permitan calcular σ_{ij} directamente, a partir de σ_{ii} y σ_{jj} .) Entonces, el valor esperado de $R(\mathbf{x})$ y la varianza $V(\mathbf{x})$ del rendimiento total de la cartera completa son

$$R(\mathbf{x}) = \sum_{j=1}^n \mu_j x_j$$

y

$$V(\mathbf{x}) = \sum_{i=1}^n \sum_{j=1}^n \sigma_{ij} x_i x_j,$$

Recuadro de aplicación

Bank Hapoalim Group es la corporación bancaria más grande de Israel, y brinda servicios en ese país a través de una red de 327 sucursales, nueve centros de negocios regionales y varias subsidiarias nacionales. También realiza operaciones a nivel mundial a través de 37 sucursales, oficinas y subsidiarias en los centros financieros más importantes de Norte y Sudamérica, así como en Europa.

Una parte fundamental del negocio de Bank Hapoalim radica en brindar asesoría en inversiones a sus clientes. Para conservar ventaja sobre sus competidores, la cúpula administrativa se ha dado a la tarea de realizar un programa de reestructuración para ofrecer a dichos asesores en inversiones la tecnología y metodología más moderna. Por esta razón se integró un grupo de investigación de operaciones.

Este grupo llegó a la conclusión de que necesitaba desarrollar un sistema de soporte a las decisiones que fuera flexible para los asesores en inversiones que pudiera ser adaptable para satisfacer las diferentes necesidades de cada cliente. A cada uno de estos clientes se les solicitó proporcionar una gran cantidad de información acerca de sus necesidades, lo que incluía la elección entre las diferentes alternativas con respecto a sus objetivos de inversión, horizonte de inversión, selección de un índice para esforzarse a exceder la meta, preferencia con respecto a la liquidez y tipo de cambio, etc. También se les planteó una serie de preguntas con el fin de conocer la clasificación del nivel de riesgo que se atrevían a asumir.

La elección natural del modelo para manejar el sistema de soporte a las decisiones resultante (llamado *Opti-Money System*) fue el *modelo de programación no lineal clásico para la selección del portafolio* descrito en esta sección del libro, con algunas modificaciones que incorporan toda la información acerca de las necesidades de cada uno de los clientes. Este modelo genera una ponderación óptima de 60 posibles clases de activos de fondos y bonos para el portafolio y el asesor de inversiones, en conjunto con el cliente, selecciona los fondos y bonos específicos dentro de esas clases.

En un año reciente, los asesores en inversiones del banco, participaron en 133 000 sesiones de consulta con 63 000 clientes, utilizando este sistema de soporte a las decisiones. Las *ganancias anuales* con respecto a los puntos de referencia de los clientes que siguieron el consejo del sistema sumaron *una total de alrededor de 244 millones de dólares, a la vez que aportaron más de 31 millones de dólares al ingreso anual del banco.*

Fuente: M. Avriel, H. Pri-Zan, R. Meiri y A. Peretz, "Opti-Money at Bank Hapoalim: A Model-Based Investment Decision-Support System for Individual Customers", en *Interfaces*, **34**(1): 39-50, enero-febrero de 2004. (En la página en internet de este libro www.mhhe.com/hillier se proporciona una liga hacia este artículo.)

donde $V(\mathbf{x})$ mide el riesgo asociado con la cartera. Una manera de considerar el trueque entre estos dos factores es usar $V(\mathbf{x})$ como la función objetivo que debe minimizarse y después imponer la restricción de que $R(\mathbf{x})$ no debe ser menor que el rendimiento mínimo aceptable esperado. El modelo de programación no lineal completo sería

$$\text{Minimizar} \quad V(\mathbf{x}) = \sum_{i=1}^n \sum_{j=1}^n \sigma_{ij} x_i x_j,$$

sujeta a

$$\sum_{j=1}^n \mu_j x_j \geq L$$

$$\sum_{j=1}^n P_j x_j \leq B$$

y

$$x_j \geq 0, \quad \text{para } j = 1, 2, \dots, n,$$

donde L es el rendimiento mínimo aceptable esperado, P_j es el precio de cada acción tipo j y B es la cantidad de dinero presupuestada para la cartera.

Uno de los defectos de esta formulación es que existe una dificultad relativa al elegir un valor adecuado de L para obtener la mejor combinación entre $R(\mathbf{x})$ y $V(\mathbf{x})$. Entonces, en lugar de terminar el proceso con una elección de L , es común usar un enfoque de programación (no lineal) *paramétrica* para generar la solución óptima como una función de L en un intervalo amplio de valores; el siguiente paso consiste en analizar los valores de $R(\mathbf{x})$ y $V(\mathbf{x})$ de estas soluciones que son óptimas para algún valor de L y después elegir la solución que dé la mejor opción para estas dos cantidades. Con frecuencia se hace referencia a este procedimiento como la generación de soluciones sobre la *frontera eficiente* de la gráfica de dos dimensiones de los puntos $R(\mathbf{x})$, $V(\mathbf{x})$ para todas las $\mathbf{x}$ factibles.

La razón es que el punto $R(\mathbf{x})$, $V(\mathbf{x})$ para una $\mathbf{x}$ óptima (para alguna L) se encuentra sobre la *frontera* (límite) de los puntos factibles. Más aún, cada punto $\mathbf{x}$ es *eficiente* en el sentido de que ninguna otra solución factible será cuando menos tan buena con una medida (R o V) y estrictamente mejor con la otra medida (V más pequeña o R más grande).

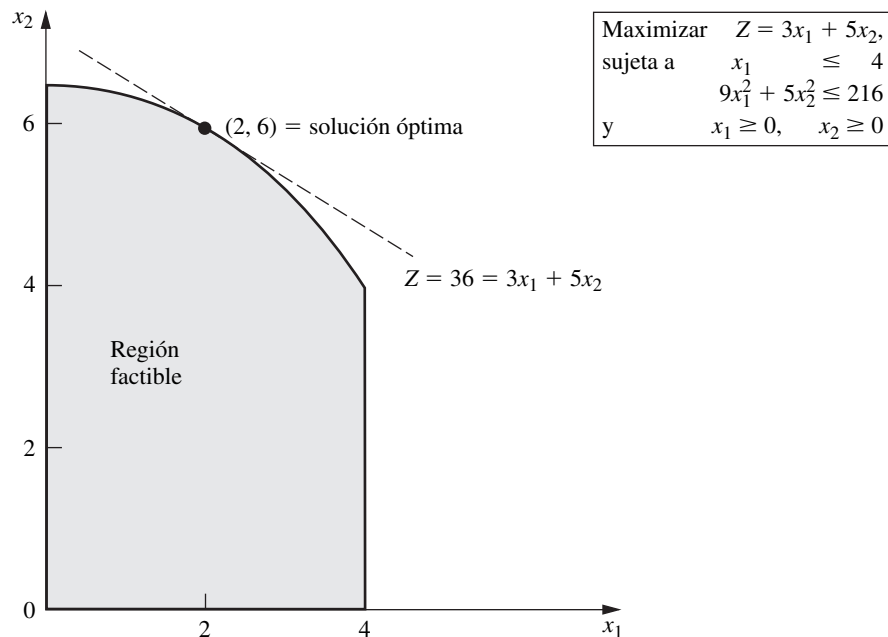
Esta aplicación de la programación no lineal es particularmente importante. En la actualidad, el uso de la programación no lineal para optimizar el portafolio forma una parte modular del análisis financiero moderno. (En palabras más generales, el relativamente novedoso campo de la ingeniería financiera ha surgido con el fin de otorgar la importancia que le corresponde a la aplicación de las técnicas de investigación de operaciones, tales como la programación no lineal, a diferentes problemas financieros, dentro de los que se incluye la optimización del portafolio.) Como lo ilustra este recuadro de aplicación, este tipo de uso de la programación no lineal ha provocado un sorprendente efecto en la práctica. De manera continua se lleva a cabo un gran esfuerzo en el área de investigación para ampliar el conocimiento acerca de las propiedades y las aplicaciones del modelo anterior y los modelos de programación no lineal relacionados en varios tipos de complejos análisis de portafolio.³

12.2 ILUSTRACIÓN GRÁFICA DE PROBLEMAS DE PROGRAMACIÓN NO LINEAL

Cuando un problema de programación no lineal tiene sólo una o dos variables, se puede representar en forma gráfica de una manera muy parecida al ejemplo de la Wyndor Glass Co., de programación lineal, de la sección 3.1. Se verán unos cuantos ejemplos, puesto que una representación gráfica de este tipo proporciona una visión global de las propiedades de las soluciones óptimas de programación lineal y no lineal. Con el fin de hacer hincapié en las diferencias entre programación lineal y no lineal, se usarán algunas variaciones no lineales del problema de la Wyndor Glass Co.

En la figura 12.5 se muestra lo que ocurre con este problema si los únicos cambios que se hacen al modelo de la sección 3.1 son que la segunda y tercera restricciones funcionales se sustituyen por la restricción no lineal $9x_1^2 + 5x_2^2 \leq 216$. Compare las figuras 12.5 y 3.3. La solución óptima sigue

FIGURA 12.5
Ejemplo de la Wyndor Glass Co., con la restricción no lineal $9x_1^2 + 5x_2^2 \leq 216$ en lugar de la segunda y tercera restricción original.



³ A continuación se mencionan investigaciones recientes en este campo. B. I. Jacobs, K. N. Levy y H. M. Markowitz, "Portfolio Optimization with Factors, Scenarios and Realistic Short Positions", en *Operations Research*, **53**(4): 586-599, julio-agosto de 2005; A. F. Siegel y A. Woodgate, "Performance of Portfolios Optimized with Estimation Error", en *Management Science*, **53**(6): 1005-1015, junio de 2007; H. Konno y T. Koshizuka, "Mean-Absolute Deviation Model", en *IEE Transactions*, **37**(10): 893-900, octubre de 2005.

siendo $(x_1, x_2) = (2, 6)$. Aún más, todavía se encuentra sobre la frontera de la región factible, pero *no* es una solución factible en un vértice (FEV). La solución óptima pudo haber sido FEV con una función objetivo diferente (verifique $Z = 3x_1 + x_2$), pero el hecho de que no necesite serlo significa que ya no se puede aprovechar la gran simplificación que se utiliza en programación lineal que permite limitar la búsqueda de una solución óptima para las soluciones FEV.

Ahora suponga que las restricciones lineales de la sección 3.1 se conservan sin cambio, pero que la función objetivo se hace no lineal. Por ejemplo, si

$$Z = 126x_1 - 9x_1^2 + 182x_2 - 13x_2^2,$$

la representación gráfica de la figura 12.6 indica que la solución óptima es $x_1 = \frac{8}{3}$, $x_2 = 5$, que de nuevo se encuentra en la frontera de la región factible. (El valor óptimo de Z es $Z = 857$; así, la figura 12.6 muestra el hecho de que el lugar geométrico de todos los puntos con $Z = 857$ tiene en común con la región factible sólo este punto, mientras que el lugar geométrico de los puntos con Z más grande no toca la región factible en ningún punto.) Por otro lado, si

$$Z = 54x_1 - 9x_1^2 + 78x_2 - 13x_2^2,$$

entonces la figura 12.7 ilustra que la solución óptima es $(x_1, x_2) = (3, 3)$, que se encuentra *dentro* de la frontera de la región factible. (Se puede comprobar que esta solución es óptima si se usa cálculo para derivarla como un máximo global no restringido; como también satisface las restricciones, debe ser óptima para el problema restringido.) Por lo tanto, es necesario que un algoritmo general para resolver problemas de este tipo tome en cuenta *todas* las soluciones en la región factible, y no sólo aquellas que están sobre la frontera.

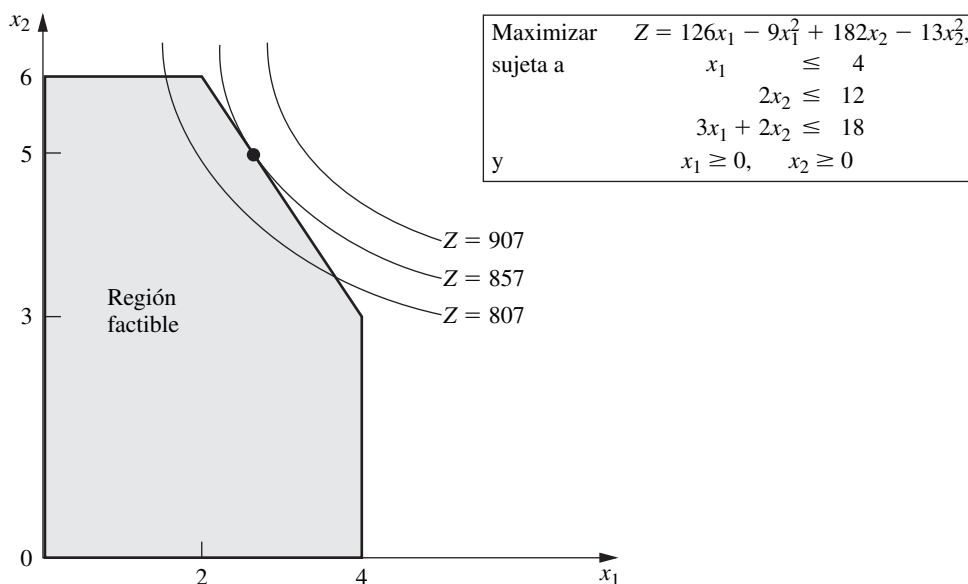
Otra complicación que surge en programación no lineal es que un máximo *local* no necesariamente es un máximo *global* (la solución óptima global). Por ejemplo, considere la función de una sola variable graficada en la figura 12.8. En el intervalo $0 \leq x \leq 5$, esta función tiene tres máximos locales — $x = 0$, $x = 2$ y $x = 4$ — pero sólo uno de éstos — $x = 4$ — es un *máximo global*. (De igual manera, existen mínimos locales en $x = 1$, 3 y 5 , pero sólo $x = 5$ es un *mínimo global*.)

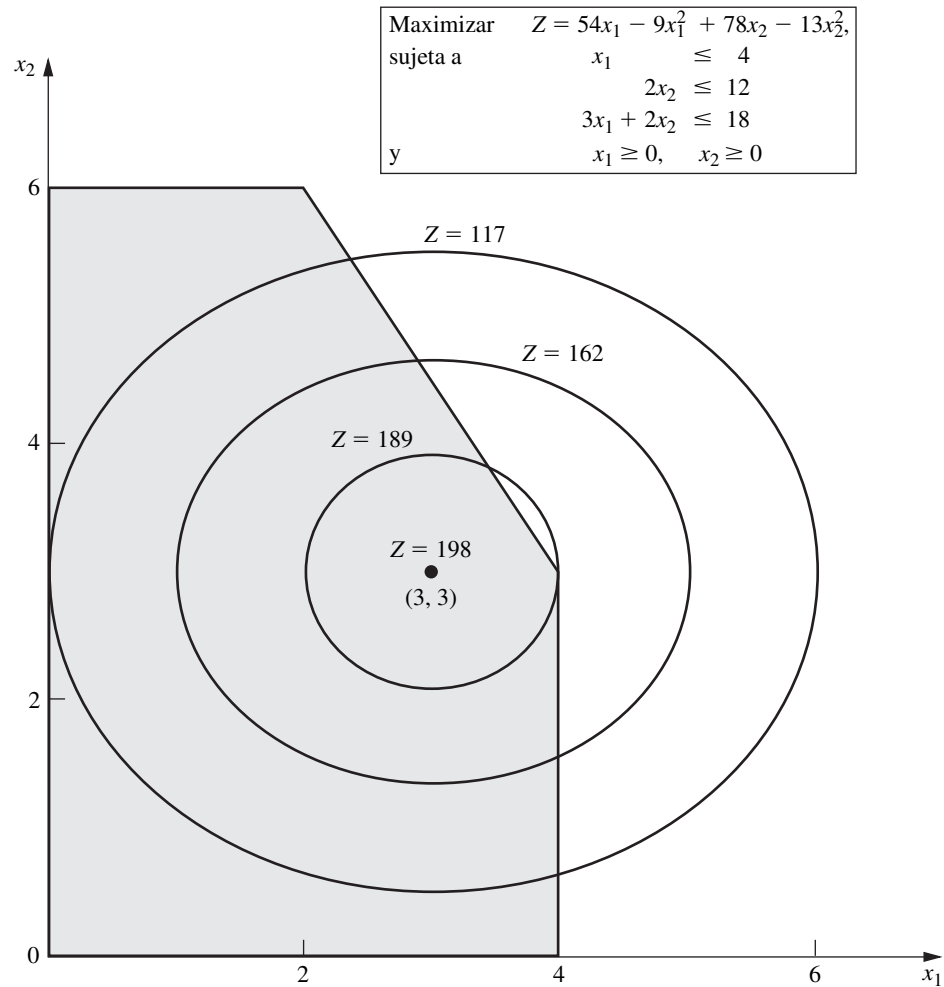
En general, los algoritmos de programación no lineal no pueden distinguir entre un máximo local y un máximo global (excepto si encuentran otro máximo local *mejor*). Por tanto, es crucial conocer las condiciones en las que se *garantiza* que un máximo local es un máximo global en la región factible. Recuerde que en cálculo, cuando se maximiza una función ordinaria (doblemente diferenciable) de una sola variable $f(x)$ sin restricciones, esta garantía está dada cuando

$$\frac{\partial^2 f}{\partial x^2} \leq 0 \quad \text{para toda } x.$$

FIGURA 12.6

Ejemplo de la Wyndor Glass Co., con la región factible original pero con la función objetivo no lineal $Z = 126x_1 - 9x_1^2 + 182x_2 - 13x_2^2$ en lugar de la función objetivo original.





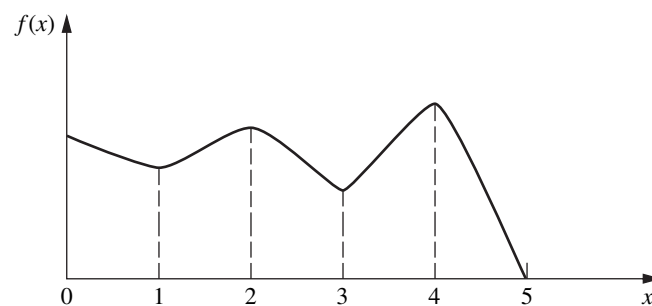
■ FIGURA 12.7

Ejemplo de la Wyndor Glass Co., con la región factible original pero con otra función objetivo no lineal, $Z = 54x_1 - 9x_1^2 + 78x_2 - 13x_2^2$, en lugar de la original.

Una función de este tipo que siempre es “cóncava hacia abajo” (o que no tiene convexidad) se llama función **cóncava**.⁴ De igual manera, si se sustituye $\leq$ por $\geq$, de manera que la función sea siempre “cóncava hacia arriba” (o que no tenga concavidad), se llama función **convexa**.⁵ (Por ello, una función *lineal* puede ser tanto cóncava como convexa.) En la figura 12.9 se pueden ver este tipo de ejemplos. Observe que la figura 12.8 ilustra una función que no es cóncava ni convexa puesto que alterna sus curvaturas hacia arriba y hacia abajo.

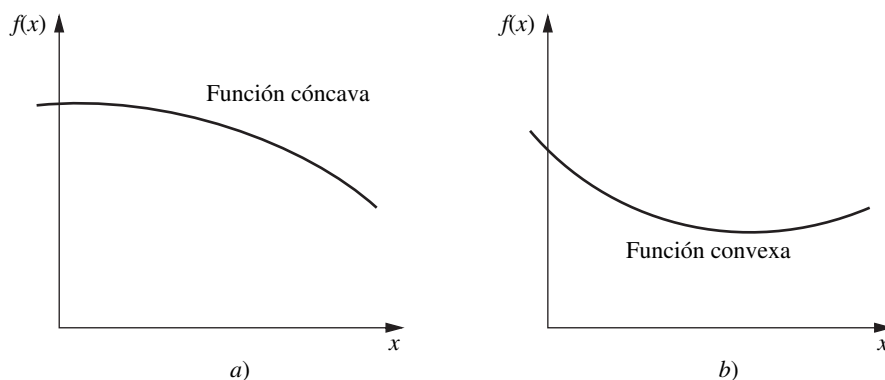
■ FIGURA 12.8

Una función con varios máximos locales ($x = 0, 2, 4$), pero sólo $x = 4$ es un máximo global.



⁴ Las funciones *cóncavas* a veces se llaman *cóncavas hacia abajo*.

⁵ Las funciones *convexas* a veces se llaman *cóncavas hacia arriba*.



■ FIGURA 12.9

Ejemplos de a) una función cóncava y b) una función convexa.

Las funciones de variables múltiples también se pueden caracterizar como cóncavas o convexas si su curvatura es siempre hacia abajo o hacia arriba. Estas definiciones intuitivas se fundamentan en términos precisos que, junto con cierta profundización en los conceptos, se presentan en el apéndice 2. (Las funciones cóncava y convexa tienen un papel fundamental en la programación no lineal, por lo que, si no se está muy familiarizado con estas funciones, se recomienda leer a conciencia el apéndice 2.) Este apéndice también proporciona una prueba conveniente para verificar si una función de dos variables es cóncava, convexa o ninguna de las dos.

La siguiente es una forma práctica de verificar esta característica en el caso de una función de más de dos variables cuando la función consiste en una suma de funciones más pequeñas cada una de sólo una o dos variables. Si cada función más pequeña es cóncava, entonces la función completa es cóncava. De manera similar, la función completa es convexa si cada función más pequeña es convexa.

Para ilustrar esta afirmación considere la función

$$\begin{aligned} f(x_1, x_2, x_3) &= 4x_1 - x_1^2 - (x_2 - x_3)^2 \\ &= [4x_1 - x_1^2] + [-(x_2 - x_3)^2], \end{aligned}$$

que es la suma de las dos funciones más pequeñas dadas en los corchetes. La primera función más pequeña $4x_1 - x_1^2$ es una función sólo de la variable x_1 , por lo que puede verse que es cóncava si se observa que su segunda derivada es negativa. La segunda función más pequeña $-(x_2 - x_3)^2$ es una función de x_2 y x_3 , por lo que se puede aplicar la prueba para funciones de dos variables dada en el apéndice 2. En realidad, este apéndice usa esta función en particular para ilustrar la prueba y encuentra que la función es cóncava. Como las dos funciones más pequeñas son cóncavas, la función completa $f(x_1, x_2, x_3)$ también debe serlo.

Si un problema de programación no lineal no tiene restricciones, el hecho de que la función objetivo sea *cóncava* garantiza que un máximo local es un *máximo global*. (De igual manera, una función objetivo *convexa* asegura que un mínimo local es un *mínimo global*.) Si existen restricciones, se necesita una condición más para dar esta garantía, a saber, que la *región factible* sea un *conjunto convexo*. Por esta razón, los conjuntos convexos tienen un papel fundamental en la programación no lineal.

Como se analiza en el apéndice 2, un **conjunto convexo** es simplemente un conjunto de puntos tales que, para cada par de puntos de la colección, el segmento de recta que los une está totalmente contenido en ella. Así, la región factible en el problema original de la Wyndor Glass Co. (vea la figura 12.6 o 12.7) es un conjunto convexo. En realidad, la región factible de *cualquier* otro problema de programación lineal es un conjunto convexo. De igual manera, la región factible de la figura 12.5 también es un conjunto convexo.

En general, la región factible de un problema de programación no lineal es un conjunto convexo siempre que todas las funciones $g_i(\mathbf{x})$ [para las restricciones $g_i(\mathbf{x}) \leq b_i$] sean convexas. En el ejemplo de la figura 12.5, las dos $g_i(\mathbf{x})$ son convexas, puesto que $g_1(\mathbf{x}) = x_1$ (una función lineal es automáticamente cóncava y convexa) y $g_2(\mathbf{x}) = 9x_1^2 + 5x_2^2$ (tanto $9x_1^2$ como $5x_2^2$ son funciones con-

vexas, por lo que su suma es una función convexa). Estas dos funciones convexas $g_i(\mathbf{x})$ conducen a que la región factible de la figura 12.5 sea un conjunto convexo.

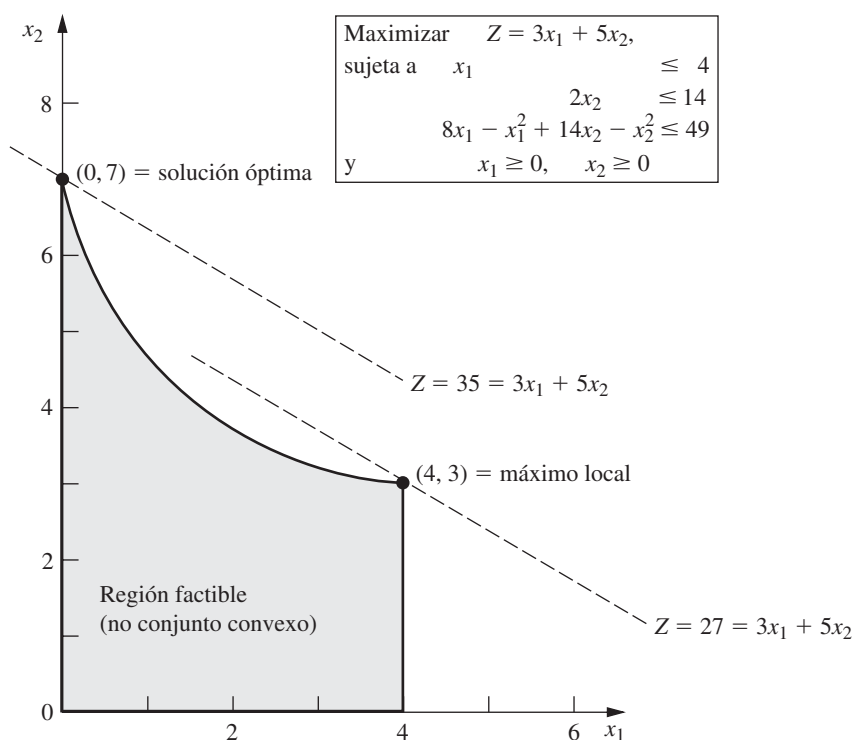
Ahora permítanos mostrarle qué sucede cuando sólo una de estas funciones $g_i(\mathbf{x})$ es una función cóncava. En particular, suponga que los únicos cambios que se introducen al ejemplo original de la Wyndor Glass Co., son que sus restricciones funcionales segunda y tercera se sustituyen por $2x_2 \leq 14$ y $8x_1 - x_1^2 + 14x_2 - x_2^2 \leq 49$. Por lo tanto, la nueva $g_3(\mathbf{x}) = 8x_1 - x_1^2 + 14x_2 - x_2^2$, que es una función cóncava ya que tanto $8x_1 - x_1^2$ como $14x_2 - x_2^2$ son funciones cóncavas. La nueva región factible que se muestra en la figura 12.10 *no* es un conjunto convexo. ¿Por qué? Porque contiene pares de puntos, como (0, 7) y (4, 3), tales que parte del segmento de recta que los une no está en la región factible. En consecuencia, no se puede garantizar que un máximo local sea un máximo global. En realidad, este ejemplo tiene dos máximos locales (0, 7) y (4, 3), pero sólo (0, 7) es un máximo global.

En consecuencia, para garantizar que un máximo local sea un máximo global en un problema de programación no lineal con restricciones $g_i(\mathbf{x}) \leq b_i$ ($i = 1, 2, \dots, m$) y $\mathbf{x} \geq \mathbf{0}$, la función objetivo $f(\mathbf{x})$ debe ser *cóncava* y cada $g_i(\mathbf{x})$ debe ser *convexa*. Un problema de este tipo se llama *problema de programación convexa* y es una de las clases más importantes de la programación no lineal que se estudiará en la sección 12.3.

12.3 TIPOS DE PROBLEMAS DE PROGRAMACIÓN NO LINEAL

Los problemas de programación no lineal se presentan de muchas formas distintas. Al contrario del método simplex para programación lineal, no se dispone de un algoritmo que resuelva todos estos tipos especiales de problemas. En su lugar, se han desarrollado algoritmos para algunas *clases* (tipos especiales) de problemas de programación no lineal. En esta sección se introducirán brevemente las clases más importantes. Después, en las secciones subsecuentes, se describirá cómo se pueden resolver algunos de estos problemas. Para simplificar la explicación, se supondrá a partir de ahora que los problemas se han formulado (o reformulado) en la forma general que se presentó al principio del capítulo.

FIGURA 12.10
Ejemplo de la Wyndor Glass Co., donde $2x_2 \leq 14$ y una restricción no lineal, $8x_1 - x_1^2 + 14x_2 - x_2^2 \leq 49$, reemplazan la segunda y tercera restricciones funcionales originales.



Optimización no restringida

Los problemas de optimización no restringida *no* tienen restricciones, por lo que la función objetivo es, sencillamente,

$$\text{Maximizar } f(\mathbf{x})$$

sobre *todos* los valores de $\mathbf{x} = (x_1, x_2, \dots, x_n)$. Según el repaso del apéndice 3, la condición *necesaria* para que una solución específica $\mathbf{x} = \mathbf{x}^*$ sea óptima cuando $f(\mathbf{x})$ es una función diferenciable es

$$\frac{\partial f}{\partial x_j} = 0 \quad \text{en } \mathbf{x} = \mathbf{x}^*, \text{ para } j = 1, 2, \dots, n.$$

Cuando $f(\mathbf{x})$ es *cóncava*, esta condición también es *suficiente*, por lo que la obtención de $\mathbf{x}^*$ se reduce a resolver el sistema de las n ecuaciones que se obtuvieron al establecer las n derivadas parciales iguales a cero. Desafortunadamente, cuando se trata de funciones *no lineales* $f(\mathbf{x})$, estas ecuaciones también suelen ser *no lineales*, en cuyo caso es poco probable que se pueda obtener una solución analítica simultánea. ¿Qué se puede hacer en ese caso? En las secciones 12.4 y 12.5 se describen *procedimientos algorítmicos de búsqueda* para encontrar $\mathbf{x}^*$, primero para $n = 1$ y luego para $n > 1$. Estos procedimientos también tienen un papel importante en la solución de varios tipos de problemas con restricciones, que se describirán en seguida. La razón es que muchos algoritmos para problemas *restringidos* están contruidos de forma que se adaptan a versiones *no restringidas* del problema en una parte de cada iteración.

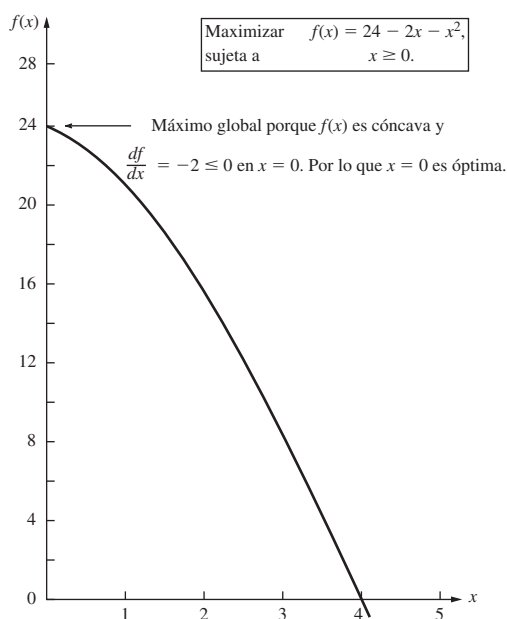
Cuando una variable x_j tiene una restricción de no negatividad, $x_j \geq 0$, la condición necesaria y —tal vez— suficiente anterior cambia ligeramente a

$$\frac{\partial f}{\partial x_j} \begin{cases} \leq 0 & \text{para } \mathbf{x} = \mathbf{x}^*, & \text{si } x_j^* = 0 \\ = 0 & \text{para } \mathbf{x} = \mathbf{x}^*, & \text{si } x_j^* > 0 \end{cases}$$

para cada j de este tipo. Esta condición se ilustra en la figura 12.11, donde la solución óptima de un problema con una sola variable es $x = 0$ aun cuando la derivada ahí es negativa y no cero. Como este ejemplo tiene una función cóncava para maximizar sujeta a una restricción de no negatividad, que su derivada sea menor o igual a 0 en $x = 0$, es una condición necesaria y suficiente para que $x = 0$ sea óptima.

■ FIGURA 12.11

Ejemplo que ilustra cómo una solución óptima puede estar en un punto en el que la derivada es negativa en lugar de cero, porque ese punto está en la frontera de una restricción de no negatividad.



Un problema que tiene algunas restricciones de no negatividad y que no tiene restricciones funcionales es un caso especial ($m = 0$) de la siguiente clase de problemas.

Optimización restringida linealmente

Los problemas de optimización restringida linealmente se caracterizan por restricciones que se ajustan por completo a la programación lineal, de manera que *todas* las funciones de restricción $g_i(\mathbf{x})$ son lineales, pero la función objetivo $f(\mathbf{x})$ es no lineal. El problema se simplifica de manera notable si sólo se tiene que tomar en cuenta una función no lineal junto con una región factible de programación lineal. Se han desarrollado varios algoritmos especiales basados en una *extensión* del método simplex para analizar la función objetivo no lineal.

Un caso especial importante que se describe a continuación es la programación cuadrática.

Programación cuadrática

De nuevo los problemas de *programación cuadrática* tienen restricciones lineales, pero ahora la función objetivo $f(\mathbf{x})$ debe ser *cuadrática*. Entonces, la única diferencia entre éstos y un problema de programación lineal es que algunos términos de la función objetivo incluyen el *cuadrado* de una variable o el *producto* de dos variables.

Se han desarrollado muchos algoritmos para manejar este caso, con el supuesto adicional de que $f(\mathbf{x})$ es *cóncava*. En la sección 12.7 se presenta un algoritmo que aplica una extensión del método simplex.

La programación cuadrática es muy importante, en parte porque las formulaciones de este tipo surgen de manera natural en muchas aplicaciones. Por ejemplo, el problema de la selección de una cartera con inversiones riesgosas, que se describió en la sección 12.1, se ajusta a este formato. Sin embargo, otra razón por la que es importante es que al resolver problemas generales de optimización restringida linealmente se puede obtener la solución de una sucesión de aproximaciones de programación cuadrática.

Programación convexa

La *programación convexa* abarca una amplia clase de problemas, entre los cuales, como casos especiales, se puede mencionar todos los tipos anteriores cuando $f(\mathbf{x})$ es una función cóncava que debe maximizarse. Los supuestos son:

1. $f(\mathbf{x})$ es cóncava.
2. Cada una de las $g_i(\mathbf{x})$ es convexa.

Como se expuso al final de la sección 12.2, estos supuestos son suficientes para asegurar que un máximo local es un máximo global. (Si, por el contrario, el objetivo fuera *minimizar* $f(\mathbf{x})$, sujeta ya sea a $g_i(\mathbf{x}) \leq b_i$ o a $-g_i(\mathbf{x}) \geq b_i$ para $i = 1, 2, \dots, m$, el primer supuesto cambiaría a que $f(\mathbf{x})$ debe ser una función *convexa*, puesto que es lo que se requiere para asegurar que un mínimo local sea un mínimo global.) En la sección 12.6 se verá que las condiciones necesarias y suficientes para obtener tal solución óptima son una generalización natural de las condiciones que se acaba de exponer para la *optimización no restringida* y su extensión a la inclusión de *restricciones de no negatividad*. En la sección 12.9 se describen los enfoques algorítmicos para resolver problemas de programación convexa.

Programación separable

La *programación separable* es un caso especial de programación convexa, en donde el supuesto adicional es:

3. Todas las funciones $f(\mathbf{x})$ y $g_i(\mathbf{x})$ son separables.

Una **función separable** es una función en la que *cada término* incluye *una sola variable*, por lo que la función se puede *separar* en una suma de funciones de variables individuales. Por ejemplo, si $f(\mathbf{x})$ es una función separable, se puede expresar como

$$f(\mathbf{x}) = \sum_{j=1}^n f_j(x_j),$$

donde cada $f_j(x_j)$ incluye sólo los términos con x_j . En la terminología de programación lineal (vea la sección 3.3), los problemas de programación separable satisfacen los supuestos de aditividad, pero violan el supuesto de proporcionalidad cuando cualquiera de las funciones $f_j(x_j)$ son funciones no lineales.

Para ilustrar, la función objetivo que se considera en la figura 12.6

$$f(x_1, x_2) = 126x_1 - 9x_1^2 + 182x_2 - 13x_2^2$$

es una función separable porque puede ser expresada como

$$f(x_1, x_2) = f_1(x_1) + f_2(x_2)$$

donde $f_1(x_1) = 126x_1 - 9x_1^2$ y $f_2(x_2) = 182x_2 - 13x_2^2$ son cada una funciones de una sola variable x_1 y x_2 , respectivamente. Si se usa el mismo razonamiento, se puede verificar que la función considerada en la figura 12.7 también es una función separable.

Es importante distinguir estos problemas de otros de programación convexa, pues cualquier problema de programación separable se puede aproximar muy de cerca mediante uno de programación lineal y, entonces, se puede aplicar el eficiente método símplex. Este enfoque se describe en la sección 12.8. (Para simplificar, se centra la atención en el caso *linealmente restringido*, en el que el tratamiento especial se necesita nada más para la función objetivo.)

Programación no convexa

La *programación no convexa* incluye todos los problemas de programación no lineal que no satisfacen los supuestos de programación convexa. En este caso, aun cuando se tenga éxito en encontrar un *máximo local*, no hay garantía de que sea también un *máximo global*. Por lo tanto, no se cuenta con un algoritmo que garantice encontrar una solución óptima para todos estos problemas; sin embargo, existen algunos algoritmos bastante adecuados para encontrar *máximos locales*, en especial cuando las formas de las funciones no lineales no se desvían demasiado de aquellas que se supuso para programación convexa. En la sección 12.10 se presenta uno de estos algoritmos. En esa misma sección se introducen dos nuevos optimizadores globales (ahora disponibles con LINGO y MPL) para encontrar una solución óptima para problemas de programación no convexa de tamaño moderado, así como un procedimiento de búsqueda que por lo general encontrará una solución cercana a la óptima para problemas aún más grandes.

Ciertos tipos específicos de problemas de programación no convexa se pueden resolver sin mucha dificultad mediante métodos especiales. Dos de ellos, de gran importancia, se presentarán más adelante.

Programación geométrica

Cuando se aplica programación no lineal a problemas de diseño de ingeniería, muchas veces la función objetivo y las funciones de restricción toman la forma

$$g(\mathbf{x}) = \sum_{i=1}^N c_i P_i(\mathbf{x}),$$

donde

$$P_i(\mathbf{x}) = x_1^{a_{i1}} x_2^{a_{i2}} \cdots x_n^{a_{in}}, \quad \text{para } i = 1, 2, \dots, N.$$

En tales casos, c_i y a_{ij} con frecuencia representan las constantes físicas, mientras que las x_j son las variables de diseño. Estas funciones por lo general no son ni cóncavas ni convexas, por lo que las técnicas de programación convexa no se pueden aplicar en forma directa a estos problemas de *programación geométrica*. Sin embargo, existe un caso importante en el que el problema se puede transformar en un problema de programación convexa equivalente. Este caso es aquel en el que *todos* los coeficientes c_i de cada función son estrictamente positivos, es decir, las funciones son *polinomios positivos generalizados* (ahora llamados **posinomios**), y la función objetivo se tiene

que minimizar. El problema equivalente de programación convexa con variables de decisión $y_1, y_2, \dots, y_n$ se obtiene al establecer

$$x_j = e^{y_j}, \quad \text{para } j = 1, 2, \dots, n$$

en todo el modelo original, de modo que ya se puede aplicar un algoritmo de programación convexa. Se ha desarrollado otro procedimiento de solución para resolver estos problemas de *programación posinomial*, al igual que para problemas de programación geométrica de otros tipos.

Programación fraccional

Suponga que la función objetivo se encuentra en la forma de una *fracción*, esto es, la razón o cociente de dos funciones,

$$\text{Maximizar} \quad f(\mathbf{x}) = \frac{f_1(\mathbf{x})}{f_2(\mathbf{x})}.$$

Estos problemas de *programación fraccional* surgen, por ejemplo, cuando se maximiza la razón de la producción entre las horas-hombre empleadas (productividad), o la ganancia entre el capital invertido (tasa de rendimiento), o el valor esperado dividido entre la desviación estándar de alguna medida de desempeño de una cartera de inversiones (rendimiento/riesgo). Se han formulado algunos procedimientos de solución especiales para ciertas formas de $f_1(\mathbf{x})$ y $f_2(\mathbf{x})$.

Cuando es posible, el enfoque más directo para resolver un problema de programación fraccional es transformarlo en un problema equivalente de algún tipo estándar que disponga de un procedimiento eficiente. Para ilustrar este enfoque, suponga que $f(\mathbf{x})$ es de la forma de *programación fraccional lineal*

$$f(\mathbf{x}) = \frac{\mathbf{c}\mathbf{x} + c_0}{\mathbf{d}\mathbf{x} + d_0},$$

donde $\mathbf{c}$ y $\mathbf{d}$ son vectores renglón, $\mathbf{x}$ es un vector columna y c_0 y d_0 son escalares. También suponga que las funciones de restricción $g_i(\mathbf{x})$ son lineales, es decir, las restricciones en forma matricial son $\mathbf{A}\mathbf{x} \leq \mathbf{b}$ y $\mathbf{x} \geq \mathbf{0}$.

Bajo algunos supuestos débiles adicionales, el problema se puede transformar en un problema equivalente de *programación lineal* si se establece

$$\mathbf{y} = \frac{\mathbf{x}}{\mathbf{d}\mathbf{x} + d_0} \quad \text{y} \quad t = \frac{1}{\mathbf{d}\mathbf{x} + d_0},$$

de manera que $\mathbf{x} = \mathbf{y}/t$. Este resultado conduce a

$$\text{Maximizar} \quad Z = \mathbf{c}\mathbf{y} + c_0t,$$

sujeta a

$$\mathbf{A}\mathbf{y} - \mathbf{b}t \leq \mathbf{0},$$

$$\mathbf{d}\mathbf{y} + d_0t = 1,$$

y

$$\mathbf{y} \geq \mathbf{0}, \quad t \geq 0,$$

que se puede resolver con el método símplex. En términos generales, se puede usar el mismo tipo de transformación para convertir un problema de programación fraccional con $f_1(\mathbf{x})$ cóncava, $f_2(\mathbf{x})$ convexa y $g_i(\mathbf{x})$ convexa, en un problema equivalente de programación convexa.

Problema de complementariedad

Cuando se estudie la programación cuadrática en la sección 12.7, se verá un ejemplo de cómo la solución de ciertos problemas de programación no lineal se puede reducir a resolver el problema de complementariedad. Dadas las variables $w_1, w_2, \dots, w_p$ y $z_1, z_2, \dots, z_p$, el **problema de complementariedad** encuentra una solución factible para el conjunto de restricciones

$$\mathbf{w} = F(\mathbf{z}), \quad \mathbf{w} \geq \mathbf{0}, \quad \mathbf{z} \geq \mathbf{0}$$

que también satisface la **restricción de complementariedad**,

$$\mathbf{w}^T \mathbf{z} = 0.$$

Aquí, $\mathbf{w}$ y $\mathbf{z}$ son vectores columna, F es una función dada con valores vectoriales y el superíndice T denota la transpuesta (vea el apéndice 4). El problema no tiene función objetivo, de manera que, desde un punto de vista técnico, no es un problema de programación no lineal completo. Se llama problema de complementariedad por las relaciones complementarias que establecen que

$$w_i = 0 \quad \text{o} \quad z_i = 0 \quad (\text{o ambas}) \quad \text{para cada } i = 1, 2, \dots, p.$$

Un caso en especial importante es el **problema de complementariedad lineal**, donde

$$F(\mathbf{z}) = \mathbf{q} + \mathbf{M}\mathbf{z},$$

donde $\mathbf{q}$ es un vector columna dado y $\mathbf{M}$ es una matriz dada de orden $p \times p$. Se dispone de algoritmos eficientes para resolver este problema bajo algunos supuestos adecuados⁶ sobre las propiedades de la matriz $\mathbf{M}$. Uno de éstos requiere pivotar de una solución básica factible (BF) a la siguiente, en forma muy parecida a la del método símplex para programación lineal.

Además de tener aplicaciones en programación no lineal, los problemas de complementariedad se utilizan en teoría de juegos, problemas de equilibrio económico y problemas de equilibrio en ingeniería.

12.4 OPTIMIZACIÓN NO RESTRINGIDA DE UNA VARIABLE

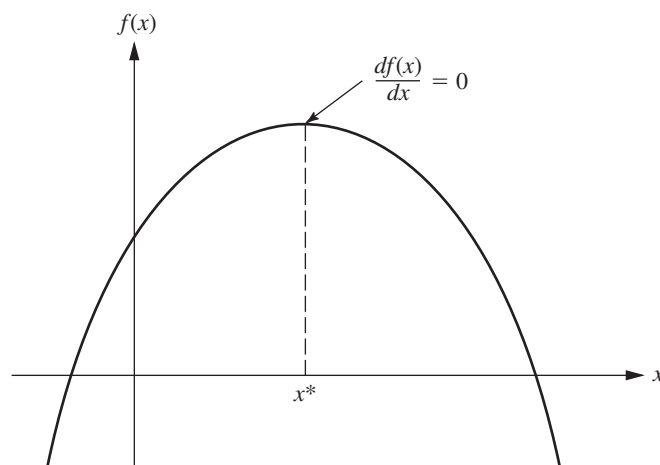
A continuación se explicará cómo se pueden resolver algunos tipos de problemas como los que se acababan de describir, comenzando por el caso más sencillo, la *optimización no restringida* con una sola variable x ($n = 1$), donde la función diferenciable $f(x)$ que debe maximizarse es *cóncava*.⁷ En consecuencia, la *condición necesaria y suficiente* para que una solución particular $x = x^*$ sea óptima (un máximo global) es

$$\frac{df}{dx} = 0 \quad \text{en } x = x^*,$$

como se ilustra en la figura 12.12. Si en esta ecuación se puede despejar x^* de modo directo, el problema llega a su fin; pero si $f(x)$ no es una función sencilla y su derivada no es una función lineal o cuadrática, tal vez sea imposible resolver la ecuación *analítica*. De ser así, existe una cantidad de *procedimientos de búsqueda* para resolver el problema en forma *numérica*.

■ FIGURA 12.12

Problema de programación no restringida de una variable cuando la función es cóncava.



⁶ Vea R. W. Cottle, J.-S. Pang y R. E. Stone, *The Linear Complementarity Problem*, Academic Press, Boston, 1992.

⁷ Vea el comienzo del apéndice 3 donde podrá encontrar una revisión del caso correspondiente cuando $f(x)$ no es cóncava.

El enfoque con cualquiera de estos procedimientos de búsqueda implica encontrar una serie de *ensayos de solución* que conduzcan hacia una solución óptima. En cada iteración se comienza con la solución de prueba actual para llevar a cabo una búsqueda sistemática, que culmina con la identificación de una nueva solución de prueba *mejorada*. El procedimiento continúa hasta que la solución de prueba haya convergido hacia una solución óptima, en caso de que exista una.

A continuación se describirán dos procedimientos de búsqueda comunes. El primero —*método de bisección*— se escogió porque es un procedimiento intuitivo y directo. El segundo —*método de Newton*— se incluyó porque tiene un papel fundamental en la programación no lineal en general.

Método de bisección

Este procedimiento de búsqueda siempre se puede aplicar cuando $f(x)$ es cóncava —de forma que la segunda derivada sea negativa o cero para toda x — como se mostró en la figura 12.12. Este método puede usarse también para algunas otras funciones. En particular, si x^* denota la solución óptima, todo lo que se necesita⁸ es que

$$\begin{aligned}\frac{df(x)}{dx} &> 0 && \text{si } x < x^*, \\ \frac{df(x)}{dx} &= 0 && \text{si } x = x^*, \\ \frac{df(x)}{dx} &< 0 && \text{si } x > x^*.\end{aligned}$$

Estas condiciones se cumplen de manera automática si $f(x)$ es cóncava, pero también se pueden cumplir cuando la segunda derivada es positiva para algunos (pero no todos) los valores de x .

La idea que subyace al método de bisección es muy intuitiva, esto es, el hecho de que la pendiente (derivada) sea positiva o negativa en una solución de prueba indica de forma definitiva si la mejoría respectiva cae de inmediato a la derecha o a la izquierda. De esta forma, si la derivada evaluada en un valor particular de x es positiva, entonces x^* debe ser *mayor* que dicha x (vea la figura 12.12), de manera que ésta se convierte en una *cota inferior* para las soluciones de prueba que deben considerarse de este punto en adelante. De manera inversa, si la derivada es *negativa*, entonces x^* debe ser *menor* que esta x , por lo cual ésta se convertiría en una *cota superior*. Por tanto, después de que se han identificado ambos tipos de cotas, cada nueva solución de prueba seleccionada entre las cotas actuales proporciona un intervalo más estrecho de alguno de los dos tipos, con lo que reduce la búsqueda de ahí en adelante. Cuando se usa una regla razonable para seleccionar cada solución de prueba de esta forma, la *secuencia* resultante de soluciones de prueba debe *converger* a x^* . En la práctica, esto significa continuar la secuencia hasta que la distancia entre las cotas sea tan pequeña como para que la siguiente solución de prueba esté dentro de una *tolerancia de error* para x^* especificada con anterioridad.

Este proceso completo se resume a continuación, dada la notación

$$\begin{aligned}x' &= \text{solución de prueba actual,} \\ \underline{x} &= \text{cota inferior actual para } x^*, \\ \bar{x} &= \text{cota superior actual para } x^*, \\ \epsilon &= \text{tolerancia del error de } x^*.\end{aligned}$$

Aunque existen algunas reglas razonables para seleccionar cada nueva solución de prueba, la que se utiliza en el método de bisección es la regla del **punto medio** —llamada en forma tradicional *plan de búsqueda de Bolzano*—, la cual dice simplemente que se debe seleccionar el punto medio entre las dos cotas actuales.

⁸ Otra posibilidad es que la parte superior de $f(x)$ sea plana de manera que x sea óptima en algún intervalo $[a, b]$. En este caso, el procedimiento aún convergirá a una de estas soluciones óptimas tan pronto como la derivada sea positiva para $x < a$ y negativa para $x > b$.

Resumen del método de bisección

Paso inicial: Seleccione ϵ . Encuentre la $\underline{x}$ y $\bar{x}$ iniciales por inspección o mediante la ubicación de cualesquiera valores respectivos de x en los cuales la derivada sea positiva y negativa. Seleccione una solución de prueba inicial

$$x' = \frac{\underline{x} + \bar{x}}{2}.$$

Iteración:

1. Evalúe $\frac{df(x)}{dx}$ para $x = x'$.
2. Si $\frac{df(x)}{dx} \geq 0$, redefina $\underline{x} = x'$.
3. Si $\frac{df(x)}{dx} \leq 0$, redefina $\bar{x} = x'$.
4. Seleccione una nueva $x' = \frac{\underline{x} + \bar{x}}{2}$.

Regla de detención: Detenerse si $\bar{x} - \underline{x} \leq 2\epsilon$, de modo que la nueva x' deba estar dentro del ϵ para x^* . En caso contrario debe realizarse otra iteración.

Ahora se ilustrará el método de bisección mediante su aplicación al siguiente ejemplo.

Ejemplo. Suponga que la función que debe maximizarse es

$$f(x) = 12x - 3x^4 - 2x^6,$$

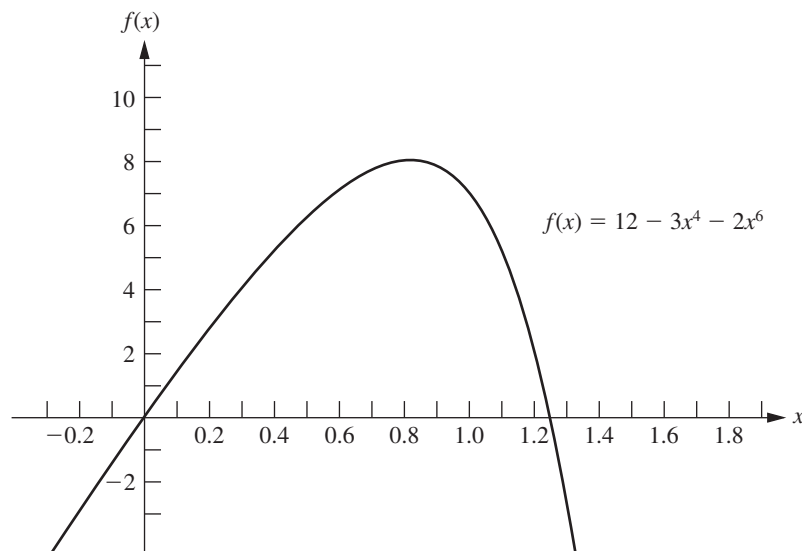
como se grafica en la figura 12.13. Sus primeras dos derivadas son

$$\frac{df(x)}{dx} = 12(1 - x^3 - x^5),$$

$$\frac{d^2f(x)}{dx^2} = -12(3x^2 + 5x^4).$$

Debido a que la segunda derivada es no positiva en todas partes, $f(x)$ es una función cóncava, por lo que el método de bisección puede aplicarse con seguridad para encontrar su máximo global (en el supuesto de que éste exista).

■ **FIGURA 12.13**
Ejemplo del método de bisección.



■ TABLA 12.1 Aplicación del método de bisección al ejemplo

Iteración	$\frac{df(x)}{dx}$	$\underline{x}$	$\bar{x}$	Nueva x'	$f(x')$
0		0	2	1	7.0000
1	-12	0	1	0.5	5.7812
2	+10.12	0.5	1	0.75	7.6948
3	+4.09	0.75	1	0.875	7.8439
4	-2.19	0.75	0.875	0.8125	7.8672
5	+1.31	0.8125	0.875	0.84375	7.8829
6	-0.34	0.8125	0.84375	0.828125	7.8815
7	+0.51	0.828125	0.84375	0.8359375	7.8839
Detener					

Una inspección rápida de esta función —sin siquiera construir su gráfica como se muestra en la figura 12.13— indica que $f(x)$ es positiva para valores positivos pequeños de x , pero es negativa para $x < 0$ o $x > 2$. Por lo tanto, $\underline{x} = 0$ y $\bar{x} = 2$ se pueden usar como las cotas iniciales, con su punto medio, $x' = 1$ como la solución de prueba inicial. Sea $\epsilon = 0.01$ la tolerancia al error para x^* en la regla de detención, entonces la $(\bar{x} - \underline{x})$ final es ≤ 0.02 con la x' final en el punto medio.

En consecuencia, la aplicación del método de bisección produce la secuencia de resultados que se muestra en la tabla 12.1. [Esta tabla incluye tanto los valores de la función como los de la derivada para su información, donde la derivada se evalúa en la solución de prueba generada en la iteración *precedente*. Sin embargo, observe que en realidad el algoritmo no necesita calcular $f(x')$ y que sólo se requiere calcular la derivada lo suficiente para determinar su signo.] La conclusión es que

$$x^* \approx 0.836,$$

$$0.828125 < x^* < 0.84375.$$

En el IOR Tutorial se incluye una rutina interactiva para ejecutar el método de bisección.

Método de Newton

Aunque el método de bisección es un procedimiento intuitivo y directo, tiene la desventaja de que converge de una manera relativamente lenta hacia una solución óptima. Cada iteración sólo disminuye la diferencia entre las cotas a la mitad. Por lo tanto, aun con la función tan simple que se consideró en la tabla 12.1, se requirieron siete iteraciones para reducir la tolerancia al error de x^* a menos de 0.01. Para reducir esta tolerancia al error a menos de 0.0001 se requerirían otras siete iteraciones.

La razón básica de esta convergencia lenta es que la única información que se ha usado sobre $f(x)$ es el valor de la primera derivada $f'(x)$ en los respectivos valores de prueba de x . Si se considerara también la segunda derivada $f''(x)$, se podría obtener información adicional útil. Este enfoque es el que aplica el *método de Newton*.⁹

La idea básica detrás del método de Newton es aproximar $f(x)$ a la vecindad de la solución de prueba inicial mediante una función cuadrática y después maximizar (o minimizar) la función aproximada exactamente para obtener la nueva solución de prueba y así iniciar la siguiente iteración. (Esta idea de trabajar con una **aproximación cuadrática** de la función objetivo se ha convertido en una característica clave de muchos algoritmos para tipos más generales de problemas de programación lineal.) Esta función cuadrática de aproximación se obtiene al truncar la serie de Taylor después del término de la segunda derivada. En particular, se considera que x_{i+1} es la so-

⁹ Este método se debe al gran matemático y físico del siglo xvii, Sir Isaac Newton. Cuando aún era un joven estudiante de la Universidad de Cambridge (Inglaterra), Newton obtuvo ventaja de que la universidad estuvo cerrada por dos años (debido a la peste bubónica que devastó Europa en 1664-1665) para descubrir la ley de la gravitación universal e inventar el cálculo (entre otros logros). Su desarrollo del cálculo condujo a este método.

lución de prueba generada en la iteración i para iniciar la iteración $i + 1$ —de manera que x_1 es la solución de prueba que proporciona el usuario para comenzar la iteración 1—, por lo que la serie de Taylor truncada para x_{i+1} es

$$f(x_{i+1}) \approx f(x_i) + f'(x_i)(x_{i+1} - x_i) + \frac{f''(x_i)}{2}(x_{i+1} - x_i)^2.$$

Al haber fijado x_i al inicio de la iteración i , observe que $f(x_i)$, $f'(x_i)$ y $f''(x_i)$ también son constantes fijas en esta función de aproximación a la derecha. Entonces, esta función de aproximación es sólo una función cuadrática de x_{i+1} . Aún más, esta función cuadrática es una aproximación tan buena de $f(x_{i+1})$ en la vecindad de x_i , que sus valores y los de su primera y segunda derivadas son exactamente los mismos cuando $x_{i+1} = x_i$.

Ahora esta función cuadrática se puede maximizar de la manera usual al igualar su primera derivada a cero y al resolver para x_{i+1} . (Recuerde que se supone que $f(x)$ es cóncava, lo cual implica que esta función cuadrática es cóncava, por lo cual cuando se iguala la primera derivada a cero la solución será un máximo global.) Esta primera derivada es

$$f'(x_{i+1}) \approx f'(x_i) + f''(x_i)(x_{i+1} - x_i)$$

como x_i , $f(x_i)$, $f'(x_i)$ y $f''(x_i)$ son constantes. Al igualar a cero la primera derivada de la derecha se obtiene

$$f'(x_{i+1}) + f''(x_i)(x_{i+1} - x_i) = 0,$$

lo cual directamente conduce de manera algebraica a la solución,

$$x_{i+1} = x_i - \frac{f'(x_i)}{f''(x_i)}.$$

Ésta es la fórmula clave que se utiliza en cada iteración i para calcular la siguiente solución de prueba x_{i+1} después de obtener la solución de prueba x_i para iniciar la iteración i y después calcular la primera y segunda derivadas en x_i . (Cuando se minimiza una función convexa se emplea la misma fórmula.)

Las iteraciones que generan nuevas soluciones de prueba de esta forma deben continuar hasta que éstas hayan convergido en esencia. Un criterio para sostener la convergencia es que $|x_{i+1} - x_i|$ se haya empequeñecido lo suficiente. Otro es que $f'(x)$ esté suficientemente cerca de cero. Y uno más es que $|f(x_{i+1}) - f(x_i)|$ sea suficientemente pequeño. Si se elige el primer criterio, se define ϵ como un valor tal que el algoritmo se detenga cuando $|x_{i+1} - x_i| \leq \epsilon$.

Aquí se presenta la descripción completa del algoritmo.

Resumen del método de Newton

Paso inicial: Seleccione ϵ . Encuentre una solución de prueba inicial x_i por inspección. Establezca $i = 1$.

Iteración i :

1. Calcule $f'(x_i)$ y $f''(x_i)$. [El cálculo de $f(x_i)$ es opcional.]
2. Establezca $x_{i+1} = x_i - \frac{f'(x_i)}{f''(x_i)}$.

Regla de detención: Si $|x_{i+1} - x_i| \leq \epsilon$, debe detenerse; en esencia x_{i+1} es la solución óptima. De otra manera, se redefine $i = i + 1$ y se realiza otra iteración.

■ **TABLA 12.2** Aplicación del método de Newton al ejemplo

Iteración i	x_i	$f(x_i)$	$f'(x_i)$	$f''(x_i)$	x_{i+1}
1	1	7	-12	-96	0.875
2	0.875	7.8439	-2.1940	-62.733	0.84003
3	0.84003	7.8838	-0.1325	-55.279	0.83763
4	0.83763	7.8839	-0.0006	-54.790	0.83762

Ejemplo. Ahora se aplicará el método de Newton al mismo ejemplo al que se aplicó el método de bisección. Como se mostró en la figura 12.13, la función que debe maximizarse es

$$f(x) = 12x - 3x^4 - 2x^6.$$

En consecuencia, la fórmula para calcular la nueva solución de prueba (x_{i+1}) a partir de la actual (x_i) es

$$x_{i+1} = x_i - \frac{f'(x_i)}{f''(x_i)} = x_i - \frac{12(1 - x^3 - x^5)}{-12(3x^2 + 5x^4)} = x_i + \frac{1 - x^3 - x^5}{3x^2 + 5x^4}.$$

Después de seleccionar $\epsilon = 0.00001$ y de elegir $x_1 = 1$ como la solución de prueba inicial, en la tabla 12.2 se muestran los resultados de aplicar el método de Newton a este ejemplo. Después de sólo cuatro iteraciones, este método ha convergido a $x = 0.83762$ como la solución óptima, con un grado muy alto de precisión.

Una comparación de esta tabla con la tabla 12.1 ilustra cuánto más rápido converge el método de Newton que el método de bisección. Si se aplica este último, se requerirían cerca de 20 iteraciones para converger con el mismo grado de precisión que logró el método de Newton después de sólo cuatro iteraciones.

Aunque esta convergencia rápida es muy usual en el método de Newton, su desempeño varía de problema a problema. Debido a que este enfoque se basa en el uso de una aproximación cuadrática de $f(x)$, su desempeño es afectado por el grado de precisión de la aproximación.

12.5 OPTIMIZACIÓN NO RESTRINGIDA DE VARIAS VARIABLES

Ahora considere el problema de maximizar una función *cóncava* $f(\mathbf{x})$ de variables *múltiples*, $\mathbf{x} = (x_1, x_2, \dots, x_n)$, en la que no existen restricciones sobre los valores factibles. Suponga de nuevo que la condición necesaria y suficiente para la optimalidad, dada por el sistema de ecuaciones que se obtiene al establecer las respectivas derivadas parciales iguales a cero (vea la sección 12.3), no se puede resolver en forma analítica, por lo que debe emplearse un procedimiento de búsqueda numérico.

Igual que en el caso de una variable, existen varios procedimientos de búsqueda para resolver este tipo de problema en forma numérica. Uno de éstos —*el procedimiento de búsqueda del gradiente*— es muy importante porque identifica y utiliza la dirección de movimiento, desde la solución de prueba actual, que maximiza la tasa a la cual se incrementa $f(\mathbf{x})$. Ésta es una de las ideas clave de la programación no lineal. Las adaptaciones de esta idea para tomar en cuenta las restricciones también son una característica central de muchos algoritmos que se utilizan para llevar a cabo una optimización *restringida*.

Después de analizar este procedimiento con algún nivel de detalle, se describirá en forma breve cómo se extiende el método de Newton al caso de las variables múltiples.

Procedimiento de búsqueda de gradiente

En la sección 12.4 se usó el valor de la derivada ordinaria mediante el método de bisección para elegir una de sólo dos direcciones posibles (aumentar x o disminuir x) para pasar de la solución de prueba actual a la siguiente. La meta era alcanzar, en algún momento, un punto en el que la derivada fuera (en esencia) cero. Ahora se tienen *innumerables* direcciones posibles hacia dónde moverse; corresponden a las *tasas proporcionales* posibles a las cuales las respectivas variables pueden cambiar. La meta es alcanzar, finalmente, un punto en el que todas las derivadas parciales sean —en esencia— 0. Por tanto, la extensión del procedimiento de búsqueda en una dimensión requiere emplear los valores de las derivadas parciales para seleccionar la dirección específica en la que conviene moverse. Esta selección implica el uso del gradiente de la función objetivo, como se describirá en seguida.

Como se supone que la función objetivo $f(\mathbf{x})$ es diferenciable, posee un gradiente denotado por $\nabla f(\mathbf{x})$ en cada punto $\mathbf{x}$. En particular, el **gradiente** en un punto específico $\mathbf{x} = \mathbf{x}'$ es el *vector* cuyos elementos son las *derivadas parciales* respectivas evaluadas en $\mathbf{x} = \mathbf{x}'$, por lo cual

$$\nabla f(\mathbf{x}') = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right) \quad \text{en } \mathbf{x} = \mathbf{x}'.$$

El significado del gradiente es que el cambio (infinitesimal) de $\mathbf{x}$, que *maximiza* la tasa a la que $f(\mathbf{x})$ aumenta, es el cambio que es *proporcional* a $\nabla f(\mathbf{x})$. Para expresar en forma geométrica esta idea, la “dirección” del gradiente, $\nabla f(\mathbf{x}')$, se interpreta como la *dirección* del segmento de recta dirigido (flecha) que va del origen $(0, 0, \dots, 0)$ al punto $(\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n)$, donde $\partial f / \partial x_j$ se evalúa en $x_j = x'_j$. En consecuencia, se puede decir que la tasa a la que aumenta $f(\mathbf{x})$ se maximiza si los cambios (infinitesimales) de $\mathbf{x}$ se hacen en la *dirección* del gradiente $\nabla f(\mathbf{x})$. Como el objetivo es encontrar la solución factible que maximice $f(\mathbf{x})$, parece adecuado intentar moverse lo más posible en la dirección del gradiente.

Debido a que el problema en cuestión no tiene restricciones, esta interpretación del gradiente sugiere que un procedimiento de búsqueda eficiente debe moverse en la dirección del gradiente hasta que (en esencia) alcance una solución óptima $\mathbf{x}^*$, en la que $\nabla f(\mathbf{x}^*) = \mathbf{0}$. Sin embargo, no resultaría práctico cambiar $\mathbf{x}$ de manera *continua* en la dirección de $\nabla f(\mathbf{x})$, puesto que esta serie de cambios requeriría una *reevaluación* continua de $\partial f / \partial x_j$ y el cambio de dirección de la trayectoria. Entonces, una mejor forma de hacerlo es continuar el movimiento en una dirección *fija* a partir de la solución de prueba actual, sin detenerse, hasta que $f(\mathbf{x})$ deje de crecer. Este punto de detención sería la siguiente solución de prueba, por lo que se debe volver a calcular el gradiente para determinar la nueva dirección de movimiento. Con este enfoque, cada iteración incluye cambiar la solución de prueba actual $\mathbf{x}'$ de la siguiente manera:

$$\text{Se modifica } \mathbf{x}' = \mathbf{x}' + t^* \nabla f(\mathbf{x}'),$$

donde t^* es el valor positivo de t que *maximiza* $f(\mathbf{x}' + t \nabla f(\mathbf{x}'))$; es decir,

$$f(\mathbf{x}' + t^* \nabla f(\mathbf{x}')) = \max_{t \geq 0} f(\mathbf{x}' + t \nabla f(\mathbf{x}')).$$

[Note que $f(\mathbf{x}' + t \nabla f(\mathbf{x}'))$ es sencillamente $f(\mathbf{x})$ donde

$$x_j = x'_j + t \left(\frac{\partial f}{\partial x_j} \right)_{\mathbf{x}=\mathbf{x}'}, \quad \text{para } j = 1, 2, \dots, n,$$

y estas expresiones de x_j incluyen sólo constantes y t , de manera que $f(\mathbf{x})$ se convierte en una función de una sola variable t .] Las iteraciones de este procedimiento de búsqueda del gradiente continúan hasta que $\nabla f(\mathbf{x}) = \mathbf{0}$ dentro de una pequeña tolerancia ϵ , o sea, hasta que

$$\left| \frac{\partial f}{\partial x_j} \right| \leq \epsilon \quad \text{para } j = 1, 2, \dots, n.^{10}$$

Una analogía puede ayudar a aclarar este procedimiento. Suponga que una persona quiere subir a la cumbre de una montaña. Esta persona es miope y no puede ver la cumbre para caminar en esa dirección, pero cuando se detiene, puede ver el piso a su alrededor y determinar la dirección en la que la pendiente de la montaña es más pronunciada. La persona puede caminar en línea recta. Mientras camina, también es capaz de percibir cuándo ya no va hacia arriba (pendiente cero en esa dirección). Bajo el supuesto de que la montaña es *cóncava*, el individuo puede usar el *procedimiento de búsqueda del gradiente* para escalar de manera eficiente hasta la cima. Este problema tiene *dos variables*, donde (x_1, x_2) representa las coordenadas (se hace caso omiso de la altura) de la localización actual. La función $f(x_1, x_2)$ da la altura de la montaña en (x_1, x_2) . La persona comienza cada iteración en el lugar en que se encuentra (solución de prueba actual) y determina la

¹⁰ En general, esta regla de detención proporcionará una solución $\mathbf{x}$ cercana a una solución óptima $\mathbf{x}^*$, con un valor de $f(\mathbf{x})$ muy cercano a $f(\mathbf{x}^*)$. Sin embargo, esto no se puede garantizar puesto que es posible que la función mantenga una pendiente positiva muy pequeña ($\leq \epsilon$) para una distancia grande entre $\mathbf{x}$ y $\mathbf{x}^*$.

dirección [en el sistema de coordenadas (x_1, x_2)] en la que la montaña tiene la mayor pendiente (la dirección del gradiente) en ese punto. Después, comienza a caminar en esa dirección fija y continúa haciéndolo mientras sigue ascendiendo. Luego se detiene en un nuevo punto de prueba (solución) cuando la montaña se nivela en la dirección en que camina; en este punto se prepara para realizar otra iteración en otra dirección. Estas iteraciones continúan, en una trayectoria en zigzag hacia arriba, hasta que alcanza un punto de prueba en el que la pendiente es en esencia cero en todas direcciones. Bajo el supuesto de que la montaña $[f(x_1, x_2)]$ es cóncava, en principio, la persona debe estar en la cima de la montaña.

Por lo general, la parte más difícil del procedimiento de búsqueda del gradiente es encontrar t^* , el valor de t que maximiza f en la dirección del gradiente, en cada iteración. Como $\mathbf{x}$ y $\nabla f(\mathbf{x})$ tienen valores fijos para la maximización y como $f(\mathbf{x})$ es cóncava, este problema se debe ver como el de maximizar una función *cóncava de una sola variable* t . En efecto, se puede resolver con el tipo de procedimientos de búsqueda de la optimización no restringida de una sola variable descritos en la sección 12.4 (donde la cota inferior inicial sobre t debe ser no negativa por la restricción de $t \geq 0$). De otra manera, si f es una función simple, es posible que se pueda obtener una solución analítica al establecer la derivada con respecto a t igual a cero para después resolver.

Resumen del procedimiento de búsqueda del gradiente

Paso inicial: Elija ϵ y cualquier solución de prueba inicial $\mathbf{x}'$. Vaya a la regla de detención.

Iteración:

1. Expresa $f(\mathbf{x}' + t \nabla f(\mathbf{x}'))$ como una función de t al establecer

$$x_j = x'_j + t \left(\frac{\partial f}{\partial x_j} \right)_{\mathbf{x}=\mathbf{x}'}, \quad \text{para } j = 1, 2, \dots, n,$$

y después sustituya estas expresiones en $f(\mathbf{x})$.

2. Utilice el procedimiento de búsqueda en una dimensión (o cálculo) para encontrar $t = t^*$ que maximiza $f(\mathbf{x}' + t \nabla f(\mathbf{x}'))$ para $t \geq 0$.
3. Establezca $\mathbf{x}'' = \mathbf{x}' + t^* \nabla f(\mathbf{x}')$. Después, pase a la regla de detención.

Regla de detención: Evalúe $\nabla f(\mathbf{x}'')$ en $\mathbf{x} = \mathbf{x}''$. Verifique si

$$\left| \frac{\partial f}{\partial x_j} \right| \leq \epsilon \quad \text{para toda } j = 1, 2, \dots, n.$$

Si es así, el proceso se detiene con la $\mathbf{x}''$ actual como la aproximación a una solución óptima $\mathbf{x}^*$ deseada. De otra manera, se realiza otra iteración.

Ahora se ilustrará este procedimiento.

Ejemplo. Considere el siguiente problema de dos variables:

$$\text{Maximizar} \quad f(\mathbf{x}) = 2x_1x_2 + 2x_2 - x_1^2 - 2x_2^2.$$

Entonces,

$$\begin{aligned} \frac{\partial f}{\partial x_1} &= 2x_2 - 2x_1, \\ \frac{\partial f}{\partial x_2} &= 2x_1 + 2 - 4x_2. \end{aligned}$$

También se puede verificar (vea el apéndice 2) que $f(\mathbf{x})$ es cóncava.

Para comenzar el procedimiento de búsqueda del gradiente ϵ , suponga que se elige $\mathbf{x} = (0, 0)$ como solución de prueba inicial. Como en este punto las respectivas derivadas parciales son 0 y 2, el gradiente es

$$\nabla f(0, 0) = (0, 2).$$

Con $\epsilon < 2$, la regla de detención indica la realización de una iteración.

Iteración 1: Con los valores de 0 y 2 para las derivadas parciales respectivas, la primera iteración inicia al establecer

$$\begin{aligned}x_1 &= 0 + t(0) = 0, \\x_2 &= 0 + t(2) = 2t,\end{aligned}$$

y después se sustituyen estas expresiones en $f(\mathbf{x})$ para obtener

$$\begin{aligned}f(\mathbf{x}' + t \nabla f(\mathbf{x}')) &= f(0, 2t) \\&= 2(0)(2t) + 2(2t) - 0^2 - 2(2t)^2 \\&= 4t - 8t^2.\end{aligned}$$

Debido a que

$$f(0, 2t^*) = \max_{t \geq 0} f(0, 2t) = \max_{t \geq 0} \{4t - 8t^2\}$$

y

$$\frac{d}{dt} (4t - 8t^2) = 4 - 16t = 0,$$

se deduce que

$$t^* = \frac{1}{4},$$

así,

$$\text{Se redefine } \mathbf{x}' = (0, 0) + \frac{1}{4}(0, 2) = \left(0, \frac{1}{2}\right).$$

Con esto termina la primera iteración. El gradiente de esta nueva solución de prueba es

$$\nabla f\left(0, \frac{1}{2}\right) = (1, 0).$$

Con $\epsilon < 1$, ahora la regla de detención indica que debe realizarse otra iteración.

Iteración 2: Para comenzar con la segunda iteración, se usan los valores de 1 y 0 para las derivadas parciales respectivas y se establece

$$\mathbf{x} = \left(0, \frac{1}{2}\right) + t(1, 0) = \left(t, \frac{1}{2}\right),$$

así,

$$\begin{aligned}f(\mathbf{x}' + t \nabla f(\mathbf{x}')) &= f\left(0 + t, \frac{1}{2} + 0t\right) = f\left(t, \frac{1}{2}\right) \\&= (2t)\left(\frac{1}{2}\right) + 2\left(\frac{1}{2}\right) - t^2 - 2\left(\frac{1}{2}\right)^2 \\&= t - t^2 + \frac{1}{2}.\end{aligned}$$

Debido a que

$$f\left(t^*, \frac{1}{2}\right) = \max_{t \geq 0} f\left(t, \frac{1}{2}\right) = \max_{t \geq 0} \left\{t - t^2 + \frac{1}{2}\right\}$$

y

$$\frac{d}{dt} \left(t - t^2 + \frac{1}{2}\right) = 1 - 2t = 0,$$

■ **TABLA 12.3** Aplicación del procedimiento de búsqueda del gradiente al ejemplo

Iteración	$\mathbf{x}'$	$\nabla f(\mathbf{x}')$	$\mathbf{x}' + t \nabla f(\mathbf{x}')$	$f(\mathbf{x}' + t \nabla f(\mathbf{x}'))$	t^*	$\mathbf{x}' + t^* \nabla f(\mathbf{x}')$
1	(0, 0)	(0, 2)	(0, 2t)	$4t - 8t^2$	$\frac{1}{4}$	$(0, \frac{1}{2})$
2	$(0, \frac{1}{2})$	(1, 0)	$(t, \frac{1}{2})$	$t - t^2 + \frac{1}{2}$	$\frac{1}{2}$	$(\frac{1}{2}, \frac{1}{2})$

entonces

$$t^* = \frac{1}{2},$$

por lo que

$$\text{Se redefine } \mathbf{x}' = \left(0, \frac{1}{2}\right) + \frac{1}{2}(1, 0) = \left(\frac{1}{2}, \frac{1}{2}\right).$$

Con esta igualdad se completa la segunda iteración. Con un valor de ϵ típicamente pequeño, el procedimiento ahora continuaría por algunas iteraciones más de una manera similar. (Se omitirán los detalles.)

Una manera sencilla de organizar este trabajo es escribir una tabla como la 12.3, que resume las dos iteraciones anteriores. En cada iteración, la segunda columna muestra la solución de prueba actual y la última muestra la nueva solución de prueba eventual, que después se escribe abajo, en la segunda columna, para la siguiente iteración. La cuarta columna proporciona las expresiones para x_j en términos de t , que se deben sustituir en $f(\mathbf{x})$ para dar la quinta columna.

Si se continúa de esta manera, las soluciones de prueba subsecuentes serán $(\frac{1}{2}, \frac{3}{4})$, $(\frac{3}{4}, \frac{3}{4})$, $(\frac{3}{4}, \frac{7}{8})$, $(\frac{7}{8}, \frac{7}{8})$, $\dots$, como se muestra en la figura 12.14. Dado que estos puntos convergen a $\mathbf{x}^* = (1, 1)$, ésta es la solución óptima, como se puede verificar por el hecho de que

$$\nabla f(1, 1) = (0, 0).$$

Sin embargo, como esta sucesión convergente de soluciones de prueba nunca alcanza su límite, en realidad el procedimiento se detendrá en algún punto (dependiendo de ϵ) un poco antes de $(1, 1)$ como aproximación final de $\mathbf{x}^*$.

En la figura 12.14 se sugiere que el procedimiento de búsqueda del gradiente marca una trayectoria en zigzag hacia la solución óptima en lugar de moverse en línea recta. Si se toma en cuenta este comportamiento, se pueden realizar algunas modificaciones que *aceleren* el movimiento hacia el óptimo.

Si $f(\mathbf{x})$ no fuera una función cóncava, de todas maneras este procedimiento convergiría a un máximo *local*. El único cambio en la descripción del procedimiento para este caso es que t^* ahora corresponde al *primer máximo local* de $f(\mathbf{x}' + t \nabla f(\mathbf{x}'))$ conforme t aumenta su valor desde 0.

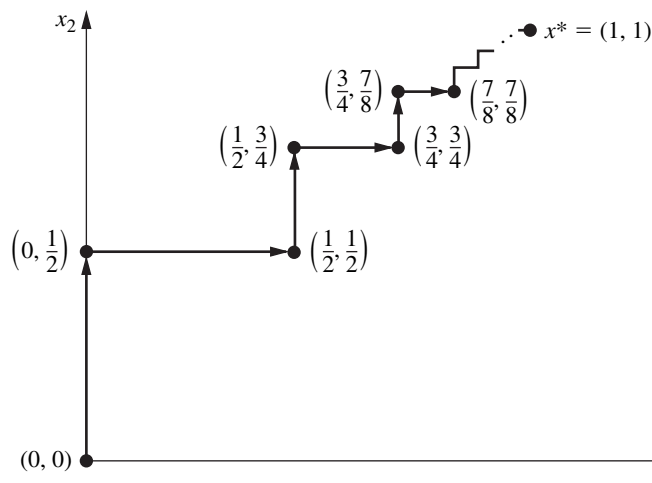
Si se quisiera *minimizar* la función objetivo $f(\mathbf{x})$, un cambio en el procedimiento sería moverse, en cada iteración, en la dirección *opuesta* al gradiente; en otras palabras, la regla para obtener el siguiente punto sería

$$\text{Redefinir } \mathbf{x}' = \mathbf{x}' - t^* \nabla f(\mathbf{x}').$$

El otro único cambio es que t^* ahora sería el valor negativo de la t que *minimiza* $f(\mathbf{x}' - t \nabla f(\mathbf{x}'))$; esto es,

$$f(\mathbf{x}' - t^* \nabla f(\mathbf{x}')) = \min_{t \geq 0} f(\mathbf{x}' - t \nabla f(\mathbf{x}')).$$

Tanto en la sección Worked Examples como en el OR Tutor del CD-ROM se incluyen **ejemplos adicionales** de la aplicación del procedimiento de búsqueda del gradiente. En el IOR Tutorial se incluye una rutina interactiva y una automática para aplicar este algoritmo.



■ FIGURA 12.14

Ilustración del procedimiento de búsqueda del gradiente cuando $f(x_1, x_2) = 2x_1x_2 + 2x_2 - x_1^2 - 2x_2^2$.

Método de Newton

En la sección 12.4 se describe cómo se puede usar el método de Newton para resolver problemas de optimización restringida con una *sola variable*. En realidad, la versión general del método de Newton está diseñada para resolver problemas de optimización restringida con *variables múltiples*. La idea básica es la misma que se describió en la sección 12.4, es decir, se trabaja con una aproximación cuadrática de la función objetivo $f(\mathbf{x})$, donde, en este caso, $\mathbf{x} = (x_1, x_2, \dots, x_n)$. Esta *función cuadrática de aproximación* se obtiene al truncar la serie de Taylor alrededor de la solución de prueba actual después del término de la segunda derivada. Después, esta función aproximada se maximiza (o minimiza) exactamente para obtener la nueva solución de prueba para iniciar la siguiente iteración.

Cuando la función objetivo es cóncava y tanto la solución de prueba actual $\mathbf{x}$ como su gradiente $\nabla f(\mathbf{x})$ se escriben como *vectores columna*, la solución $\mathbf{x}'$ que maximiza la función cuadrática de aproximación tiene la forma

$$\mathbf{x}' = \mathbf{x} - [\nabla^2 f(\mathbf{x})]^{-1} \nabla f(\mathbf{x}),$$

donde $\nabla^2 f(\mathbf{x})$ es la matriz de $n \times n$ (llamada *matriz hessiana*) de la segunda derivada parcial de $f(\mathbf{x})$ evaluada en la solución de prueba actual $\mathbf{x}$ y $[\nabla^2 f(\mathbf{x})]^{-1}$ es la inversa de esta matriz hessiana.

Los algoritmos de programación no lineal que emplean el método de Newton—incluso aquellos que lo adaptan para ayudar a tratar con problemas de optimización *restringida*—por lo general aproximan el inverso de la función hessiana de varias maneras. Estas aproximaciones del método de Newton son conocidas como **métodos cuasi-Newton** (o *métodos métricos variables*). Se comentará a profundidad el papel importante de estos métodos en programación no lineal en la sección 12.9.

Una descripción más extensa de estos métodos está fuera de los alcances del libro, pero se pueden encontrar mayores detalles en textos dedicados a la programación no lineal.

12.6 CONDICIONES DE KARUSH-KUHN-TUCKER (KKT) PARA OPTIMIZACIÓN RESTRINGIDA

La pregunta ahora es cómo reconocer una *solución óptima* para un problema de programación no lineal (con funciones diferenciables). ¿Cuáles son las condiciones necesarias y (tal vez) suficientes que esa solución debe cumplir?

En las secciones anteriores se hicieron notar estas condiciones para *optimización no restringida*, como se resume en los primeros dos renglones de la tabla 12.4. Al principio de la sección 12.3 también se expusieron estas condiciones para la ligera *extensión* de optimización no restringida cuando *sólo* se tienen restricciones de no negatividad. En el tercer renglón de la tabla 12.4 se